

БОРИСЛАВ ПАНАЙОТОВ СТОЯНОВ

РУКОВОДСТВО ЗА УПРАЖНЕНИЯ
ПО ПРОГРАМИРАНЕ НА
АСЕМБЛЕР

ПЪРВА ЧАСТ

Научен редактор:

доц. д-р Петър Ангелов Милев

Рецензенти:

доц. д-р Янко Костадинов Янков

гл. ас. д-р Найденов Вълков Ненков

© Борислав Панайотов Стоянов – автор, 2008

© Химера ООД, гр. Шумен, 2008

ISBN 978-954-9775-37-2

2011

Кратко съдържание

Списък на фигурите.....	9
Списък на таблиците.....	10
Увод.....	11
1 Бройни системи.....	15
2 Микропроцесорна фамилия 80x86	33
3 Микропроцесорна архитектура 80x86– програмна гледна точка	44
4 MS-DOS	63
5 Създаване на работеща програма на асемблер.....	78
6 Основи на програмирането в MS-DOS.....	110
7 Задачи	135
8 Основни инструкции на Асемблер.....	150
Литература.....	168

Съдържание

Списък на фигурите.....	9
Списък на таблиците.....	10
Увод.....	11
1 Бройни системи.....	15
1.1 Основни определения.....	15
1.2 Десетична бройна система.....	16
1.3 Преобразуване от р-ична в десетична и от десетична в р-ична бройна система.....	16
1.4 Двоична бройна система.....	17
1.4.1 Групиране на битовете.....	18
1.5 Шестнадесетична бройна система.....	21
1.6 Числа в допълнителен код.....	22
1.7 Двоично събиране и двоично допълнение.....	22
1.8 Допълнителен код на двоичните числа.....	23
1.9 Двоично изваждане.....	24
1.10 ASCII/IBM система от знаци.....	25
1.10.1 ASCII система от знаци.....	25
1.10.2 IBM система от знаци.....	27
1.11 Дефиниране на променливи и области от паметта.....	27
1.11.1 Директиви за дефиниране на данни.....	27
1.11.2 Директиви за дефиниране на променливи.....	28
1.11.3 Директива за еквивалентност.....	30
1.12 Задачи.....	31
2 Микропроцесорна фамилия 80x86.....	33
2.1 Основни части.....	33
2.1.1 Системна шина.....	34
2.1.2 Памет.....	35
2.1.3 Вход/Изход.....	36
2.2 Системно тактуване.....	36
2.2.1 Системен таймер.....	37
2.2.2 Достъп до паметта и системния таймер.....	38
2.2.3 Състояние на изчакване.....	38
2.2.4 Кеш памет.....	38
2.3 Фамилия процесори 80x86.....	40
2.3.1 Регистри.....	41
2.3.2 Аритметико и логическо устройство.....	42
2.3.3 Шинно интерфейсно устройство.....	42
2.3.4 Управляващо устройство.....	42
2.3.5 Изпълнение на инструкции.....	43
2.6 Задачи.....	43
3 Микропроцесорна архитектура 80x86– програмна гледна точка	44

3.1 Регистри в процесорите 8086.....	44
3.1.1 Сегментни регистри.....	44
3.1.2 Регистри с общо предназначение.....	44
3.1.3 Специални регистри.....	46
3.2 Регистри в процесорите 80286.....	47
3.3 Регистри в процесорите 80386.....	48
3.4 Регистри в процесорите 80486/Pentium.....	49
3.5 Сегменти. Принцип на сегментацията.....	49
3.6 Стек.....	51
3.7 ROM-BIOS и DOS.....	51
3.7.1 ROM-BIOS.....	51
3.7.2 DOS интерфейс.....	52
3.7.3 Достъп до хардуера.....	52
3.8 Прекъсвания.....	52
3.9 Видове адресиране.....	56
3.9.1 Регистрово адресиране.....	56
3.9.2 Непосредствено адресиране.....	56
3.9.3 Пряко адресиране.....	56
3.9.4 Косвено адресиране.....	58
3.10 Въпроси.....	61
4 MS-DOS.....	63
4.1 Команден процесор.....	63
4.2 Файлова структура.....	66
4.2.1 Имена на устройства.....	66
4.2.2 Файлове.....	67
4.2.3 Папки.....	67
4.2.4 Файлова организация.....	67
4.3 Често използвани команди на MS-DOS.....	68
4.3.1 TREE.....	68
4.3.2 DIR.....	69
4.3.3 TYPE.....	70
4.3.4 MORE.....	70
4.3.5 MKDIR (MD).....	71
4.3.6 RMDIR (RD).....	72
4.3.7 CHDIR (CD).....	72
4.3.8 PATH.....	73
4.3.9 COPY.....	75
4.3.10 RENAME (REN).....	75
4.3.11 DEL (ERASE).....	76
4.4 Въпроси.....	77
5 Създаване на работеща програма на асемблер.....	78
5.1 Синтаксис на инструкциите на асемблер.....	78
5.1.1 Идентификатори.....	78

5.1.2 Оператори	78
5.1.3 Директиви	80
5.1.4 Низови константи	81
5.2 Структура на асемблерската програма	81
5.2.1 Сегментационни директиви	82
5.2.2 Процедури	89
5.2.3 Макро директиви	90
5.2.4 EXE и COM програми	91
5.2.5 Структура на командите, типове, основен цикъл.....	97
5.3 Получаване на изпълнима програма	98
5.3.1 Инсталиране на асемблер	98
5.3.2 Набиране на текста на програма на асемблер.....	99
5.3.3 Асемблиране на първичен файл.....	99
5.3.4 Свързване на обектните модули	104
5.3.5 Стартиране на изпълним файл	106
5.3.6 Средства за тестване и настройка	106
6 Основи на програмирането в MS-DOS.....	110
6.1 Изход на екрана в текстов режим	110
6.1.1 Средства на DOS	110
6.1.2 Средства на BIOS	112
6.1.3 Пряк достъп до видео паметта	115
6.2 Вход от клавиатурата	118
6.2.1 Средства на DOS	118
6.2.2 Средства на BIOS	123
6.3 Извеждане на екрана на числа	131
6.3.1 Извеждане на числа в десетична бройна система.....	131
6.3.2 Дъмп	132
6.4 Задачи.....	134
7 Задачи	135
7.1 Решени примерни задачи	135
7.2 Задачи за самостоятелна работа	145
8 Основни инструкции на Асемблер.....	150
Означения:	150
ADD.....	150
CMP	151
DEC	152
DIV	152
INC	153
INT.....	153
INTO.....	153
IRET	154
JMP	154
Jxx.....	155

LODS/LODSB/LODSW/LODSD	159
LOOP/LOOPxx	160
MOV	161
POP	162
POPA/POPAD	163
POPF/POPFD	163
PUSH.....	164
PUSHA/PUSHAD	164
PUSHF/PUSHFD	165
REP/REPE/REPZ/REPNE/REPZ	165
RET.....	166
STOS/STOSB/STOSW/STOSD.....	167
XOR.....	167
Литература.....	168

Списък на фигурите

Фиг.1.1	Брое на битовите в байт	18
Фиг.1.2	Тетради в байт	19
Фиг.1.3	Брое на битовите в дума.....	19
Фиг.1.4	Байтове в дума	19
Фиг.1.5	Тетради в дума.....	19
Фиг.1.6	Брое на битовите в двойна дума.....	19
Фиг.1.7	Думи, байтове и тетради в двойна дума.....	20
Фиг.1.8	Пример за двоично събиране	23
Фиг.1.9	8 битов допълнителен код на числото 92.....	24
Фиг.1.10	8 битов допълнителен код на числото -92.....	24
Фиг.1.11	Сума на 8 битовите числа 92 и -92.....	24
Фиг.1.12	Пример за 8 битово изваждане	25
Фиг.1.13	Пример за ASCII кодовете на букви от английската азбука..	26
Фиг.2.1	Фон Нойманова архитектура.....	33
Фиг.2.2	Байт, дума и двойна дума на различни адреси	36
Фиг.2.3	Две нива на кеш паметта	39
Фиг.2.4	Блокова схема на фамилия процесори 80x86.....	41
Фиг.3.1	Регистри за данни в 8086	45
Фиг.3.2	Флагов регистър	46
Фиг.3.3	Регистри в 80386.....	48
Фиг.3.4	Изчисляване на адрес (сегмент + отместване) в сегмент.....	49
Фиг.3.5	Получаване на 20 битов физически адрес.....	50
Фиг.4.1	Меню Run.....	63
Фиг.4.2	Прозорец Run, стартиране на команден процесор cmd.....	64
Фиг.4.3	Работа на командния процесор cmd	64
Фиг.4.4	Прозорец Run, стартиране на команден процесор command...	65
Фиг.4.5	Работа на командния процесор command.....	65
Фиг.4.6	Грешка при извикване на програма tasm	73
Фиг.4.7	Преглеждане на допълнителните пътища с PATH.....	74
Фиг.4.8	Изтриване на допълнителните пътища с PATH	74
Фиг.5.1	Основен прозорец на програмата Turbo Debugger	107
Фиг.5.2	Текст Hello World в програмата Turbo Debugger.....	108
Фиг.6.1	Работа на програмата debug	133
Фиг.8.1	Посока на прехвърляне на данни	161

Списък на таблиците

Табл.1.1	Двоични, осмични, десетични и шестнадесетични цифри.....	15
Табл.1.2	Двоично представяне на 43_{10}	17
Табл.1.3	Двоично представяне на десетичните числа до 15.....	18
Табл.1.4	Списък с метрични префикси за количество информация.....	20
Табл.1.5	Преобразуване на шестнадесетично в съответното му двоично число	21
Табл.1.6	Интервали на числата в допълнителен код в различни групи битове.....	22
Табл.1.7	Двоично събиране	22
Табл.1.8	Операция NOT.....	23
Табл.1.9	ASCII управляващи знаци	26
Табл.1.10	Директиви за определяне на данни с различни размери	28
Табл.2.1	Размер на шината за данни при процесорна фамилия 80x86..	34
Табл.2.2	Размер на адресната шина при процесорна фамилия 80x86 ..	35
Табл.3.1	Хардуерни прекъсвания	52
Табл.3.2	Прекъсвания предизвикани от ЦП	53
Табл.3.3	Прекъсвания на BIOS	53
Табл.3.4	Често използвани функции на прекъсване 21h на DOS	53
Табл.3.5	Тип на оператор ptr	59
Табл.5.1	Модел на паметта.....	85
Табл.5.2	Опростени сегментационни директиви.....	86
Табл.5.3	Модификатори на модела на паметта	87
Табл.5.4	Идентификатори, създадени от директива model и опростените сегментационни директиви	88
Табл.5.5	Прехвърляне на данни	98
Табл.6.1	Често използвани функции на прекъсване 10h на BIOS	112
Табл.6.2	Видове цветове в текстов режим	114
Табл.6.3	Функции на 16h прекъсване на BIOS за работа с клавиатурата	124
Табл.8.1	Изпълнение на инструкция DIV	152
Табл.8.2	Видове инструкции за условен преход Jxx	156
Табл.8.3	Видове префикси за повторение.....	166

Увод

Настоящото учебно пособие е първа част от цикъл учебни пособия предназначени за осигуряване на модул „Програмиране на асемблер” от упражненията по учебната дисциплина Компютърни архитектури за студентите от бакалавърските специалности Компютърна информатика, Компютърни и информационни технологии и Икономическа информатика към Шуменския университет „Епископ Константин Преславски”, гр. Шумен.

ЗАЩО Е НЕЖНО ДА СЕ ИЗУЧАВА АСЕМБЛЕР?

Защото асемблерът е езика на процесора и той ще бъде нужен дотогава, докато съществуват процесорите. По-пространственият отговор на този въпрос съдържа в себе си разсъждения за това, че асемблерът може да се използва за оптимизация кода на програма, написване на софтуерни драйвери, транслятори, компилатори и интерпретатори, програмиране на външни устройства.

Асемблерът е универсален език за програмиране, чрез който машинния език на дадена процесорна архитектура се представя в удобен за четене формат. Машинният език представя машинните операции в двоичен еквивалент, т.е. всяка машинна операция се кодира със специфична комбинация от битове. За да бъде четим машинният код, тези комбинации от битове се заменят със символи, наречени мнемоничен код. Например машинната инструкция

```
10110000 01100001
```

се заменя на асемблер с

```
mov al, 61h
```

За разлика от компютърните езици от високо ниво, тук има еднозначно съответствие между представянето на машинната инструкция на асемблер и на машинен език. Асемблерният език се преобразува в машинен код чрез асемблиране, а обратно – чрез дизасемблиране. Всяка процесорна архитектура има собствен машинен език, следователно и собствен асемблер. Процесорите се различават по броя и типа на инструкциите, които поддържат. Всеки процесор може да има различен брой регистри с различна дължина.

За структурирането на учебния материал по теми и раздели е използван основно педагогическият подход и материали от издадените книги по програмиране на асемблер на Д. Бадли, П. Абел и А. Купник.

Материалът обсъждан в учебното пособие е свързан с асемблера Turbo Assembler (TASM), линкера Turbo Link (TLINK) Version 5.1 Copyright © 1992 Borland International и дебъгера Turbo Debugger (TD) Version 3.1 Copyright © 1988, 1992 Borland International.

В учебника са разяснени особеностите на различните бройни системи, начините за преобразуване на числа между тях. Обърнато е внимание на числата в допълнителен код. Посочени са начините за дефиниране на данни и променливи в паметта.

Приведени са основните хардуерни особености на микропроцесорната фамилия 80x86. Разгледано е подробно системното тактуване.

Микропроцесорната архитектура 80x86 е разгледана и от програмна гледна точка. Разяснени са различните видове регистри и областите на тяхното използване. Подробно е обяснен принципа на сегментацията даващ възможност за преместваемост на програмите и данните в паметта.

Изведени са основните характеристики на софтуерните и хардуерните прекъсвания и стъпките за тяхното изпълнение. Въведени са основните характеристики, подлатени с множество примери, на регистровото, непосредственото, прякото и косвено адресиране.

Направено е кратко въведение в операционната система MS DOS, необходима среда за изпълнение на всички примери от учебника.

След това студентите се запознават със синтаксиса на асемблерните инструкции, структурата на асемблерните програми и сегментационни директиви. Подробно е разяснено асемблирането на първичния асемблерен код и последващото свързване на обектните модули.

Основните конструкции и асемблерни директиви се въвеждат чрез разглеждането на различните способности на DOS и BIOS за изход на екрана в текстов режим и вход от клавиатурата. Специално внимание е отделено на обработката и извеждането на екрана на числа.

Студентите се запознават последователно с различните асемблерни инструкции в процеса на изучаване на учебния материал.

Всички имена на продукти и услуги, използвани в настоящото учебно пособие представляват търговски марки, собственост на съответните фирми, и са използвани само за учебни примери.

2008

Авторът

1 Бройни системи

1.1 Основни определения

Бройната система е метод за символно представяне на числата, посредством ограничен брой символи, наречени цифри. Стойността на всяка цифра се определя от нейното място в записването на числото. Броят на различните цифри, използвани в една бройна система, се нарича **основа**. Основата приема стойности, които са по-големи или равни на две. Най-често срещаните бройни системи са двоичната, осмичната, десетичната и шестнадесетичната. Поради трудността от определянето на това в коя бройна система е записано дадено число в асемблер е възприето веднага след неговото записване то да бъде последвано от латинските букви **B, O (Q), D и H**, което определя еднозначно принадлежността му към горните бройни системи. В **Табл.1.1** са показани възможните цифри с които се означават числата в бройните системи.

Табл.1.1 Двоични, осмични, десетични и шестнадесетични цифри

Бройна система	Основа	Цифри
двоична	2	0 1
осмична	8	0 1 2 3 4 5 6 7
десетична	10	0 1 2 3 4 5 6 7 8 9
шестнадесетична	16	0 1 2 3 4 5 6 7 8 9 A B C D E F

Цифрите от A до F представят числата от 10 до 15 в шестнадесетична бройна система.

Задаването на числови константи в асемблер става по следния начин:

55h ; 55 в шестнадесетична бройна система
10011b ; 10011 в двоична бройна система
75 ; интерпретацията зависи от текущо
; използваната бройна система

Установяването на бройната система става с директивата **radix** в режим **ideal** (**.radix** в режим **masm**):

radix Основа

Основа може да приема стойности 2, 8, 10 и 16. Например:

radix 10
radix 16.

По подразбиране се използва десетичната бройна система.

1.2 Десетична бройна система

За изобразяване на числата с десетична бройна система се използват цифрите 0, 1, 2, 3, 4, 5, 6, 7, 8 и 9. Всяка цифра от числото, вляво от десетичната запетая, е свързана с положителна степен на числото 10 в зависимост от позицията която заема. Броенето започва от нулата и отлясно наляво. А всяка цифра вдясно от десетичната запетая е свързана с отрицателна степен на числото 10. В този случа броенето започва от единица и отляво надясно. Числото „345” се представя по следния начин:

$$345 = 3 \cdot 10^2 + 4 \cdot 10^1 + 5 \cdot 10^0 = 300 + 40 + 5.$$

Представянето на числото 345, 678 става по следния начин:

$$\begin{aligned} 3456,789 &= 3 \cdot 10^3 + 4 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0 + 7 \cdot 10^{-1} + 8 \cdot 10^{-2} + 9 \cdot 10^{-3} \\ &= 3000 + 400 + 50 + 6 + 0,7 + 0,08 + 0,009. \end{aligned}$$

1.3 Преобразуване от p-ична в десетична и от десетична в p-ична бройна система

Преобразуването на число a от p -ична в десетична бройна система става по следния начин: първо нека е дадено числото

$$a = (a_k, a_{k-1}, \dots, a_0)_p$$

в p -ична бройна система, където $p \geq 2$. След това търсеното число n_{10} се получава по следната формула:

$$a_k p^k + a_{k-1} p^{k-1} + \dots + a_0 p^0 = n_{10}.$$

Преобразуване на числото n_{10} към число a в p -ична бройна система се извършва по следния алгоритъм:

```
i := 0
while n > 0 do
    ai := n mod p, n := n div p, i := i + 1
    print ai
```

1.4 Двоична бройна система

Двоичната бройна система изобразява числата само с 0 и 1. Всяка една цифра от двоичното число се нарича бит – bit (*binary digit*). С всеки бит от двоично число, спрямо позицията в числото, е свързана степен на двойката. Степенуването започва отлясно наляво и от нулата. Например:

$$\begin{aligned} 101011_2 &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 8 + 2 + 1 \\ &= 43_{10} \end{aligned}$$

По този начин се извършва преобразуването от двоично към десетично число.

Обратното преобразуване, от десетично към двоично представяне, изисква повече изчисления. Извършва се многократно деление на две с остатък, до получаване на частно равно на нула. Получените остатъци в обратен ред формират търсеното двоично представяне. В следващия пример (**Табл.1.2**) е направено постъпково получаване на битовете от двоичното представяне на 43_{10} .

Табл.1.2 Двоично представяне на 43_{10}

Деление	Частно	Остатък
43 / 2	21	1
21 / 2	10	1
10 / 2	5	0
5 / 2	2	1
2 / 2	1	0
1 / 2	0	1

Получените остатъци, взети в обратен ред, образуват двоичното число 1010112.

1.4.1 Групиране на битовете

Битът е най-малката единица информация използвана при двоичната аритметика. Чрез него могат да се обозначат всеки две различни стойности. Най-левият бит, във всяка група битове, се нарича старши бит, а най-десният – младши бит.

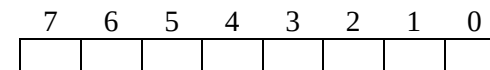
Групирайки битовете в редици по четири се получава т.н. **тетрада**. В **Табл.1.3** са показани двоичните представяния на десетичните числа от нула до 15.

Табл.1.3 Двоично представяне на десетичните числа до 15

Десетични числа	Двоични числа	Десетични числа	Двоични числа
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

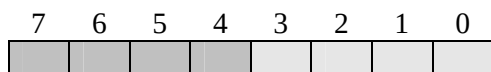
Забелязва се, че 15 е най-голямото двоично число, което може да се представи с една тетрада.

Най-важната даннова структура, използвана в съвременните компютри е байтът (*byte*). Той е поредица от осем бита. Битовете в байта се номерират от нула до седем, **Фиг.1.1**.



Фиг.1.1 Броене на битовете в байт

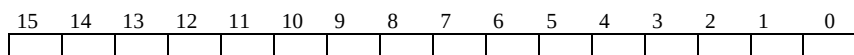
Битовете с номера от нула до три образуват младша тетрада, а с номера от четири до седем старша тетрада, **Фиг. 1.2**.



Фиг.1.2 Тетради в байт

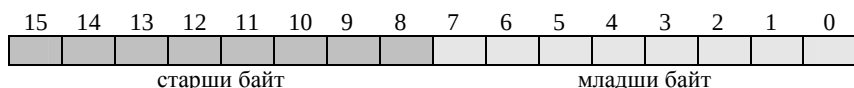
Поради това, че байтът съдържа осем бита той може да представи 2^8 , или 256 различни стойности.

Групата от 16 бита се нарича дума (word). Битовите в думата се номерират от нула до 15, **Фиг. 1.3**.



Фиг.1.3 Броене на битовите в дума

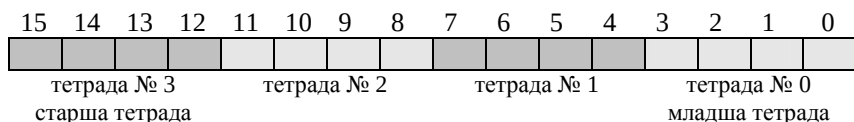
Всяка дума е съставена от два байта. Битовите от 0 до 7 са образуват младши байт, а от 8 до 15 старши байт, **Фиг. 1.4**.



Фиг.1.4 Байтове в дума

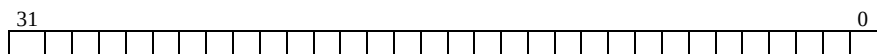
Представянето в паметта става по същият начин - младшият байт предхожда старшия байт. Нарича се *little endian* формат.

Думата може да бъде разделена и на четири тетради, както е показано на **Фиг.1.5**.



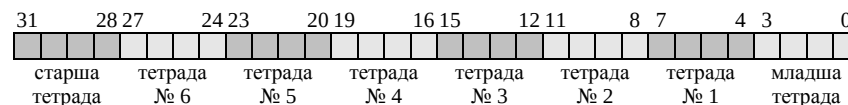
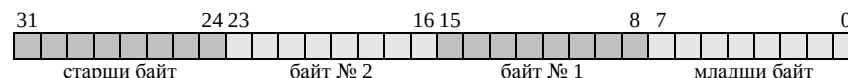
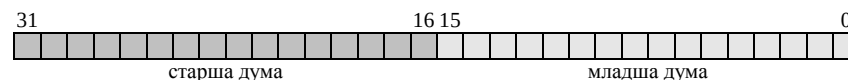
Фиг.1.5 Тетради в дума

С всичките 16 бита от думата могат да бъдат представени 216, или 65 536, различни стойности. Група от 32 бита се нарича двойната дума (double word), **Фиг.1.6**.



Фиг.1.6 Броене на битовите в двойна дума

Двойната дума е съставена от две думи, от четири байта или от осем тетради, **Фиг.1.7**.



Фиг.1.7 Думи, байтове и тетради в двойна дума

Двойната дума може да представи 2^{32} , или **4 294 967 296** различни стойности.

По същият начин се определят и понятията: четворна дума (*quad word*), състояща се от 4 думи = 8 байта = 16 тетради = 64 бита, параграф (*paragraph*) – изграден е от 8 думи = 16 байта = 32 тетради = 128 бита и страница (*page*) – 32 байта = 256 бита.

Табл.1.4 Списък с метрични префикси за количество информация

SI означения		ICE означения	
Десетично име	Стойност	Двоично име	Стойност
Kilobyte (kB) Килобайт	$(10^3)^1 = 10^3$	Kibibyte (KiB) Кибибайт	$(2^{10})^1 = 2^{10}$
Megabyte (MB) Мегабайт	$(10^3)^2 = 10^6$	Mebibyte (MiB) Мебибайт	$(2^{10})^2 = 2^{20}$
Gigabyte (GB) Гигабайт	$(10^3)^3 = 10^9$	Gibibyte (GiB) Гибибайт	$(2^{10})^3 = 2^{30}$
Terabyte (TB) Терабайт	$(10^3)^4 = 10^{12}$	Tebibyte (TiB) Тебибайт	$(2^{10})^4 = 2^{40}$
Petabyte (PB) Петабайт	$(10^3)^5 = 10^{15}$	Pebibyte (PiB) Пебибайт	$(2^{10})^5 = 2^{50}$
Exabyte (EB) Ексабайт	$(10^3)^6 = 10^{18}$	Exbibyte (EiB) Ексбибайт	$(2^{10})^6 = 2^{60}$
Zetabyte (ZB) Зетабайт	$(10^3)^7 = 10^{21}$	Zebibyte (ZiB) Зебибайт	$(2^{10})^7 = 2^{70}$
Yotabyte (YB) Йотабайт	$(10^3)^8 = 10^{24}$	Yobibyte (YiB) Йобибайт	$(2^{10})^8 = 2^{80}$

В зависимост от използваната бройна система са познати два вида метрични префикси (**Табл.1.4**):

- **SI префикси** – използват десетичната бройна система и започват с числото $1000 = 10^3$ – кило (k). Един kB (килобайт) е равен на 1000 B.
- **ICE префикси** – използват двоичната бройна система и започват с числото $1024 = 2^{10}$ – киби (Ki). Един KiB (kilo binary byte, кибибайт) е равен на 1024 B.

1.5 Шестнадесетична бройна система

Дългите двоични числа са неудобни за четене. Вместо тях се използват съответните им шестнадесетични числа. Всяка цифра от шестнадесетичната бройна система е свързана с четири бита или една **тетрада**. Две шестнадесетични цифри образуват байт. Една шестнадесетична цифра може да има стойност от 0 до 15. След броене от 0 до 9 се продължава с A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. Тъй като е възможно шестнадесетични числа да започват с буква, например A78C₁₆, то е прието на асемблер отпред да се прибавя една нула, т.е. 0A78Ch.

За да се преобразува шестнадесетично число в двоично е необходимо всяка шестнадесетична цифра да се замени със съответните ѝ четири двоични бита. Например, шестнадесетичното число 0BCF8Ah се преобразува в двоично чрез съпоставянето в **Табл.1.4**.

Табл.1.5 Преобразуване на шестнадесетично в съответното му двоично число

Шестнадесетично число	B	C	F	8	A
Двоично число	1011	1100	1111	1000	1010

Преминаването от двоично число към съответното му шестнадесетично представяне започва първо с допълване на двоичното число отляво с толкова нули, че полученият общ брой битове да може да се дели на четири без остатък. Тогава числото се разделя на тетради и всяка от тях се заменя със съответната ѝ шестнадесетична цифра. Например, разглеждайки двоичното число

10110100111011b се установява, че битовете му са 14 на брой. Извършва се допълване на числото отляво с две нули и получаваме 0010110100111011b. На следващата стъпка двоичното число се разделя на тетради: 0010 1101 0011 1011b. Замествайки всяка тетрада със съответната шестнадесетична цифра се получава шестнадесетичното число 2D3Bh.

1.6 Числа в допълнителен код

Допълнителният код на числата се използва за обозначаване на положителни и отрицателни числа. Валидна е таблица 1.5, обобщаваща минималните и максималните стойности, които могат да се записват в различните групи от битове.

Табл.1.6 Интервали на числата в допълнителен код в различни групи битове

Групи битове	Граници	Степени на 2
байт	-128..127	$-2^7..(2^7-1)$
дума	-32 768..32 767	$-2^{15}..(2^{15}-1)$
двойна дума	-2 147 483 648..2 147 483 647	$-2^{31}..(2^{31}-1)$

1.7 Двоично събиране и двоично допълнение

Събирането на двоични числа започва от дясно наляво, бит по бит, започвайки с двата младши бита на събираемите. Използват се следните правила, **Табл.1.7**:

Табл.1.7 Двоично събиране

+	0	1
0	0	1
1	1	10

Числото 10b е равно на 2d. Например, да се извърши събиране на числата 00001110b и 00000111b, **Фиг.1.8**:

$$\begin{array}{r}
 \text{пренос} \quad 1 \ 1 \ 1 \\
 \boxed{0} \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \boxed{0} \quad (14) \\
 + \\
 \boxed{0} \boxed{0} \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \quad (7) \\
 \hline
 \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{0} \boxed{1} \boxed{0} \boxed{1} \quad (21) \\
 \begin{array}{cccccccc}
 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0
 \end{array}
 \end{array}$$

Фиг.1.8 Пример за двоично събиране

Събирането започва с двата младши бита в позиции № 0: $0 + 1 = 1$. В позиция № 1 се събират $1 + 1 = 10$; като резултат се записва 0, а 1 е пренос за събирането в следващата позиция. В позиция № 2 се събират преноса и двата бита в тези позиции, т.е. $1 + 1 + 1 = 11$; като резултат се записва 1 и 1 е преносът за следващата позиция. В позиция № 3 събирането се събират отново преноса и двата бита: $1 + 1 + 0 = 10$; записва се като резултат 0 и 1 е пренос към следващата позиция. В следващата позиция събирането е $1 + 0 + 0 = 1$. Резултатът е 1 в позиция № 4, без пренос. В позиции № 5, 6 и 7 няма единици и пренос, събират се само нули; получават се само нули.

Двоичното допълнение на двоично число е допълнение до единица на числото, т.е. побитово инвертиране на битовете му. Операцията се записва като NOT (НЕ) и се изпълнява според следните правила, **Табл.1.8**:

Табл.1.8 Операция NOT

NOT	0	1
	1	0

Например да се инвертират битовете на числото 00010110b. Получава се двоичното число 11101001b.

1.8 Допълнителен код на двоичните числа

За да могат да се означават двоичните числа като положителни и отрицателни микропроцесорите 80x86 използват т.н.

допълнителен код на двоичните числа. При това обозначение в старшият бит от числото се записва 0, ако числото е положително и 1, ако числото е отрицателно. Преобразуването на положително число към съответното отрицателно се извършва като първо се инвертират битовете му и към получения резултат се прибави 1. Например, нека се изчисли осем битовият допълнителен код на десетичното число **92**, **Фиг.1.9**:

$$\begin{array}{r}
 01011100 \quad 92 \\
 10100011 \quad \text{инвертиране на всички битове} \\
 + \quad \quad \quad 1 \quad \text{събиране с 1 за получаване на резултата} \\
 \hline
 10100100 \quad -92
 \end{array}$$

Фиг.1.9 8 битов допълнителен код на числото 92

Действията по намирането на допълнителния код на числото - **92** ще доведат до получаване на първоначалната стойност от 01011100, **фиг. 1.10**:

$$\begin{array}{r}
 10100100 \quad -92 \\
 01011011 \quad \text{инвертиране на всички битове} \\
 + \quad \quad \quad 1 \quad \text{събиране с 1 за получаване на резултата} \\
 \hline
 01011100 \quad 92
 \end{array}$$

Фиг.1.10 8 битов допълнителен код на числото -92

Сумата на 92 и -92 трябва да е равна на нула:

$$\begin{array}{r}
 01011100 \quad 92 \\
 +10100100 \quad -92 \\
 \hline
 (1)00000000
 \end{array}$$

Фиг.1.11 Сума на 8 битовите числа 92 и -92

8-те бита имат нулеви стойности. Преносът от единица към девети бит се загубва.

1.9 Двоично изваждане

Изваждане между две числа се извършва като се намира допълнителния код на умалителя и той се събира с умаляемото.

Например, при 8 битово изваждане между умалителя 00001110b (14d) и умаляемото 00010101b (21d) е необходимо да се намери 8 битовия допълнителен код на 00001110b (14d), а това е 11110010b (-14). След това се извършва следното събиране:

$$\begin{array}{r} 00010101 \quad (21) \\ +11110010 \quad (-14) \\ \hline (1)00000111 \quad (7) \end{array}$$

Фиг.1.12 Пример за 8 битово изваждане

Преносът от единица не се взема под внимание.

Броенето в различните бройни системи се извършва чрез прибавяне на единица към най-младшата цифра от съответното число.

1.10 ASCII/IBM система от знаци

ASCII (American Standard Code for Information Interchange, Американски стандартен код за обмен на информация) системата от знаци (кодвата таблица) използва 7 бита за представяне на 128 символа с числата от 0 до 127. Тъй като се използва цял байт (8 бита), то останалите 128 числа, от 128 до 255, се използват за т.н. IBM система от знаци (кодвата таблица) – разширение на ASCII системата от знаци.

1.10.1 ASCII система от знаци

ASCII системата от знаци е разделена на четири групи по 32 знака. Първите 32 знака, от 0h до 1Fh, са управляващи знаци. Те извършват управленски операции, изпратени към принтер, а същевременно и се изписват като графични знаци, изпратени към дисплей. Най-използваните от тях са следните, **Табл.1.9**:

Втората група от ASCII системата от знаци, от 20h до 3Fh, включват различни пунктуационни символи, специални знаци и цифрите от 0 до 9. Важно е да се отбележи, че кодовете на цифрите са различни от техните стойности. Чрез изваждане на 30h от ASCII кода на всяка отделна цифра се получава числената ѝ стойност.

Табл.1.9 ASCII управляващи знаци

Стойност		Символ	Значение
(10)	(16)		
00	00	NUL	Празно (край на низ)
07	07	BEL	Звънец
08	08	BS	Връщане знак назад
09	09	HT	Хоризонтална табулация
10	0A	LF	Нов ред
11	0B	VT	Вертикална табулация
12	0C	FF	Нова страница
13	0D	CR	Връщане на каретката
24	18	CAN	Отказ
27	1B	ESC	Изход

Третата група от 32 знака, от 40h до 5Fh, са запазени за главните букви от английската азбука и различни специални символи. Четвъртата група, от 60h до 7Fh, включва малките букви от английската азбука, пет специални символа и управляващия знак DEL.

Тъй като асемблерът работи със стойности на знаци от 20h до 0FFh, то управляващите знаци е необходимо да се изписват с техните числени стойности.

Преминаването от малки към главни букви и обратно се извършва по следния начин: нека са дадени буквите d и D и техните ASCII кодове, **Фиг.1.13**:

	7	6	5	4	3	2	1	0
D	0	1	0	0	0	1	0	0
d	0	0	1	0	0	1	0	0

Фиг.1.13 Пример за ASCII кодовете на букви от английската азбука

Единствената разлика в битово представяне на двете букви е бит № 5. Чрез неговото инвертиране се извършва търсеното преминаване от малки към големи букви и обратно.

1.10.2 IBM система от знаци

Стойностите от 80h до 0FFh се използват за представяне на специални знаци, международни знаци, математически символи и линейни графични знаци.

1.11 Дефиниране на променливи и области от паметта

1.11.1 Директиви за дефиниране на данни

Всички директиви за дефиниране на данни могат да генерират инициализирани данни и да резервират място за неинициализирани данни. За неинициализираните данни началното значение е неопределено. В директивите за определяне на данни неинициализираните се указват с помощта на символа **?**.

Друго средство, което се използва във всички директиви за дефиниране на данни е използване на указание **dpl** (*duplicate*) за повторение на блока данни. Общият синтаксис за всички директиви за разпределение на данни е следния:

```
[ИмеНаДирект [Израз_dpl [Израз_dpl...]]]
```

Всеки Израз_dpl има следния синтаксис:

- **?**,
- Значение,
- Брой dpl (Израз_dpl[, Израз_dpl]),

където **Брой** задава колко пъти се повтаря блока данни.

За получаване на неинициализирани данни се използва идентификатор **?**.

Значение се използва за фактическо описание на отделния сегмент данни във всяка директива.

1.11.2 Директиви за дефиниране на променливи

Дефинирането на променливи се извършва с директивите **db** (*define byte*), **dw**, **dd**, **dq**, **df**, **dp**, **dt**.

Табл.1.10 Директиви за определяне на данни с различни размери

Директива	Значение
db	резервира един байт
dw	резервира една дума
dd	резервира двойна дума
dq	резервира четворна дума
df	резервира 6 байта (48-битов далечен указател за процесор 80386)
dp	резервира 6 байта (48-битов далечен указател за процесор 80386)
dt	резервира 10 байта

Полето Значение е различно за всяка от горните директиви. При **DB** може да се задават значения от -128 до 255; 8 битови относителни изрази, използващи операторите **high** и **low**; символни низове, заградени в кавички.

Например:

```
msg db "Hello World", 0dh, 0ah, '$'
```

Резервират се 14 байта за променливата **msg** запълнени низ от символи **Hello World**, следван от **;** символ за връщане на каретката, символ за нов ред и символ **\$**.

```
fname db 25 dup(?)
```

За променлива **fname** се резервират 25 байта от паметта, които остават с непроменено съдържание.

```
str db 1, 2, 3, 4, 5
```


1.12 Задачи

Задача 1.1 Преобразувайте следните десетични числа в двоични:

- a) 127 e) 256 i) 32767 m) 178 q) 511 u) 512 y) 278
b) 989 f) 23 j) 129 n) 15 r) 45 v) 2048 z) 19
c) 300 g) 84 k) 32768 o) 254 s) 81 w) 16
d) 128 h) 255 l) 1025 p) 4095 t) 1023 x) 1024

Задача 1.2 Преобразувайте следните двоични числа в десетични:

- a) 1000 0001 b) 1001 0001 c) 1111 0000 d) 1111 1111 e) 0101 1010
f) 1100 0010 g) 0100 0100 h) 1101 0001 i) 0101 0101 j) 1010 1011
k) 0110 0111 l) 0011 1100 m) 0100 1110 n) 0001 0011 o) 0111 1110
p) 1110 0111 q) 1010 1100 r) 1011 0101 s) 0101 1111 t) 0110 1110
u) 0111 1101 v) 1101 1100 w) 0011 1110 x) 1000 0111 y) 1001 1100
z) 1100 0001

Задача 1.3 Преобразувайте двоичните числа от **Задача 1.2** в шестнадесетични.

Задача 1.4 Преобразувайте следните шестнадесетични числа в двоични.

- a) ABCD e) CD0A i) ADD m) BEBE q) BADA u) 911A y) 17C9
b) 41D9 f) 1208 j) 95A3 n) 759C r) 180D v) FACE z) FADE
c) 15FA g) FFFF k) 74B3 o) 5432 s) DAA w) 359B
d) BAD h) DEAF l) C0DE p) DADA t) 2D3D x) BEEF

Задача 1.5 Извършете шестнадесетичните изчисления и запишете резултата в шестнадесетичен вид.

- a) $5234 + 7976$ f) $FFFF-1024$ k) $CCCC + 4444$ p) $1001 + ABD9$
b) $ABDC - 5B81$ g) $5050 + A505$ l) $FA01 + D1FF$ q) $5214 - 1015$
c) $FFA0 + 617C$ h) $1000 - 1$ m) $4321 + BCDE$ r) $DADE + 3522$
d) $8353 + 2797$ i) $ADB8 - 1111$ n) $21AC + 21AC$ s) $7353 - 4ABC$
e) $2A9C + E564$ j) $10000 - FFFF$ o) $1821 + E7DF$ t) $9919 + ABCF$

Задача 1.6 Колко шестнадесетични цифри се съдържат в следващите групи битове?

- a) байт b) дума c) двойна дума

Задача 1.7 Представете следните десетични числа в 8 битов допълнителен двоичен код (отбележете числата, за които това е невъзможно).

- a) -12 e) -83 i) -128 m) -67 q) -11 u) -1 y) -1024
b) -32 f) -15 j) -17 n) -512 r) -100 v) -14 z) -2
c) -132 g) -45 k) -29 o) -10 s) -127 w) -64
d) -129 h) -8 l) -256 p) -4 t) -23 x) -18

Задача 1.8 Преобразувайте двоични числа със знак от **Задача 1.2** в десетични числа със знак.

Задача 1.9 Кой № е старшият бит в следните групи битове?

- a) байт b) дума c) двойна дума

2 Микропроцесорна фамилия 80x86

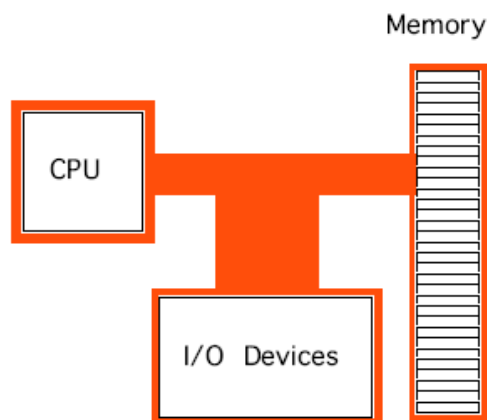
Основният работен дизайн на една компютърна система се нарича архитектура. Съвременните компютърни архитектури, микропроцесорните фамилии 80x86, се градят върху т.н. Фон Нойманова архитектура. Тя разделя компютъра на три големи части – памет (*memory*), централен процесор (ЦП, *Central Processor Unit*) и вход/изход (*Input/Output*). Компонентите от Фон Ноймановата архитектура са свързани помежду си със системна шина, т.н. кабелна информационна система.

2.1 Основни части

Всички изчисления в компютърната система се извършват в ЦП. Данните и инструкции стоят в паметта докато не бъдат извикани от ЦП. Инструкциите са специфични задачи, които казват на компютърната система какво действие да извърши. Те се появяват последователно в паметта.

ЦП работи с входно/изходните устройства както с паметта. Той може да записва данни в изходните устройства и да чете данни от входните устройства. Основната разлика между паметта и входно изходните направления е, че вторите са асоциирани с външни устройства извън компютъра.

На **Фиг.2.1** е показана типичната Фон Нойманова архитектура.



Фиг.2.1 Фон Нойманова архитектура

2.1.1 Системна шина

Системната шина използва две различни електрически нива за означаване на 0 и 1. За нула от 0,0 V до 0,8 V, а за единица 2,4 V до 5 V. Системната шина се дели на даннова, адресна и управляваща.

Даннова шина

Данновата шина пренася данни между паметта или входно/изходните устройства и централния процесор. Тя може да има големина (ширина) 8, 16, 32 или 64 бита (линии за данни). Ширината на данновата шина определя и големината на групите битове, които процесорът обработва. Съществуват съответно 8, 16, 32 и 64 битови процесори, **Табл.2.1**.

Табл.2.1 Размер на шината за данни при процесорна фамилия 80x86

Процесор	Размер на шината за данни (битове)
8088	8
8086	8
80188	8
80186	16
80286	16
80386sx	16
80386dx	32
80486	32
80586/Pentium (Pro)	64

Адресна шина

Адресната шина пренася адресите на специфични клетки от паметта или на входно/изходни устройства, с които ЦП взаимодейства. При n адресна шина ЦП може да работи с 2^n различни адреса. Процесорите 8088 и 8086 използват 20 битови адресни шини, работят с 2^{20} различни адреса. В следващата таблица са показани ширините на адресните шини и максималното адресируемо пространство.

Табл.2.2 *Размер на адресната шина при процесорна фамилия 80x86*

Процесор	Размер на адресната шина (битове)	Максимално адресируема памет
8088	20	1 MB
8086	20	1 MB
80188	20	1 MB
80186	20	1 MB
80286	24	16 MB
80386sx	24	16 MB
80386dx	32	4 GB
80486	32	4 GB
80586/Pentium (Pro)	32	4 GB

Управляваща шина

Управляваща шина има различни писти, които служат за разпределение на времеви и контролни сигнали в компютърната система.

2.1.2 Памет

Компютърната памет е номерирана последователност от клетки с ограничен обем. В нея се записват данни и програми, които се различават едни от други с това как се интерпретират от компютъра. Номерата на клетките се наричат адреси. Броенето им става в посока на нарастване на адресите. Клетката е с големина един байт. Така паметта представлява байтово адресируем масив. Байтът се записва в една клетка от паметта, думата в две последователни клетки – първо младшия байт, след това старшия байт. Двойната дума се записват в четири последователни клетки – започва се от младшия байт, **Фиг.2.2.**

	стойности	адреси
Двойна дума на адрес 578		582
	байт No 3	581
	байт No 2	580
	байт No 1	579
Дума на адрес 575	байт No 0	578
		577
	байт No 1	576
	байт No 0	575
Байт на адрес 572		574
		573
	байт No 0	572
		571

Фиг.2.2 Байт, дума и двойна дума на различни адреси

2.1.3 Вход/Изход

За разлика от 20, 24 или 32 битовите адресни линии, които извършват достъп до паметта, фамилията процесори 80x86 има 16 битова входно/изходна адресна шина. Достъпни за адресиране са не повече от 65 536 (2¹⁶) входно/изходни адреси. Адресирането е начина за обръщение към данните от паметта.

Съществуват две различни адресни пространства: едно за паметта и едно за входно/изходните операции. Линиите от управляващата шина се поделят между паметта и входно/изходните адреси. Входно/изходното адресиране се извършва както адресирането на паметта.

ЦП изисква специални инструкции за достъп до входно/изходните устройства.

2.2 Системно тактуване

Повечето от изпълняваните операции при процесорната фамилия 80x86 се изпълняват последователно. Това означава, че компютърът изпълнява команди в определен ред. За да осигури изпълнението на операциите в точното време, процесорната фамилия 80x86 използва редуващи се сигнали наречени системен таймер.

2.2.1 Системен таймер

Системният таймер управлява цялата синхронизация в компютърната система. Той представлява периодичен електрически сигнал, редуващ единици и нули. Един таймерен цикъл е времето за което системният таймер преминава от нула в единица и обратно към нула. Нарича се още таймерен период. Единицата *Hertz (Hz)* се използва за измерване на честотата на периодични процеси. 1 Hz съответства на един цикъл за една секунда. Тактовата честота на таймера се измерва в броя на циклите за една секунда. Използват се следните производни: 1 kHz = 1000 Hz и 1 MHz = 1000 kHz = 1 000 000 Hz. Видно е, че времето за което ще се изпълни един цикъл е реципрочно на таймерната честота. Например, един такт от 1 MHz има продължителност от 1 μ s (1 микросекунда), т.е. 1/1 000 000 част от секундата.

Един цикъл е най-малкият интервал от време, който може да съществува в работата на процесора. Всяко действие продължава най-малко един, а обикновено и повече цикли.. Например, при прехвърляне на данни от и към паметта на процесор 8086 са му необходими четири цикъла плюс състояние на изчакване. Състоянието на изчакване е цикъл, при който процесорът не извършва никакво действие, за да не изпревари останалата част от компютъра. За същата операция на процесор 80286 са му необходими два цикъла плюс състояние на изчакване.

Времето, необходимо за изпълнение на инструкциите, е също различно за различните процесори. Оригиналните процесори 8088 и 8086 изпълняват една инструкция средно за 12 цикъла. Процесорите 80286 и 80386 извършват същата инструкция за около 4–5 цикъла, а 80486 за два цикъла. При Pentium процесорите една инструкция се изпълнява за един цикъл, а при Pentium 4 за половин.

Сравнението на различните системи само на базата на тактовата честота е трудно поради различните времена (в цикли) за изпълнение на инструкциите. Така един 100 MHz Pentium се равнява приблизително на 200Mhz 80486 процесор или на 400Mhz процесор 80386 или 80286. Както се вижда, сравняването на различните системи само на базата на тактовата честота не е правилно, тъй като има много други фактори, които влияят върху производителността на системата.

2.2.2 Достъп до паметта и системния таймер

Достъпът до паметта е най-често извършваната дейност от ЦП. Тя се синхронизира с помощта на системния таймер. Времето за достъп до паметта е броят от цикли, които системата изисква за да се направи достъп до определена клетка от паметта. Различните 80x86 процесори имат различно време за достъп до паметта. То се променя в границите от един до четири цикъла.

Времето за достъп до паметта се равнява на сбора от времената на заявката за достъп до паметта (четене или запис) и времето до приключване на операцията.

2.2.3 Състояние на изчакване

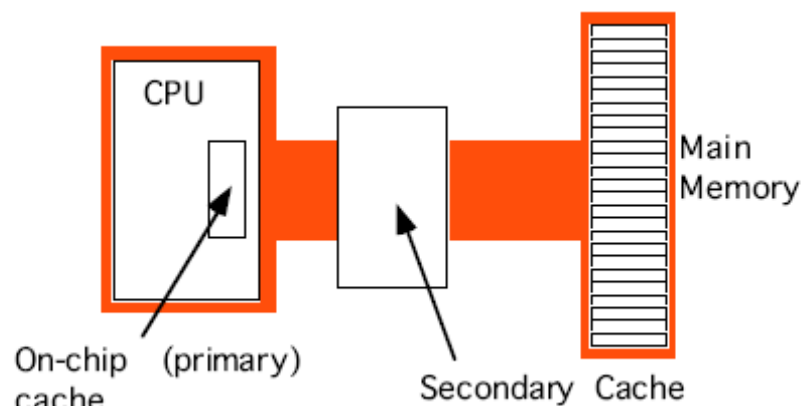
Състоянието на изчакване е допълнителен таймерен цикъл. То дава време на някое устройство да завърши операция. В повечето компютри между ЦП и паметта има допълнителни механизми (например за декодиране), в които се губят част от общото време за достъп до паметта. Най-често използвания подход за избягване на състоянията на изчакване е включването на т.н. кеш (*cache*) памет.

2.2.4 Кеш памет

ЦП работи много по-ефективно, когато има достъп до вече използвани данни и инструкции и такива на съседни адреси или до кеш-паметта. След като процесорът завърши това, над което е работил, той може да се обърне към нея, вместо към обикновената (и по-бавна) RAM памет, която се намира по-далеч и получаването на данни от нея изисква повече време.

Двата най-разпространени типа кеш памет се означават като L1 (*Level 1*- ниво 1) и L2 (*Level 2*– ниво 2). Има и кеш памет L3 (*Level 3*), но този вид не е много популярен. Въпреки, че в техническо отношение кеш паметта е вид памет, в повечето случаи L1 и L2 са вградени в процесорния чип или в самата процесорна карта. Така че тя е по-скоро елемент на процесора, отколкото на паметта.

Всяко ниво на кеш паметта представлява отделна част памет и се третира от процесора независимо. По традиция кеш паметта L1 е по-малка от двете и се разполага в самия процесор, а L2 се разполага извън него, но в непосредствена близост, **Фиг.2.3**.



Фиг.2.3 Две нива на кеш паметта

Когато процесорът работи с няколко вида кеш памет, първо проверява кеш паметта L1, след това кеш L2 и накрая – основната памет. Реално кешът е „прозрачен” за процесора, тъй като CPU се обръща към RAM паметта чрез виртуални адреси, а кеш паметта използва своите механизми за да определи дали има съвпадение на даден адрес вътре в регистрите си. При съответствие на адрес кеш паметта подава данните към процесора, като обръщението към основната памет се прекратява. Друга съществена разлика между кеш L1 и L2 е бързината, с която процесорът може да осъществява достъп до различните видове памет. Тъй като кеш паметта L1 е интегрирана във вътрешността на микропроцесора, тя обикновено работи със същата бързина, с каквато и централният процесор.

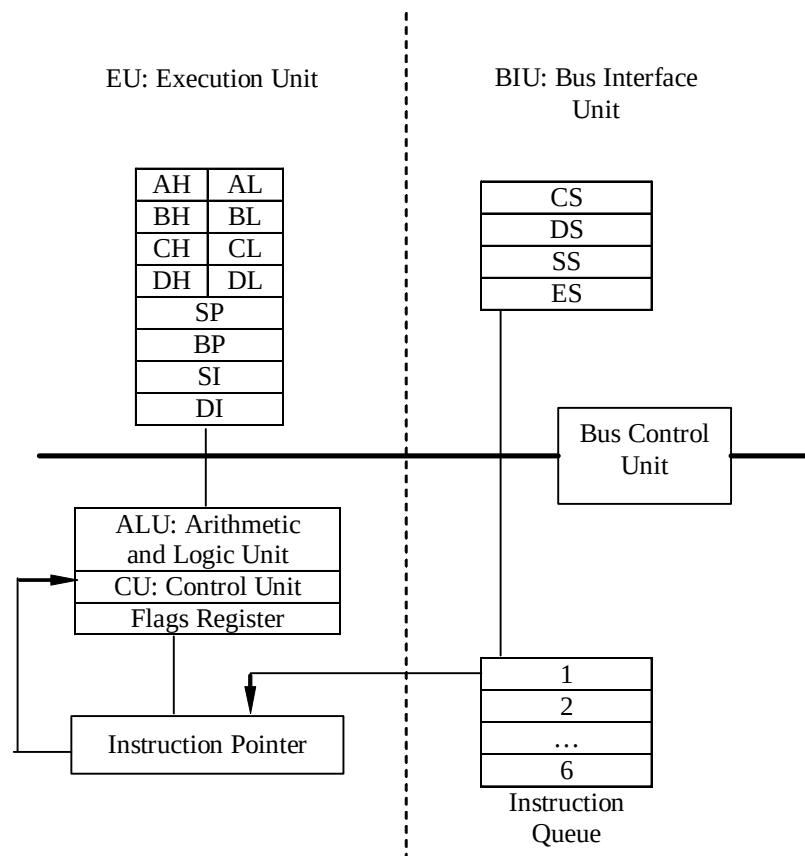
Например при процесор с тактова честота 500 MHz, скоростта на връзката към кеш паметта L1 е също 500 MHz. Кеш паметта L2 се свързва при по-старите системи с процесора със същата скорост като на оперативната памет. Тази скорост се определя от системната шина на компютъра. Ако кеш паметта се намира в самия процесор или на процесорната платка, както е при повечето процесори Pentium II и Pentium III, връзката процесор-кеш L2 става през така наречената задна шина (*backside bus*). Тази шина работи по-бързо от системната шина, но наполовина от скоростта на процесора. Това се нарича „съотношение 1:2“. Така при процесор Pentium III с тактова честота 500 MHz, скоростта на връзката процесор – кеш L2 е 250 MHz. При системи, при които кеш паметта

L2 е вградена в самия чип, има съотношение 1:1 между скоростта на процесора и скоростта на връзката процесор – кеш L2.

2.3 Фамилия процесори 80x86

Централният процесор е физическо устройство, което чете инструкции от паметта и ги изпълнява. Той е сърцето на компютърната система, изпълнява инструкциите и борава с данните. Представлява малка капсулирана силициева пластина с вградени микроелектронни елементи (транзистори). Микропроцесорите дълго време представляваха един-единствен чип, който се свързваше към различни по размер гнезда (*sockets*) върху дънната платка. Много от по-новите модели се произвеждат върху обособена платка с интегрални схеми, която се поставя в специален процесорен слот върху дънната платка. Най-важните характеристики на един процесор са: тип на процесора, скоростта, с която работи, размер и тип на включената в него кеш-памет, колко бита е шината за данни, колко битова адресна шина поддържа, допълнителни процесорни инструкции, които поддържа, тип на физическото свързване, което поддържа.

ЦП се разделя условно на два логически блока: изпълнително устройство (*execution unit, EU*) и шинно интерфейсно устройство (*bus interface unit, BIU*), **Фиг.2.4**.



Фиг.2.4 Блокова схема на фамилия процесори 80x86

Интерфейсът е съвкупност от средства осигуряващи взаимодействието между устройствата и програмните модули както на физическо така и на логическо равнище. Взаимодействието между интерфейса за вход и изход и шината се осъществява чрез входно/изходни портове, наречени *регистри*.

2.3.1 Регистри

Инструкциите, които ЦП изпълнява могат да изискват данни. За временно съхраняване на данни ЦП използва специални клетки памет в изпълнителното устройство, наречени *регистри*. ЦП има

достъп до данни в регистрите много по-бързо отколкото до данните в паметта. Броят на регистрите във ЦП е ограничен.

Микропроцесорната фамилия 80x86 има четири регистъра с големина 16 бита. Всички аритметически и логически операции се извършват в тези регистри. Имената на регистрите са следните: AX - акумулаторен (*accumulator*) регистър, BX - основен (*base*) регистър, CX - броячен (*count*) регистър, DX - даннов (*data*) регистър. Горните регистри са видими за програмиста, за разлика от регистър IP - указател на инструкциите (*instruction pointer*), който съдържа адресите на следващата инструкция за изпълнение. Съществува още флагов регистър - *Flags*, който съдържа резултати от сравнение. Флаговите регистри запомнят, ако една стойност е по-малка, равна или по-голяма от друга.

2.3.2 Аритметико и логическо устройство

Аритметико и логическото устройство (АЛУ), като част от изпълнителното устройство, е мястото, където се извършват повечето действия вътре в ЦП. Например, за събиране прибавяне на цифрата седем към стойността в регистър AX ЦП:

- копира стойността от AX в АЛУ;
- изпраща стойност седем към АЛУ;
- инструктира АЛУ да събере тези две стойности;
- премества резултата в регистър AX.

2.3.3 Шинно интерфейсно устройство

Шинното интерфейсно устройство (ШИУ) образува интерфейс към адресната шина и шината за данни, чете инструкции от паметта, обменя данни с паметта или периферни устройства, предава команди към изпълнителното устройство. ШИУ отговаря за достъпа до данните в поставената в ЦП кеш памет.

2.3.4 Управляващо устройство

Управляващото устройство (УУ) е сложна електронна схема, която извършва управлението и координирането на повечето от дейностите на компютъра. То не изпълнява инструкции, а казва на отделните части на компютърната система какво да правят. То определя движението на електронните сигнали между паметта и

АЛУ, а също и на контролните сигнали между ЦП и входно-изходните устройства.

Управляващото устройство следи къде се намира следващата за изпълнение инструкция в паметта и се грижи резултатът от изпълнението на инструкцията да бъде записан в паметта, ако това е посочено в програмата.

2.3.5 Изпълнение на инструкции

Изпълнението на инструкциите се осъществява на следните четири стъпки:

1. ЦП прочита байта инструкция от паметта. За да направи това той копира стойността от **IP** регистъра на адресната шина и прочита байта или байтовете от този адрес.

2. Декодиране на инструкцията.

3. Изпълнение на инструкцията. Тук, ако е необходимо, процесора прочита данни от паметта.

4. Ако е необходимо, то ЦП връща резултатите от изпълнението на инструкцията в паметта. Увеличава се съдържанието на **IP** спрямо броя байтове в инструкцията. Така **IP** се насочва към следващия байт в потока от инструкции.

2.6 Задачи

Задача 2.1 Опишете основните части от компютърната система.

Задача 2.2 Посочете четирите стъпки за изпълнение на инструкция.

3 Микропроцесорна архитектура 80x86– програмна гледна точка

Към изучаването на асемблер се пристъпва при условие, че е изяснено каква част от компютъра е оставена видима и е достъпна за програмиране на този език.

3.1 Регистри в процесорите 8086

Броят на регистрите в процесорите 8086 е 14. Те могат да се използват в т.н. реален режим (*real mode*). В него програмите работят с памет до 1 МВ и се разделят на сегменти с големина до 64 kB. Регистри се групират в сегментни, с общо предназначение и специални. Всички те са 16 битови.

3.1.1 Сегментни регистри

Сегментите са области, които започват на границата на параграф, т.е. на произволен адрес, дялящ се на 16 без остатък. Те могат да се разполагат на произволно място в паметта и имат размер до 64 kB. Съществуват четири сегментни регистъра.

CS – регистър за кодовия сегмент (*code segment*). Съдържа адреса на сегмента с текущо изпълняваните инструкции.

DS – регистър за данновия сегмент (*data segment*). Сочи началния адрес на сегмента за данни в текущата програма.

SS – регистър за стековия сегмент (*stack segment*). Съдържа адреса на област от паметта, наречена *стек*.

ES – регистър за допълнителния сегмент (*extra segment*) за данни. Използва се от инструкции работещи с низове.

3.1.2 Регистри с общо предназначение

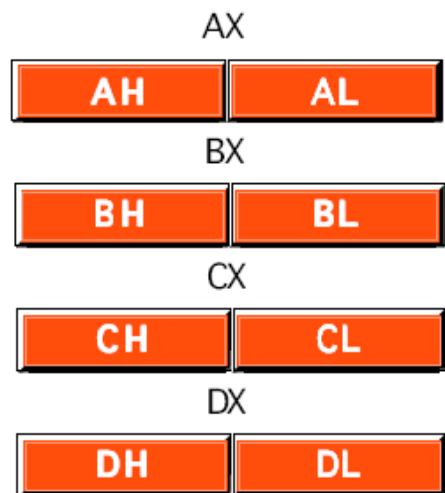
Делят се на индексни регистри, регистри за данни и регистри указатели.

Индексни регистри

Индексните регистри са два: **SI** - регистър за източника на данни (*source index*) и **DI** – регистър за назначението на данни (*destination index*).

Регистри за данни

Регистрите за данни са четири 16 битови регистъра: AX, BX, CX и DX. Всеки от които е разделен на две части по 8 бита (8 битови регистри): AX: AH (*high*, старша част) и AL (*low*, младша част), BX: BH и BL, CX: CH и CL, DX: DH и DL.



Фиг.3.1 Регистри за данни в 8086

AX - използва се за временно съхранение на данни, за изпълнение на различни аритметически и логически операции.

BX - служи като индекс при различни видове адресации, съхранява адреси на области от паметта и участва при някои изчисления.

CX - използва се за брояч при управление на повторението на цикли и за операции за преместване наляво и надясно.

DX - служи за съхраняване на данни при входно-изходни операции. Приема препълвания при различни аритметически операции.

Трите регистъра: L, H и X не могат да бъдат разгледани независимо един от друг. Например, ако бит № 4 на H (старшият байт) се промени, тогава бит № 12 на X се променя автоматично.

Стойността на X регистъра може да бъде определена от стойностите на H и L регистрите и обратно. Извършва се по

следната формула: $X = H * 256 + L$. Например, ако BH = 12, BL = 117, то BX = BH * 256 + BL = 12 * 256 + 117 = 3189.

Регистри указатели

Регистрите указатели SP и BP обезпечават достъпа до данните в стековия сегмент.

SP – стеков указател (*stack pointer*). Определя отместването в сегмента на текущия връх на стека. Използва се заедно с регистър SS за адресиране на стека.

BP – базов указател (*base pointer*). Използва се за организация и свободен достъп до данните в стека.

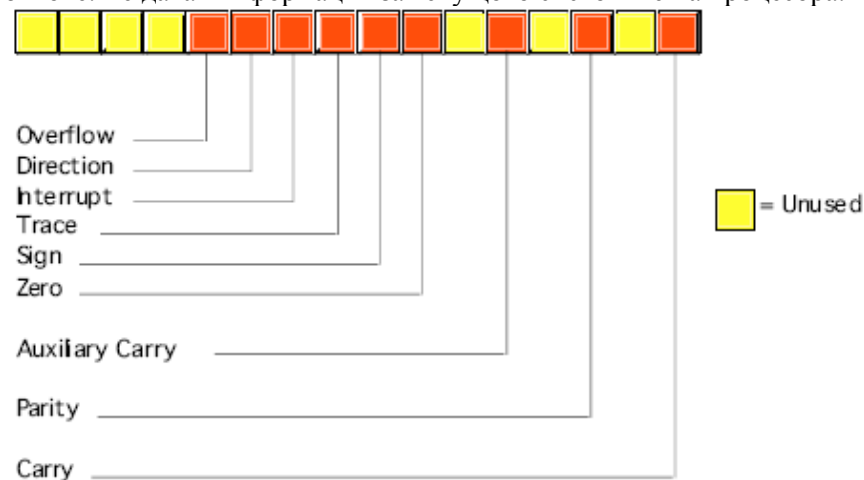
Поради използването на регистри BX, BP, SI и DI при различните видове адресации, то те се наричат още *адресни* регистри. Използват се за съхранение на адреси от паметта и към тях се обръщат командите за работа с данни от паметта.

3.1.3 Специални регистри

Специалните регистри са IP и Flags.

IP – указател на инструкциите (*instruction pointer*). Съдържа относителното отместването на следващата подлежаща на изпълнение команда спрямо съдържащия се адрес в регистър CS.

Flags – флагов регистър. Флаговият регистър е набор от битове. Те дават информация за текущото състояние на процесора.



Фиг.3.2 Флагов регистър

Девет от шестнадесетте бита от флаговия регистър са активни. Чрез тях се получават резултати от изпълнявани команди.

CF – флаг за пренос (*carry*). Съдържа пренос от старшия бит след аритметически операции. Установява се като индикатор на заема при изваждане.

PF – флаг за четност (*parity*). Показва четността на единиците на младшия байт на резултата (1 четно и 0 нечетно число).

AF – флаг за междинен (*auxillary*) пренос. Показва пренос от бит 3 към бит 4.

ZF – нулевият (*zero*) флаг показва дали резултатът от последната аритметическа или логическа операция е нулев (1 нулев и 0 ненулев резултат).

SF – флагът за знак (*sign*) съдържа резултатния знак след аритметическа операция (1 - минус и 0 - плюс).

TF – флаг за следене (*trace, trap*). Нарича се още флаг за постъпково изпълнение на програмата. Установява се в 1 когато процесорът работи в такъв режим.

IF – флаг за прекъсване (*interrupt*). Приема стойност 1 когато когато външно устройство генерира прекъсване.

DF – флаг за посока (*direction*). Използва при работа с низове. Ако е установен в 1, операциите за работа с низове намаляват съдържанието на индексния регистър.

OF – флагът за препълване (*overflow*) се установява в 1, ако резултатът от аритметическата операция е извън интервала на числата със знак.

При програмиране на асемблер най-често се използват флагове CF, ZF, SF, OF при работа с аритметически операции и операции за сравнения и флаг DF за обозначение на посоката в операциите с низове.

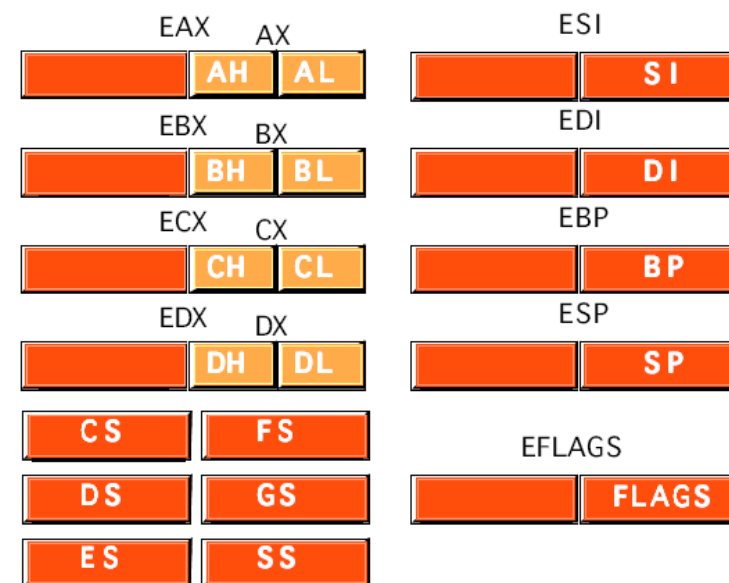
3.2 Регистри в процесорите 80286

Процесорът 80286 добавя т.н. *16 битовия защитен режим*. При него е възможно да се адресират до 16 MB и да се защитят една от друга програмите в паметта. Все още програмите се разделят на сегменти, не по-големи от 64 kB. Добавен е още един регистър – дума за състоянието (*machine status word*). Добавени са два флага: IOP – флагът за нивото на приоритетите (I/O-protection level) е

двубитов и показва нивото на приоритета за вход/изход; NTF – флаг за състоянието на влагане (*nested task flag*). Следи влагането на различни задачи една в друга.

3.3 Регистри в процесорите 80386

С този процесор се въвеждат 32 битовите регистри: EAX (*extra*), EBX, ECX, EDX, ESI, EDI, EBP, ESP, EIP, EFlags и се добавят два нови 16 битови сегментни регистъра FS и GS.



Фиг.3.3 Регистри в 80386

Въвежда се новия *32 битов защитен режим*. Програмите отново се разделят на сегменти, но всеки сегмент може да има размер до 4 GB. Появяват се три нови флага: RF – флаг за възстановяване (*resume flag*) на работа, който управлява операциите за настройка на програмата; VF – флаг за виртуален 8086 режим (*virtual 8086 mode*) – симулира многозадачна среда.

3.4 Регистри в процесорите 80486/Pentium

Появяват се няколко нови флага: ACF – флаг за проверка на подреждането (*alignment check*); VIF – флаг за виртуално прекъсване (*virtual interrupt*); VIP – флаг за активно виртуално прекъсване (*virtual interrupt pending*); IF – флаг за идентификация (*identification*) за инструкция CPUID.

3.5 Сегменти. Принцип на сегментацията

При работа в реален режим 20 битовата адресна шина в ЦП 8086 и 8088 може да приема адреси с дължина до 20 бита, т.н. физически адреси. Проблем е образуването на 20 битов адрес от 16 битови регистри. За да се получи 20 битов адрес са необходими още 4 бита. Те се вземат от сегментните регистри чрез допълване на стойността от съответния регистър с четири нулеви бита като по този начин го правят пълна 20 битова стойност. Към получения 20 битов резултат се прибавя отместването (*offset*) в сегмента. Получава се 20 битов физически адрес.

Сегментен регистър	Базов адрес					20 битова стойност
	xxxx	xxxx	xxxx	xxxx	0000	
+						
Отместване	xxxx	xxxx	xxxx	xxxx		
	xxxx	xxxx	xxxx	xxxx	xxxx	20 битов адрес

Фиг.3.4 Изчисляване на адрес (сегмент + отместване) в сегмент

Стойността в сегментния регистър се нарича *базов адрес*. Това адресиране се нарича адресиране от вида базов адрес + отместване. То се записва с двоеточие – <базов_адрес>: <отместване>. Например нека е дадено следното адресиране, използващо шестнадесетични числа: 1AC3h:2221h. Физическият адрес се получава със следното събиране (шестнадесетичен вид):

$$\begin{array}{rcl}
 & 1AC30h & 20 \text{ битова стойност} \\
 + & 2221h & \text{отместване} \\
 \hline
 & 1CE51h & 20 \text{ битов адрес}
 \end{array}$$

Фиг.3.5 Получаване на 20 битов физически адрес

Отместването спрямо базов адрес се нарича още ефективен адрес.

Посредством сегментите се дава възможност за преместваемост на програмите и данните в паметта. При асемблирането всяка програма се разполага на определени адреси в паметта. Всеки сегмент се стартира от асемблера от отместване 0. В листингите, в съседство с някои адреси е записан символът **R**. Например:

```

9 0000    B8 ---- R  mov  ax, data
10 0003    8E D8      mov  ds, ax
11 0005    BA 0000 R  mov  dx, offset mes

```

Това означава, че адресът е преместваем. Ако програмата се премести по такъв начин, че началото ѝ се намира на адрес, различен от отместване 0, адресът отбелязан с R, трябва да бъде променен. Преместванията се управляват от свързващата програма.

За получаване на адреса на сегмента се използва операторът **seg**:

seg Израз

Той връща сегментния адрес на Израз. Например при условие, че е даден следния откъс от програма на асемблер:

```

dataseg
msg1 db "Hello World", 0dh, 0ah, '$'
msg1 db 'Good Morning', 0dh, 0ah, "

```

следващите редове са еквивалентни един на друг:

```

mov  ax, seg msg1
mov  ax, seg msg2

```

Регистър AX получава адреса на сегмента, в който се намира променливите msg1 и msg2.

Получаването на адреса на отместването на Израз в текущия сегмент се извършва с оператор offset:

offset Израз

Например:

```
mov ax, offset msg1
mov ax, offset msg2
```

3.6 Стек

Стекът (*stack*) е структура от данни от тип LIFO (*last in - first out*, последен влязъл – първи излязъл) за временно съхраняване на информация, в която чрез специални инструкции се записват отделни думи. Нови данни могат да се запишат в стека само след последните записани до този момент данни или, както е прието да се казва, данните се поместват във върха на стека. Прочетен може да бъде само последният елемент от стека, което води до неговото премахване. Стекът се намира в оперативната памет.

3.7 ROM-BIOS и DOS

Една от задачите на програмирането включва достъпа до компютърния хардуер. Вместо този достъп да се осъществи направо от програма, обръщаща се директно към него, то тя може да използва ROM-BIOS (*Read Only Memory-Basic Input Output System*) и DOS (*Disc Operating System*) като посредници. ROM-BIOS и DOS са програмни интерфейси, които са създадени специално за управление на хардуера. Обръщението към хардуера става чрез стандартни функции на ROM-BIOS и DOS. След като задачата е изпълнена се връща съответна информация необходима за програмата.

3.7.1 ROM-BIOS

ROM-BIOS може да бъде разгледан като софтуерен пласт покриващ хардуера. Намира се на ROM чип, поставен на дънната платка на компютъра. Той предлага функции за достъп до следните

устройства: видеоконтролери, RAM, флопи дискове, хард дискове, серийни портове, паралелни портове, клавиатура, часовник за реално време.

3.7.2 DOS интерфейс

За разлика от BIOS DOS разглежда хардуера като логически, а не като физически устройства. Функциите на DOS обслужват хардуера по различен начин. Например ROM-BIOS разглежда дисковите устройства като групи от пътечки и сектори, а DOS като групи от файлове и директории.

3.7.3 Достъп до хардуера

Съществуват три възможности за достъп до хардуера: директно управление, използване функциите на BIOS и използване функциите на DOS. За чертане на геометрични фигури не съществуват функции на BIOS и DOS, необходимо е директно управление на хардуера. В зависимост от поставената задача се определя и начина за достъп до хардуера.

3.8 Прекъсвания

Прекъсване се нарича временно спиране на изпълнението на текущата програма. Различават се хардуерни (наричани още: апаратни, външни или асинхронни) и софтуерни (наричани още: програмни или вътрешни) прекъсвания. Първите са инициирани от сигнали, подавани от периферни устройства, а вторите - от състояние на ЦП или от команда. Типичните хардуерни прекъсвания са показани в Табл.3.1:

Табл.3.1 Хардуерни прекъсвания

Номер	Предназначение
08h	Таймер – извиква се 18.2 пъти в секунда
09h	Клавиатура
0dh	Твърд диск
0eh	Флопи диск
0fh	Принтер
70h	Часовник за реално време

Типичните софтуерни прекъсвания предизвикани от ЦП са показани в Табл.3.2:

Табл.3.2 Прекъсвания предизвикани от ЦП

Номер	Предназначение
00h	Деление на нула
03h	Достигане на точка на прекъсване
04h	Числово препълване

От прекъсванията на BIOS често се използват показаните в Табл.3.3:

Табл.3.3 Прекъсвания на BIOS

Номер	Предназначение
10h	Достъп до видео функции
11h	Определяне на конфигурацията
12h	Определяне на размера на RAM
13h	Флопи/хард дискови функции
14h	Достъп до серийни портове
16h	Четене от клавиатурата
17h	Достъп до паралелни портове
19h	Рестарт на системата (Ctrl-Alt-Del)
1Ah	Функции за дата и час

Най-често използваното прекъсване на DOS е 21h (33d). При неговото извикване е възможно изпълнението на множество функции, определяни чрез поставянето на номера им в регистър AH. В таблицата с stdout е означено стандартното устройство за изход (обикновено екрана), а с stderr – устройството за изход на съобщения за грешки (екрана).

Табл.3.4 Често използвани функции на прекъсване 21h на DOS

Функция	Вход	Действие	Изход
01h	ah = 01h	Вход от клавиатурата с ехо на екрана.	al = прочетен символ

Функция	Вход	Действие	Изход
02h	ah = 02h dl = код на символ	Извежда символ от dl на екран с проверка за Ctrl+Break.	--
06h	ah = 06h dl = код на символ (без ffh)	Запис на символ в stdout без проверка на Ctrl+Break.	--
07h	ah = 07h	Вход от stdin без ехо и без проверка за Ctrl+Break.	al = код на символ
08h	ah = 08h	Вход от stdin без ехо с проверка за Ctrl+Break	al = код на символ
09h	ah = 09h ds:dx = адрес на низа.	Извеждане на низ на екрана, до срещане на символ за долар (\$, 24h) и проверка за Ctrl+Break.	--
0Ah	0Ah = 0Ah DS:DX = адрес на низа	Прочитане на символи от stdin и запис в низ, адресиран от DS:SI до натискане на Enter. Първият байт съдържа максималното число символи за прочитане (1-254). Като последен байт се записва 0Dh.	--
40h	ah = 40h bx = 1 за stdout или 2 за stderr; ds:dx = адрес на низа; cx = дължината на низа	Запис във файл или устройство	CF = 0; ax = число записани байтове
4Ch	ah = 4Ch	Завършване на програма с връщане на код	--

В компютърната система се допуска включване най-много на 256 периферни устройства. Те изискват различни условия за прекъсване. Всяко от тях доставя посредством шината за данни един номер на прекъсване. От номера на прекъсване, чрез умножение по 4, ЦП образува адрес в паметта, където се намира вектор на прекъсването. Първите 1024 байта от паметта са резервирани за вектори на прекъсвания. Векторите на прекъсванията са адреси с големина 4 байта (сегментен адрес и отместване). Те съдържат указатели към подпрограма за обработка на съответното прекъсване.

При условие, че се появи заявка за прекъсване в главната програма, то обработката му протича по следния начин:

Текущо изпълняваната инструкция се довежда докрай и тогава се прекъсва програма.

Номерът на прекъсването се умножава по 4 и се пресмята вектора на прекъсването.

1. В стека се записват стойностите на **Flags** и **IP**, **CS** (*адрес за връщане*).
От таблицата за векторите на прекъсване ЦП намира адреса за преход към подпрограмата за обработка на прекъсването.
Управлението се предава на тази подпрограмата за обработка на прекъсването.
2. Накрая на подпрограмата се намира инструкция за обратен преход: **IRET** (*interrupt return*), която в обратен ред на постъпването им извлича от стека стойностите на **CS**, **IP**, и **Flags**.

Софтуерните прекъсвания се извикват с инструкция **INT** (*interrupt*). Тя изисква един операнд, който е номера на прекъсването. Към някои от прекъсванията има асоциирани множество функции, които се задават в регистрите. Допустими са и подфункции.

Извикването на софтуерното прекъсване започва първо с поставянето (зареждането) на регистрите с номерата на функциите и подфункциите. Например за функция 02h на 21h прекъсване на DOS е необходимо да се направи следното: да се прехвърли в регистър **dl** код на символ, а в регистър **ah** да се прехвърли числото 02h:

```
mov dl, 67h ; 67h = 'g'
mov ah, 02h
```

След това номера на прекъсването се подава на инструкция **INT**:

```
int 21h
```

Като резултат от горния програмен фрагмент на екрана на монитора ще се изобрази символа 'g'.

3.9 Видове адресиране

3.9.1 Регистрово адресиране

Обръщението на процесора към регистрите си се нарича регистрово адресиране. В качество на операнди в операторите на асемблер се като адреси се използват означенията на регистрите:

```
push cs
mov ah, ch
inc bx
```

3.9.2 Непосредствено адресиране

Това адресиране използва непосредствени операнди за изчисляване на адресите:

```
mov ax, 2
mov bx, 'q'
```

Данните са в инструкцията и не е необходимо допълнителното изчисляване на адреса.

3.9.3 Пряко адресиране

При този вид адресиране инструкциите използват променливи като названия на адреси от паметта. Прякото (директно, *direct*) адресиране се нарича още адресиране по отместване. Прякото адресиране бива абсолютно и относително.

Абсолютно пряко адресиране

Ако са декларирани следните променливи:

```
dataseg
var1 dw 3478h
var2 dw ?
```

и регистър **ds** се насочи към началото на сегмента за данни:

```
codeseg
start:
mov ax, dgroup
```

```
mov ds, ax
```

тогава променлива `var1` е записана в паметта на адрес `ds:0000`, а `var2` на адрес `ds:0002`. Съдържанието на паметта е следното:

```
ds:0003..00
ds:0002..00
ds:0001..34
ds:0000..78
```

При абсолютното пряко адресиране единият операнд е адрес от паметта, а другият е регистър. В режим **IDEAL** е задължително използването на квадратните скоби [и] при извличане на стойности. Например:

```
mov ax, var1
```

ще доведе до предупредително съобщение при опит да се прехвърлена стойността на променлива `var1` в регистър `AX`. Правилна е следната форма:

```
mov ax, [var1]
```

Означението `[var1]` следва да се чете като стойност на паметта от адрес съдържащ се в променливата `var1`. Регистър `AX` приема стойност `3478h`.

Извличането на отместването на променливата вътре в сегмента се извършва с оператор **offset**. Например в резултат от израза:

```
mov ax, offset var2
```

в регистър `AX` се записва стойност `2h`, колкото е отместването на `var2` от началото на сегмента за данни.

При изпълнението на следните два реда

```
mov ax, 0ffffh
mov [var2], ax
```

първо в регистър `AX` се записва стойност `0ffffh`. След това стойността от регистър `AX` се записва на адреса на променливата `var1`:

```
ds:0003..ff
ds:0002..fe
```

Изпълнението на следния ред

```
inc [var2]
```

увеличава с единица стойността намираща се на адрес сочен от `var2`:

```
ds:0000 79
ds:0000 34
```

Относително пряко адресиране

Този вид адресиране се използва от команди за условни преходи за указване на относителния адрес на прехода. Извършва се с помощта на етикети:

```
jmp to_code
msg db "Hello World", 0dh, 0ah, '$'
to_code:
    mov dx, offset msg
```

3.9.4 Косвено адресиране

При косвеното (индиректно, *indirect*) адресиране абсолютният адрес се формира от комбинацията на сегментния адрес в един от сегментните регистри и отместването в регистри `BX`, `BP`, `SI` или `DI`.

Регистрово косвено адресиране

За извличане на стойности сочени от изброените регистри се използват квадратните скоби [и]. Например на основата на следния програмен откъс:

```
dataseg
var1 dw 3478h
```

```
var2 dw 4455h
```

```
codeseg
start:
    mov ax, dgroup
    mov ds, ax
    mov bx, 02h
```

чрез регистровото косвеното адресиране:

```
mov ax, [bx]
```

регистър AX приема стойност 4455h. Записът [BX] следва да се чете като стойност на паметта от адрес съдържащ се в регистър BX.

Използването на следния запис:

```
mov [bx], 5
```

довежда до поява на предупреждение, тъй като не се знае дали се иска константата да се запише в байт, дума, или др. За да се избегнат грешки се използва оператор ptr за предефиниране или уточняване на типа:

Тип ptr Операнд

които предефинира Операнд. Тип може да приема една от следните стойности:

Табл.3.5 Тип на оператор ptr

Тип	Стойност
byte	1
word	2
dword	4
qword	8
tbyte	10
near	0FFFFh
far	0FFFEh

Правилният запис на горния израз е следния:

```
mov word ptr [bx], 5
```

Във всички нееднозначни изрази е препоръчително използването на оператор ptr.

Базово косвено адресиране

При базовото косвено адресиране в инструкцията се задава допълнително и константа: [База+Отместване]. В спомагателният регистър се съдържа началния адрес, наречен база, на масив (низ, списък), а отместването, зададено с константата) дава възможност за достъп до произволен елемент от този масив. Константата сочи към относителната позиция (*displacement*) спрямо началния адрес на масива. Например:

```
.data
var1 dw 1, 2, 3

.code
start:
    mov ax, @data
    mov ds, ax
    mov bx, offset var1
    mov ax, 0
```

чрез базовото косвеното адресиране:

```
add ax, [bx+0]
add ax, [bx+2]
add ax, [bx+4]
```

в регистър AX се сумира масив var1.

При този вид адресиране също се използва оператор ptr в изрази като

```
mov word ptr [bx+4], 5
```

Индексно косвено адресиране

При индексното косвено адресиране адресирането се извършва отново във вида [База+Отместване]. Тук обаче, за разлика от базовото косвено адресиране, База се намира в константата. Там е началния адрес на масива. Отместване (индекса) на елемента се съдържа в спомагателния регистър. Например:

```
mov bx, 3
mov word ptr [string+bx], 5
add ax, [string+bx]
```

Базово-индексно косвено адресиране

При това адресиране ефективният адрес се формира като сума от стойностите на два регистъра базов и индексен в следните комбинации: [bx+si], [bx+di], [bp+si], [bp+di] (използват се и 32 битовите регистри). Например:

```
mov eax, [ebx+esi]
```

Базово-индексно косвено адресиране с отместване

Този вид адресиране използва базовото-индексно косвено адресиране като при него адреса на клетката от паметта се намира като към съдържанието на двата спомагателни регистъра се прибавя Отместване намиращо се в константа: [bx+si+Отместване], [bx+di+Отместване], [bp+si+Отместване], [bp+di+Отместване] (използват се и 32 битовите регистри). Базовото-индексно адресиране с отместване се използва за директно обръщение към елементи от двумерни масиви (масиви, таблици).

- | | | |
|------------------|-----------------|------------------|
| d) 4444h:0CCCCCh | e) 4567h:0ABCDh | f) 0FA01h:0D1FFh |
| g) 0FF0h:617Ch | h) 1000h:0001h | i) 0ACDCh:0ACDh |
| j) 8888h:22h | k) 8D1h:0D12Dh | l) 0ACBDh:0ACBDh |
| m) 0E64Eh:0E21Ah | n) 1010h:0FFFCh | o) 55h:87h |

3.10 Въпроси

3.10.1 Опишете микропроцесорна архитектура 80x86 от програмна гледна точка.

3.10.2 Получете 20 битовите физическите адреси от следните адресирания:

- a) 28F3h:7211h b) 231h:1221h c) 0CCCh:4444h

4 MS-DOS

MS-DOS е операционна система, което означава, че това е системен софтуер чрез който се осъществява началното зареждане на компютъра и служебни действия, необходими за ефективното му използване.

Трите основни части на MS-DOS са DOS-BIOS, ядрото на DOS и командния процесор. Тук се въвеждат основните понятия свързани с работата на командния процесор (интерпретатор).

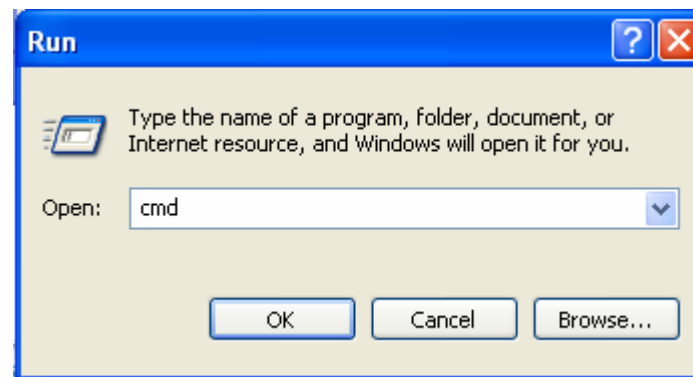
4.1 Команден процесор

Командният процесор се съхранява във файл `command.com` или `cmd.exe`. В операционната система Windows XP двата файла обикновено се намират в следната папка: `C:\WINDOWS\system32`. Стартирането му се извършва по следния начин: от менюто на Start бутона се избира менюто Run (или с клавишна комбинация Win+R):



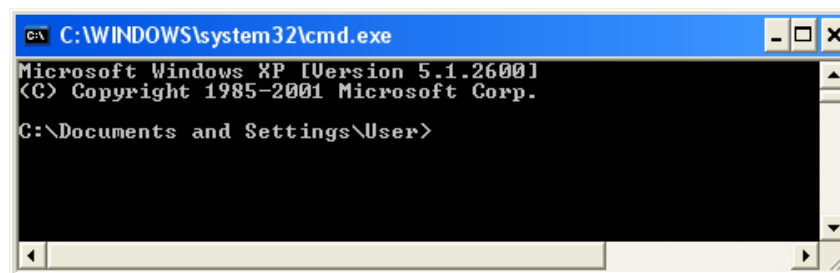
Фиг.4.1 Меню Run

Отваря се прозореца Run. В него се изписва името `cmd` по следния начин:



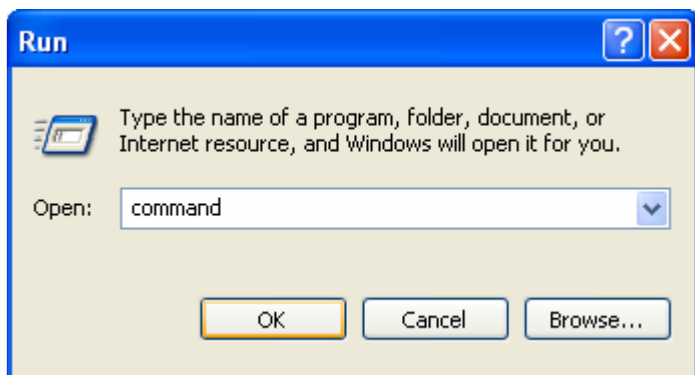
Фиг.4.2 Прозорец Run, стартиране на команден процесор cmd

Натиска се бутон OK. Стартира се командният процесор:



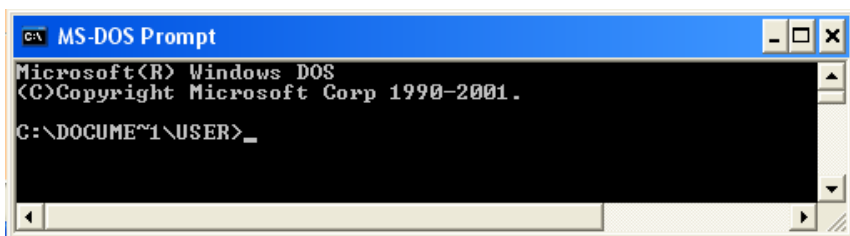
Фиг.4.3 Работа на командния процесор cmd

Или в прозорец Run може да се изпише името `command` по следния начин:



Фиг.4.4 Прозорец Run, стартиране на команден процесор *command*

След натиска на бутон OK се стартира командния процесор:



Фиг.4.5 Работа на командния процесор *command*

В него се извежда т.н. команден ред с подканващото съобщение “C:\>”, “D:\>” и др. на екрана на компютъра. Мигащата хоризонтална чертичка се нарича курсор и показва текущата позиция за въвеждане и извеждане.

16 битовата програма *command.com* се нарича още виртуална DOS машина (*virtual DOS machine, VDM*). Използва се за стартиране на MS-DOS приложения и скриптове. Програмата *cmd.exe* е 32 битова версия на Windows XP за команден процесор.

Командният процесор приема команди от потребителя, изпълнява ги и връща резултат от това изпълнение. След изписване на командата и нейните параметри се натиска клавиш Enter. Например:

C:\DOCUME~1\USER>Команда [/параметри]

Командите, които са включени във файл *cmd.exe* (или *command.com*) се наричат вътрешни. Останалите команди на MS-DOS са външни и представляват файлове с разширения *.com*, *.exe*, *.bat* или *.cmd*. Параметрите съдържат информация за изпълнението на командата. Ако командата се изпише без параметри, то командният процесор подразбира стойностите им.

За изход от командния процесор се използва командата *exit*.

4.2 Файлова структура

4.2.1 Имена на устройства

На всеки носител на информация (флопи диск, хард диск, флаш памет и др), свързан към компютърната система, MS-DOS предоставя уникално име. Това име е буква от английската азбука: a..Z или A..Z, последвана от двоеточие (:). Например:

- A: Име на първо флопи дисково устройство,
- B: Име на второ флопи дисково устройство,
- C: Име на хард диска.

Допуска се разделянето на хард диска на няколко логически устройства: C:, D:, E:. След това са имената дадени на свързаните с компютърната система CD и DVD устройства, примерно:

- F: Име на CD устройството,
- G: Име на DVD устройството.

Възможно е да има само едно такова устройство или да са повече от две (дори повече от две от един и същ вид). След тях следват имената на включените флаш памет, примерно: G:, H: и др.

Текущото устройство е устройството, с което се работи в момента. Смяната на текущото устройство се извършва чрез изписване на неговото име командния ред: d:, e:, a:. Например:

C:\Documents and Settings\User>a:

A:\>

Текущото устройство вече е A:.

4.2.2 Файлове

Файлът е съвкупност от данни, записана върху носител на информация. Всеки файл си има име. Чрез него се осъществява достъп до файла. Името на файла се състои от две части. Първата част има дължина от 1 до 8 символа, а втората (разширение), незадължителна част до 3 символа. Двете части се разделят със символа точка (.). Точката е разделител, а не част от името. Символи невалидни в името на файл са: ., / \ [] : | < > + = ; , и всички символи с ASCII код по-малък от 20. Например: `readme.txt`, `autoexec.bat`, `command.com`, `gpl.txt`, `config.sys` и др.

4.2.3 Папки

За по-лесен достъп до файловете те са разпределени в т.н. папки (*folders*), наричани още директории (*directories*). Папката има име, което се определя както името на файл. Например: папка с име `docs` може да съдържа документи свързани с дейността на фирма: `guide.doc`, `manual.doc`, `info.doc` и др. Не е задължително името на папката да съответства на наличните в нея файлове.

4.2.4 Файлова организация

Всяка папка може да съдържа в себе си освен файлове и други папки. Чрез такова влагане на папки се образува т.н. дървовидна файлова структура. Родителска е тази папка, която съдържа подпапки. При създаването на всяка папка автоматично се създават две служебни папки `.` и `..`, които съдържат информация за текущата и родителската папка. Знакът `\` означава текущото устройство.

Последователността от папки, които MS-DOS трябва да следва, за да намери файл или папка се нарича път. При работа с текущата папка това указание не е необходимо.

Например следният път `D:\docs\post\today\manual.doc`, означава че файл `manual` има разширение `doc` и се намира в `D:` устройство, в папка `docs`, подпапка `post`, подпапка `today`.

Под термина *главна папка (директория)* на текущото устройство се разбира устройството, с което се работи в момента. Показване

съдържанието на главната папка, означава показване съдържанието (папки и файлове) на текущото устройство.

4.3 Често използвани команди на MS-DOS

4.3.1 TREE

С външната командата `tree`:
`tree [Устр:][Път] [/f] [/a]`

графично се показва дървовидната файлова структура, организирана в папки. Параметрите имат следните значения:

`/f` показва имената на файловете в съответните папки;

`/a` използват се текстови вместо графични символи за изобразяване на връзките между папките. Например при изпълнение на командата:

```
tree /a
```

част от резултата е следния:

```
Folder PATH listing for volume WIN
Volume serial number is 00008102 4088:0C69
```

```
C:.
```

```
+---WINDOWS
```

```
| +---system32
```

```
| | +---config
```

```
| | | \---systemprofile
```

```
| | | +---Start Menu
```

```
| | | | \---Programs
```

```
| | | | +---Accessories
```

```
| | | | | +---Entertainment
```

```
| | | | | \---Accessibility
```

```
| | | | \---Startup
```

```
| | | +---My Documents
```

```
| | | +---Favorites
```

```
| | | \---Desktop
```

```
| | +---drivers
```

```
| | | +---etc
```

```
| | | \---disdn
```

```
| | +---ras
```

Ясно се вижда дървовидното разположение на папките.
Стартирана със следния ред:

```
tree c: /f
```

част от изхода на командата е следния:

Folder PATH listing for volume WIN

Volume serial number is 4088-0C69

```
C:\
|   CONFIG.SYS
|   AUTOEXEC.BAT
|   log.txt
|   DataTEST.XLS
|   csb.log
|   test.jpg
|
+---WINDOWS
|   |   TDW.INI
|   |   win.ini
|   |   RtlRack.ini
|   |   _default.pif
|   |   explorer.scf
|   |   msdfmap.ini
```

извеждайки всички файлове от папките на устройство C:.

Извеждането може да се пренасочи към файл чрез използване на символ > и посочване на име на файл. Например:

```
tree > tr.txt
```

4.3.2 DIR

Преглед на съдържанието на папка (списък от файловете и папките), информация за големината на файловете в байтове, дата и час на последна модификация се извършва с вътрешната команда **dir** (*directory*). Извежда се още етикета на устройството, серийния му номер и името на текущата папка. Командата има следния синтаксис:

```
dir [Устр:][Път][ИмеНаФайл] [...] [/p] [/q] [/w]
    [/d] [/a[:атрибути]] [/o[:СортРед]]
    [/t[:ДатаПоле]] [/s] [/b] [/l] [/n] [/x]
    [/c] [/4]
```

Параметрите, които се използват по-често са следните:

/p списъкът се извежда екран по екран, като за всеки следващ екран се натиска произволен клавиш.

/w извежда се списък на файловете и папките – по пет на ред

/d също като параметър /w, но тук елементите са сортирани по колони.

Възможно е използването на т.н. глобални символи: * и ?. Символ ? замества който и да е символ от името или разширението на файла, в позицията в която е поставен. Например следващата команда ще изведе списък на файловете, които имат имена *info*, чието разширение е от три знака, и последните два са *xe*:

```
dir info.?xe
```

Символът * замества всички символи от позицията на която е поставен налясно в името на файла или разширението му. Например командата

```
dir *.asm
```

показва списък на всички файлове в текущата папка, които имат разширение *asm*. Записите *. * и ????????.??? са еквиваленти.

4.3.3 TYPE

Извеждането на съдържанието текстов файл, без възможност на модифицирането му се извършва с вътрешната команда **type**:

```
type [Устр:][Път][ИмеНаФайл]
```

За извеждане на съдържанието на файл *tr.txt* се използва следната команда:

```
type tr.txt
```

4.3.4 MORE

Извеждането на съдържанието на файл на екрана по страници се извършва чрез добавяне на символа | и външната команда **more**. Например:

```
type c:\windows\system.ini | more
```

Тази команда показва съдържанието на файл `system` с разширение `ini`, който се намира на устройство `C:` в папка `windows`. Преминването към следващата страница става с натискане на клавиша за интервал, а с натискане на клавиш `Enter` се преминава на следващия ред. Общият вид на командата е следният:

```
Команда | more [/c] [/p] [/s] [/tn] [+n]
more [[/c][/p][/s][/tn][+n]] <
[Устр:][Път]ИмеНаФайл
more [/c] [/p] [/s] [/tn] [+n] [Файлове]
```

Последният пример е еквивалентен на следния ред:

```
more < c:\windows\system.ini
```

4.3.5 MKDIR (MD)

Вътрешната команда `md` създава нова папка или подпапка (*make directory*). Тя има следния вид:

```
mkdir [Устр:][Път]
md [Устр:][Път]
```

Папките започват да се създават отляво надясно в посочения път от тази, която първа не съществува. Командният ред:

```
md c:\tasm\examples\msdos
```

ако не съществува в устройство `C:` папката `tasm`, ще я създаде, като след това ще създаде в нея подпапка `examples`, а в нея подпапка `msdos`. Ако в устройство `C:` съществува папката `tasm`, ще се създадат само останалите подпапки от посочения път.

Със следващият ред се създава папка `help` като подпапка на текущата папка.

```
md help
```

4.3.6 RMDIR (RD)

Тази команда премахва (*remove directory*), изтрива папки. Тя има следния вид:

```
rmdir [Устр:][Път] [/s] [/q]
rd [Устр:][Път] [/s] [/q]
```

Параметрите имат следните значения:

`/S` изтриване на определена папка заедно с нейните подпапки и включените файлове;

`/q` изтриване на папки без потвърждение от страна на потребителя.

Вмъквайки символ `\` пред папката, която ще се изтрива, то командата подразбира, че папката се намира в текущото устройство. Например, ако текущото устройство е `D:`, то следващата команда, ще изтрие папка `temp` намираща се в него:

```
rd \temp
```

Без символ `\` командата `rd` подразбира, че се изтрива подпапка на текущата папка. Например, ако текущата папка е `names` в `C:` устройство, то с командата:

```
rd temp
```

ще се изтрие нейната подпапка `temp`.

С командата `rd` не е възможно изтриване на текущата папка.

4.3.7 CHDIR (CD)

Вътрешната команда `cd` (*change directory*) се използва за смяна на текущата папка или показване на нейното име. Командата има следния синтаксис:

```
chdir [[/d] [Устр:][Път][...]] [[/d]
[Устр:][Път][...]]
cd [[/d] [Устр:][Път][...]] [[/d]
[Устр:][Път][...]]
```

Параметрите имат следните значения:

/d промяна на текущото устройство или текущата папка,
.. текуща папка става родителската папка.

Командата без параметри показва текущата папка. Смяната на текущата папка с главната папка в текущото устройство се извършва с използване на командата със следния синтаксис:

```
chdir [Устр:\[Папка]]  
cd [Устр:\[Папка]]
```

Например:

```
cd
```

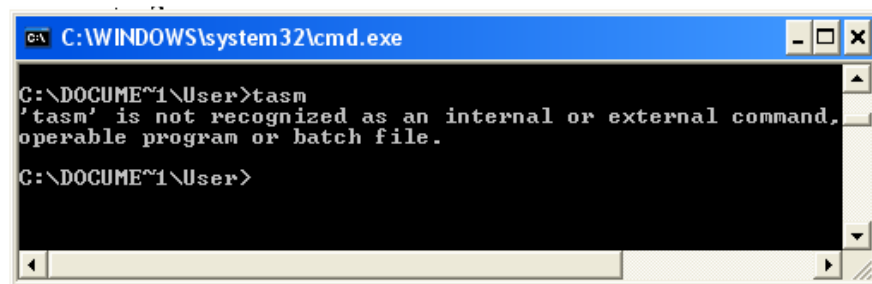
показва името на текущата папка. Командата

```
cd bin
```

променя текущата папка на bin, която е подпапка на текущата папка.

4.3.8 PATH

При изписване на команда на MS-DOS или на програма, която се намира в устройствата на компютърната система, но не се стартира, а се изпише съобщение за грешка, то командата (програмата) не може да се намери нито в текущата папка нито в допълнителния път за търсене. Например при опит за стартиране на програмата **tasm** се изписва следното съобщение:

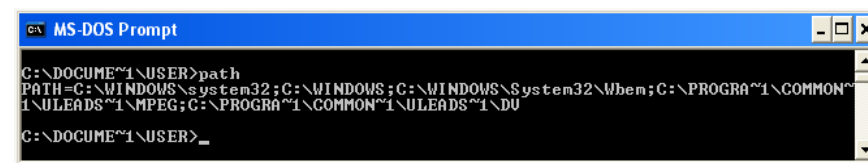


Фиг.4.6 Грешка при извикване на програма tasm

Допълнителният път за търсене са папките, които MS-DOS трябва да претърси, ако не намери командата (програмата) в текущата папка. Допълнителният път се задава с вътрешната команда path, която има следния вид:

```
path [[%path%][Устр:]Път[;...]]
```

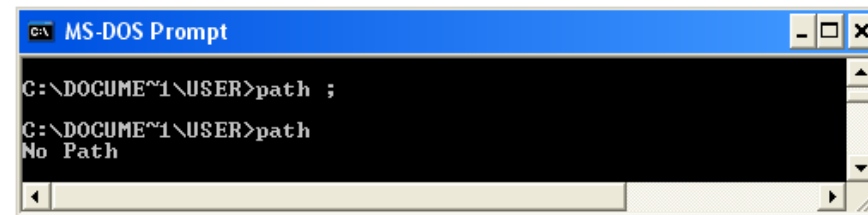
Тя установява допълнителен път в променливата PATH от средата, която е множество от папки използвани за търсене на изпълними файлове. При опит за стартиране на файлове с еднакви имена, но с различни разширения претърсването се извършва в следната последователност от разширения: .com, .exe, .bat, .cmd. Използвана без параметри path показва текущия команден път:



Фиг.4.7 Преглеждане на допълнителните пътища с PATH

В операционната система Windows XP командният процесор наследява този допълнителен път от реда Path, намиращ се в My Computer/Properties/Advanced/Environment Variables. Редактирането му може да се извърши с промяна в полето Variable Value, чрез избиране на реда Path и натискане на бутон Edit.

С параметър ; се разделят отделните пътища. Използването на ; като единствен параметър изтрива всички установени допълнителни пътища:



Фиг.4.8 Изтриване на допълнителните пътища с PATH

Последното важи за всеки стартиран команден процесор.

Използването на параметър `%path%` замества допълнителния път, наличен в променлива `Path`. По този начин се избягва писането на съществуващия път при добавяне на нов път към променливата `Path`. Следващият пример добавя път `c:\tasm\bin` към променливата `Path`:

```
path %path%;c:\tasm\bin
```

Примерът работи коректно в команден процесор `cmd.exe`, но не и в `command.com`.

4.3.9 COPY

Вътрешната команда `copy` копира един или повече файлове. Тя има следния синтаксис:

```
copy [/d] [/v] [/n] [{/y|/-y}] [/z] [{/a|/b}]  
    Източник [{/a|/b}] [+Източник [{/a|/b}]  
    [+ ...]] [Направление [{/a|/b}]]
```

Параметри **Източник** и **Направление** съдържат данни за местата от носителите на информация за източника и направлението за копиране на файл или множество файлове. Те включват име на устройство, път и име на файл. Например с реда:

```
copy help.txt newhelp.txt
```

файл `help.txt` се копира в текущата папка под името `newhelp.txt`.

Различните параметри на командата `copy` позволяват копирането на няколко файла в един файл, копиране на двоични файлове (`/b`), проверка за правилно копиране (`/v`) и др.

4.3.10 RENAME (REN)

Вътрешната команда **ren** позволява преименуване на файл или множество от файлове.

```
rename [Устр:][Път] ИмеНаФайл1 ИмеНаФайл2  
ren [Устр:][Път] ИмеНаФайл1 ИмеНаФайл2
```

Параметърът *ИмеНаФайл1* определя файла или множеството от файлове, които ще се преименуват. *ИмеНаФайл2* установява новото име на файла. Използването на глобални символи преименува множество от файлове, например:

```
ren *.cmd *.bat
```

Преименуването се извършва без значение дали зададените параметри са имена на файлове или папки. Например, командният ред:

```
ren slovo duma
```

преименува файла или папката от `slovo` на `duma`.

Командата `ren` не се изпълнява, ако *ИмеНаФайл2* вече съществува.

4.3.11 DEL (ERASE)

Тази вътрешна команда се използва за изтриване (*delete*) на файлове. Тя има следния синтаксис:

```
del [Устр:][Път] ИмеНаФайл [ ...] [/p] [/f]  
    [/s] [/q] [/a[:атрибути]]
```

```
erase [Устр:][Път] ИмеНаФайл [ ...] [/p] [/f]  
    [/s] [/q] [/a[:атрибути]]
```

С параметър `/p` се изисква от потребителя потвърждение за всеки изтриван файл (у за изтриване, п за отказ, `Ctrl+C` за прекратяване на работата на командата), с `/f` се изтриват файлове, които са само за четене, с `/s` се изтриват файлове от текущата папка и всички нейни подпапки, с `/q` не се потвърждава при изтриване на файлове. Параметър `/a` определя тези файлове за изтриване, които имат определени *атрибути*. Параметър

атрибути може да приема следните стойности: *r* – само за четене, *a* – архивен, *s* – системен, *h* – скрит, *-* – префикс за отрицание. Възможно е използването на глобални символи, например командата:

```
del *.obj
```

изтрива всички файлове с разширение *.obj* от текущата папка.

4.4 Въпроси

4.3.1 Опишете параметрите използвани от командния процесор *cmd.exe*. Покажете резултати от тяхното използване.

4.3.2 Опишете всички параметри, които не са обяснени в представените команди на MS-DOS. Предложете примери за работа с тези параметри. Покажете резултати от изпълнението на примерите.

4.3.3 Съберете информация за останалите команди в командния процесор *cmd.exe*. Покажете работата на всяка команда с примери за всеки от нейните параметри.

5 Създаване на работеща програма на асемблер

Програмата на асемблер представлява съвкупност от блокове памети, наречени сегменти от памети. Програмата може да се състои от един или няколко такива блока-сегменти. Синтактическите конструкции могат да бъдат инструкции (команди), макроинструкции (макрокоманди), директиви или коментари. За да може асемблерът да ги разпознава те са длъжни да се подреждат по определени синтактични правила.

5.1 Синтаксис на инструкциите на асемблер

5.1.1 Идентификатори

Идентификаторът е последователност от допустими символи, използвани за обозначение на обекти от програмата, като код на операцията, имена на променливи и названия на етикети. Идентификаторът може да се състои от един или няколко символа. В качеството на символи може да се използват латинските букви: A-Z, a-z, цифрите от 0 до 9 и някои специални знаци – *(.)* (само като първи символ), *(_)*, *(?)*, *(\$)*, *(@)*. Цифра, за начало на идентификатор, не се допуска.

Дължината на идентификатора може да бъде до 255 символа, но трансляторът на асемблер възприема само първите 32 символа, останалите се игнорират. Регулиране на дължината на идентификаторите може да стане с използване опцията на командния ред */mv*.

Съществува възможност да се укаже на транслятора на асемблер да различава малки и големи букви, а не да игнорира техните различия, както прави по подразбиране. Това се прави с опциите от командния ред */mi*, */ml*, */mx*.

5.1.2 Оператори

Оператор се нарича всеки изпълним израз. Всеки оператор е съставен от не повече от четири части. Те имат следната подредба:

[Етикет:] КОП [Операнди] [;Коментар]

Например:

```
start:  add  ax, bx    ;събиране
        mov  cx, 5
        nop
        ret
        jmp  part2
```

Идентификаторът **Етикет** дава име на конкретна клетка от паметта. Той е символично обръщение към адреса на инструкцията. Етикетите се преобразуват от асемблера в действителни адреси. При необходимост от обръщение към адреса на инструкцията, то това се прави чрез нейния етикет.

Единствената задължителна част на този оператор е КОП (*кодът на операцията*). Кодът на операцията в оператора на асемблер определя инструкцията (командата), която процесорът трябва да изпълни. В случая това са инструкциите **add** и **mov**.

Операндите, това са обекти (регистри, клетки от паметта), върху които действат командите или директивите. Тези обекти уточняват действието на командата или директивата. Те могат да определят началното значение на данните. Операндите биват следните видове:

-- непосредствени (постоянни, константни) - число, низ или израз, имащ някакво фиксирано значение; тези операнди се задават в инструкцията;

-- адресни - задават се с помощта на записа <сегмент>:<отместване>;

-- променливи - това са адреси от паметта, означаващи инструкции, ако операндът е етикет или данни, ако операндът е име на област от паметта в сегмента за данни; не са свързани с конкретен физически адрес; например:

```
msg db "Hello world", 0dh, 0ah, '$'
```

Променлива `msg` е свързана с област от паметта с размер от 14 байта. Пълният физически адрес от паметта ще бъде известен след зареждането на програмата в паметта за изпълнение.

-- адресен брояч - бележи се със знак \$; връща стойността на текущия адрес, намираща се в IP, например:

```
byte_block db 50 dup(?)
byte_block_lenght equ $-byte_block
```

```
scan_table db 10 dup (0,1,2,3,4,5,6,7,8,9)
scan_table_lenght equ $-scan_table
```

```
jmp $+3
```

-- регистрови - използват имената на регистрите – AX, BX, CX и т.н.;

-- базови и индексни – използват се за реализация на различни адресации;

-- структурни – използват се за достъп до структури;

-- записи – използват се за достъп до полета на записи.

Полето за коментар е незадължително поле. Използва се за пояснение на използвания алгоритъм на работа. За него може да се отдели и цял ред, като се запише (;) като първи символ на реда.

5.1.3 Директиви

Директивите на асемблер действат само в период на компилация на програмата и позволяват да се установи режима на компилация, задаването на сегменти в програмата, определянето на данните в програмата, обезпечаването на условната компилация и др. В резултат на обработка на директивите от компилатора не се генерира обектен код; директивите нямат машинно представяне.

Валиден е следният формат на директивите:

ИмеНаДир Директива Операнди [;Коментар]

Например:

```
include macro.inc
assume CS:code_segment
code_segment segment para 'CODE' ; code segment
insert_message db 'Insert symbol: ','$'
```


ИмеНаДир е идентификатор, който отличава директивата от други едноименни директиви.

5.1.4 Низови константи

Низовите константи започват с единична ' или двойна " кавичка и завършват със съответната кавичка. TASM преобразува символите в кавичките в ASCII значения.

```
"Hello World"
'Good Morning'
```

Използването на кавичка в низовата константа се извършва като преди съответната кавичката се запише отново същата такава:

```
'I'm a programmer' ; I'm a programmer
```

Дефинирането на низова константа може да се извърши на няколко реда. Например кодът

```
msg db "Hello World", 13d, 10d, '$'
```

е еквивалентен на кода

```
msg db "Hello "
      db 'World', 13d, 10d, '$'
```

5.2 Структура на асемблерската програма

TASM работи в два режима: MASM (по подразбиране) и IDEAL. Разширеният режим IDEAL съдържа средства облекчаващи процеса на програмиране.

Програмата на асемблер се състои от отделни програмни модули, съдържащи се в различни файлове. Всеки модул се състои от процесорни инструкции или директиви на асемблер и завършва с директивата **end** [НачаленАдрес]. НачаленАдрес съдържа идентификатор или израз, който определя адреса от който е задължително да започне изпълнението на програмата и се нарича входна точка на програмата. Например

end start.

Turbo Assembler игнорира всякакъв текст след директивата **end**.

5.2.1 Сегментационни директиви

Стандартни сегментационни директиви

Всеки модул също така се разбива на отделни части т.н. сегменти. Те започват със стандартната директива за начало на сегмента **segment** и завършват с директивата **ends**.

Директивата **segment** има следния вид:

segment	[Име]	[Подравняване]	[Свързване]	[Размер]	['Клас']
		byte	at xxxx	use16	
		word	common	use32	
		dword	memory		
		para	private		
		page	public		
			stack		

В режим **masm** име и **segment** се допуска да са с разменени места.

Указанието Подравняване определя границата на паметта откъдето трябва да започне сегмента: **byte** (байт), **word** (дума), **dword** (двойна дума), **para** (параграф – по подразбиране, **page** (страница).

Свързване определя как да бъдат свързани сегментите от различни модули имащи едно и също име. По подразбиране е **private**, т.е. сегментът не се обединява с други едноименни сегменти. При **public** всички сегменти с едно и също име се комбинират в един сегмент (еквивалентно с **memory**). Използването на **common** разполага всички сегменти с еднакви имена на един и същи адрес. Общият адрес е толкова голям, колкото е най-големият отделен сегмент. При използване на **at xxxx** сегмента се разполага по абсолютния адрес на параграфа зададен с изрази **xxxx**. Всички указания за преход и адреси на променливи се пресмятат спрямо адреса **xxxx**. Указанието **stack** съединява всички сегменти с това име като ги обединява в един общ сегмент. Регистър **SS** се инициализира с началото на сегмента **stack**, а **SP** с дължината на сегмента. Ако не е указан нито един

стеков сегмент, то се издава предупредително съобщение, че не е намерен такъв.

Указанията `Размер` се използва от процесорите 80386. Те установяват максималния размер на сегмента. `use16` установява 16 битов размер на сегмента, т.е. максимално 64 KB код или данни. `use32` установява 32 битов размер на сегмента, т.е. максимално 4 GB код или данни.

Класът на сегмента 'Клас' се поставя в кавички. Той спомага за определянето на реда на сегментите по време на свързване: сегментите с едни и същи имена 'Клас' се зареждат последователно в паметта. Класът на сегмента се изисква само когато асемблерната програма се включва в програмата на език от високо ниво.

Директивата `ends` има следния синтаксис:

```
ends [ИмеНаСегмент | ИмеНаСтруктура]
```

Например `ends cseg`.

Тази директива определя края на текущия сегмент, структура или обединение. Установеното `ИмеНаСегмент` или `ИмеНаСтруктура` трябва да съвпада с определеното име в свързаната `segment`, `struc` или `union` директива. В режим MASM е допустимо размяната на местата между директивата `ends` и нейните незадължителни параметри.

Директивите `segment` и `ends` не съдържат информация за функционалните назначения на сегмента. За да се използва той като сегмент за код, данни или стек е необходимо предварително да се съобщи на транслятора за това. За целта се използва стандартната директива `assume`, имаща следния синтаксис:

```
assume СегмРег:ИмеНаСегм [,СегмРег:ИмеНаСегм]
assume СегмРег:nothing
assume nothing
```

Тази директива съобщава на транслятора, кой сегмент с кой регистър е свързан. Това позволява на транслятора коректно да свързва символните имена, определени в сегментите. Например:

```
assume cs:code_segment, ds:data_segment
```

Тук регистър CS сочи сегмент с име `code_segment`, а DS - сегмент с име `data_segment`.

Директивата

```
assume ds:nothing
```

информира асемблера, че за програмата не е известно към кой сегмент сочи регистър `ds`.

Ако се използва само

```
assume nothing
```

то всички предишни назначения на сегментни регистри се анулират.

Опростени сегментационни директиви

При разработка на програми съдържащи по един сегмент за код, данни и стек съществува възможност за използване на опростени сегментационни директиви. Извършва се с директивата `model`, която служи за задаването на няколко стандартни модели на сегментация. Тя има следния синтаксис:

```
model [МодфНаМод] МодНаПаметта [ИмеНаСегм]
      [, [МодфНаЕзика] Език] [, МодфНаМод]
```

В режим MASM се използва същия синтаксис, но директивата има вида `.model`.

Директивата `model` частично управлява назначението на сегментите и изпълнява функциите на директива `assume`. `МодНаПаметта` (моделът на паметта) и `МодфНаМод` (модификаторът на модела) определят сегментационния модел на паметта използван в програмата.

В програми на асемблер се използват основно два вида типа преходи: близки (*near*) и далечни (*far*). Близки са вътрешно сегментните преходи, а далечни са междусегментните преходи.

Единственият задължителен параметър за директивата **model** е МодНаПаметта. В **Табл.5.1** са посочени използваните значения за параметър МодНаПаметта.

Табл.5.1 Модели на паметта

Модел	Тип на кода	Тип на данните	Регистри	Описание
tiny	близък	близък	cs=dgroup ds=ss=dgroup	Кодът и данните се комбинират в една група с името dgroup . Този модел се използва за програми в .com формат.
small	близък	близък	cs=_text ds=ss=dgroup	Кодът е в един сегмент. Всички данни са в група с името dgroup . Моделът се използва в повечето програми на асемблер.
medium	далечен	близък	cs=<модул>_text ds=ss=dgroup	За код се използват няколко сегмента – по един за всеки модул. Данните се намират в група с името dgroup .
compact	близък	далечен	cs=_text ds=ss=dgroup	Кодът се намира в един сегмент. Всички близки данни са в група dgroup . При работа с данни се използват далечни указатели.
large	далечен	далечен	cs=<модул>_text ds=ss=dgroup	За код се използват няколко сегмента, по един в модул. Всички близки данни са в група dgroup . При работа с данни се използват далечни указатели.

Модел	Тип на кода	Тип на данните	Регистри	Описание
huge	далечен	далечен	cs=<модул>_text ds=ss=dgroup	Еднакъв с модел large .
tchuge	далечен	далечен	cs=<модул>_text ds=nothing ss=nothing	Еднакъв с модел large , но с други назначения на сегментните регистри.
tpascal	близък	далечен	cs=code, ds=data, ss=nothing	Този модел поддържа ранните версии на Turbo Pascal. В по-късните версии не се използва.
flat	близък	близък	cs=_text ds=ss=flat	Еднакъв с модел small , но се използват 32 битови сегменти.

Определянето на модел на паметта с директива **.model** създава групата **dgroup**. В нея се обединяват сегментите създадени с директиви **data**, **data?**, **const** и **stack** (**Табл.5.2**).

Табл.5.2 Опростени сегментационни директиви

Формат на директивата (Ideal)	Формат на директивата (MASM)	Действие
codeseg [име]	.code [име]	Начало или продължение на кодовия сегмент.
dataseg	.data	Начало или продължение на сегмента за инициализираните данни.
udataseg	.data?	Начало или продължение на сегмента за неинициализираните данни.
const	.const	Начало или продължение на сегмента за константите.

Формат на директивата (Ideal)	Формат на директивата (MASM)	Действие
stack [размер]	.stack [размер]	Начало или продължение на стековия сегмент. Незадължителният параметър [размер] определя обема на резервираната памет за стека (в думи). По подразбиране се резервират 200h думи (1 kB).
fardata [име]	.fardata [име]	Определя началото на сегмента за инициализираните данни от далечен (<i>far</i>) тип. Името по подразбиране е <i>far_data</i> .
ufardata [име]	.fardata? [име]	Начало или продължение на сегмента за неинициализираните данни от далечен (<i>far</i>) тип.

Използването на параметър **МодфНаМод** позволява да се уточнят някои особености при използването на избрания модел памет (**Табл. 5.3**).

Табл.5.3 Модификатори на модела на паметта

МодНаМод	Функция
nearstack	Указва включването на сегмента на стека в <i>dgroup</i> и <i>ss</i> сочи <i>dgroup</i> .
farstack	Указва, че стека не е част от <i>dgroup</i> .
use16	Сегментите се използват като 16 битови (при избор на процесор 80386 или 80486).
use32	Сегментите се използват като 32 битови (при избор на процесор 80386 или 80486).
dos, os_dos	Указва, че програмата ще работи в MS-DOS.
os2, os_os2	Указва, че програмата ще работи в OS/2.

Използването на **Език** и **МодфНаЕзика** (модификатор на езика) определят съгласения използвани при извикване на процедури. Необходимостта от използване на тези параметри се появява при писане и извикване на програми на различни езици за програмиране. Турбо Асемблер автоматично генерира код за вход и изход за всяка процедура използваща едно от следните езикови съгласения: PASCAL, C, CPP (C++), SYSCALL, BASIC, FORTRAN, PROLOG и NOLANGUAGE (не се задава език). Ако не се зададе конкретен език, то асемблерът предполага използването на параметър NOLANGUAGE. МодфНаЕзика се използва за задаване на начало и край на процедури за Windows със стойности NORMAL, WINDOWS, ODDNEAR и ODDFAR. Ако не се зададе параметър, то Турбо Асемблер по подразбиране използва NORMAL.

След избора на модела на паметта с директивата **model** става възможна употребата на **опростените сегментационни директиви**, **Табл.5.2**. В един модул се указват толкова от тях колкото е необходимо. За получаване на един сегмент TASM комбинира всички части с еднакви имена.

При използване на директивата **model** и опростените сегментационни директиви трансляторът прави достъпни няколко идентификатора към които е възможно обръщение по време на работа на програмата, **Табл.5.4**.

Табл.5.4 Идентификатори, създадени от директива **model** и опростените сегментационни директиви

Име на идентификатор	Стойност
@model	Съдържа численото значение на текущо използвания модел на паметта: 1 = <i>tiny</i> , 2 = <i>small</i> или <i>flat</i> , 3 = <i>compact</i> , 4 = <i>medium</i> , 5 = <i>large</i> , 6 = <i>huge</i> , 7 = <i>thuge</i> , 0 = <i>tpascal</i> .
@32bit	0 = 16 битови сегменти, 1 = 32 битови сегменти.
@CodeSize	сегмент на кода: 0 = близък (<i>near</i>), 1 = далечен (<i>far</i>)

Име на идентификатор	Стойност
@DataSize	сегмент на данните: 0 = близък (<i>near</i>), 1 = далечен (<i>far</i>), 2 = огромен (<i>huge</i>)
@Interface	Число, чиито битове носят информация за използвани език и операционната система: 0 = <i>nolanguage</i> , 1 = <i>c</i> , 2, <i>syscall</i> , 4 = <i>pascal</i> , 5 = <i>fortran</i> , 6 = <i>basic</i> , 7 = <i>prolog</i> , 8 = <i>cpp</i> . 8 бит е 0 за MS-DOS и 1 за OS/2.
@code	Физически адрес на кодовия сегмент.
@data	Физически адрес на данновия сегмент от близък (<i>near</i>) тип.
@fardata	Физически адрес на данновия сегмент от далечен (<i>far</i>) тип.
@fardata?	Физически адрес на данновия сегмент за неинициализираните данни от далечен (<i>far</i>) тип.
@cureg	Физически адрес на текущия сегмент.
@stack	Физически адрес на стековия сегмент.

5.2.2 Процедури

Процедурата, или подпрограмата, е основната функционална единица за декомпозиция (разделяне на части) на произволна задача. Тя представлява група инструкции за решаване на конкретна подзадача и притежава средства за получаване на управлението от точката на извикване в задачата от по-висока степен и за връщане на управлението в тази точка. Най-простия случай първичната програма може да се състои от една процедура.

За описание на последователните инструкции във вид на процедура в асемблер се използват две директиви: PROC и ENDP:

```
[ИмеПроцедура] PROC [[МодфНаЕзика] Език] [Разстояние]
    [ARG СписъкАргументи] [RETURNS СписъкЕлементи];
    [LOCAL СписъкПроменливи] [USES СписъкЕлементи,]
```

```
[ИмеПроцедура] ENDP
```

МодфНаЕзика и Език действат по същия начин както в директивата MODEL. Разстояние може да е *near* или *far* и определя типа връщане на инструкцията RET, която би могла да се асемблира в края на процедурата.

С ARG се задава СписъкАргументи, които се предават посредством стека. Вътрешното им представяне е във вид на положително отместване от регистър BP или EBP. А с директива LOCAL се задават локалните променливи СписъкПроменливи за процедурата. Чрез оператор USES автоматично се генерира код за съхраняване и възстановяване на посочени регистри или единици от данни в СписъкЕлементи. RETURNS съдържа СписъкЕлементи за връщане.

Възможно е отделно написани и асемблирани процедури да се свържат в една главна програма с използване на програма TLINK. За тази цел се използват директиви EXTRN и PUBLIC. Директива EXTRN служи за обявяване на име като външно по отношение на даден модул. Същото име е обявено в друг модул чрез директива PUBLIC. Двете директиви имат следния синтаксис:

```
EXTRN име:тип, ..., име:тип
PUBLIC име
```

В ролята на идентификатор име е възможно използването на имена на променливи, определени с директиви DB, DW и др., имена на процедури и имена на константи, определени с оператори = и EQU. Аргументът **тип** определя идентификатора. Указването на типа е необходимо за правилното сформирание на съответната машина инструкция. Възможните значения на тип са: ако име е променлива, то тип може да приема значения BYTE, WORD, DWORD, PWORD, FWORD, QWORD и TBYTE.; ако име е име на процедура, то тип може да приема значения *near* или *far*.; ако име е име на константа, то тип трябва да е ABS.

5.2.3 Макро директиви

Макросите са блокове от асемблерен програмен код, започващ с директива MACRO и завършва с директива ENDM:

```
име MACRO [[формален-параметър, ...]]
    тяло на макроса
ENDM
```

Името на макроса е **име**, указано в директивата MACRO. То трябва да бъде уникално и може да се използва в изходния файл чрез извикване на макроса.

Фактическите параметри могат да бъдат имена, числа или други стойности. Те трябва да се поместят в един ред. Дефинирането на локални променливи, валидни само за макроса, става с помощта на директивата

```
LOCAL формално име, ...
```

която стои непосредствено след заглавието на макроса. За осигуряване на уникалността на променливата, определена в директива LOCAL, асемблерът я замества с име от вида ??номер, където **номер** е шестнадесетично число от 0000 до FFFF. Макросите не се асемблират, а се съхраняват като файлове, които се включват с директива:

```
INCLUDE име на файл
```

Цялото съдържание на файла с макроси се вмъква на мястото, където се постави директивата INCLUDE. При всяко срещане на макрос, подлежащ на изпълнение, асемблерът претърсва наличната макрос библиотека и поставя тялото на макроса на мястото на неговото извикване.

5.2.4 EXE и COM програми

MS-DOS може да зарежда в паметта и да изпълнява два типа файлове: EXE и COM. При зареждането им в паметта MS-DOS установява върху първите 256 байта на всяка програма префикса на програмния сегмент (*program segment prefix, PSP*). Той съдържа данни за правилното изпълнение на програмата, например параметри, написани от потребителя при стартиране на програмата от командния ред.

Програми от тип COM

Програмите от тип COM са едносегментни програми, в които кода, данните и стека се съхраняват в една област от 64 kB. Всички сегментни регистри (CS, DS, SS и ES) сочат началото на PSP. По тази причина началото на програма от тип COM (относно началото на PSP) е винаги на адрес 100h. За да се извърши отместването се използва директивата **org** израз, където израз може да бъде константа или отместване в текущия сегмент. С тази директива регистър IP получава стойността на израз. За COM програмите се използва следния команден ред:

```
org 100h
```

По този начин IP получава стойност 100h.

SP сочи края на PSP (обикновено 0FFFFh, но може и по-малко, ако пълния 64 kB сегмент е недостъпен).

Програмите от тип COM могат да съдържат инструкции за преход само от близък (*near*) тип, които имат адрес само на отместването, а не и на сегмента.

Препоръчително е при приключването на програмата да се използва функция 4Ch на MS-DOS, Табл.3.4.

Следващата програма демонстрира как се създава програма от тип COM в режим *Ideal* и използване на опростените сегментационни директиви. Програмата изписва съобщение *Hello World* на екрана., След това каретката се връща в началото на реда и се преминава на нов ред.
;prog51.asm

```
ideal
model tiny
codeseg
    org 100h
start:
    mov ah, 9h
    mov dx, offset msg
    int 21h
    mov ax, 4c00h
    int 21h
msg db "Hello World", 0dh, 0ah, '$'
```

```
end start
```

Асемблирането на програма се извършва с помощта на следната команда:

```
tasm prog51.asm
```

За свързването на програмата се използва следния синтаксис:

```
tlink /t prog51.obj
```

По този начин полученият файл ще е от тип COM: prog51.com.

Използването на опростените сегментационни директиви в режим MASM за създаване на програми от тип COM се извършва по следния начин:

```
;prog52.asm
```

```
masm
.model tiny
.code
    org 100h
start:
    mov dx, offset msg
    mov ah, 9h
    int 21h
    mov ax, 4c00h
    int 21h
msg db "Hello World", 0dh, 0ah, '$'
end start
```

И в двата листинга е възможно декларирането на данните да се направи преди кода на програмата, но тази декларация трябва да се прескочи с команда `jmp`. Например:

```
;prog53.asm
```

```
ideal
```

```
model tiny
```

```
codeseg
    org 100h
start:
    jmp to_code
msg db "Hello World", 0dh, 0ah, '$'
to_code:
    mov dx, offset msg
    mov ah, 9h
    int 21h
    mov ax, 4c00h
    int 21h
end start
```

При използване на стандартните сегментационни директиви рамката на програма от тип COM има следния вид:

```
;prog54.asm
```

```
ideal
segment code
    org 100h
    assume cs:code, ds:code, ss:code, es:code

start:
    mov ah, 9h
    mov dx, offset msg
    int 21h
    mov ax, 4c00h
    int 21h
msg db "Hello World", 0dh, 0ah, '$'
ends code

end start
```

Програма prog54.asm е в режим `ideal`. Преобразуването ѝ в режим MASM се извършва чрез премахване на директивата `IDEAL` или нейното заместване с директива `MASM`. Превключването между

режими MASM и IDEAL в изходния код на програмата може да стане на произволно място чрез включването на съответните директиви в изходния файл.

Програми от тип EXE

Изпълнимите файлове от тип EXE ;съдържат различни сегменти за код, данни и стек, които могат да бъдат организирани в произволна последователност. При тях отсъства ограничението от 64 kB за големина на програмата. Изпълнимите файлове съдържат данни, необходими за определяне на тези адреси, които не могат да се определят преди зареждането на програмата в паметта. DS и ES сочат началото на PSP. Регистри CS, IP, SS и SP се инициализират със значения, указани в началния блок на .EXE файла.

Следващият пример илюстрира рамката на програма от тип EXE, в режим `ideal`, изпълнена със стандартните сегментационни директиви.

```
;prog55.asm

ideal
segment stackseg stack
    db 256 dup (?)
ends stackseg

segment data
    msg db "Hello World", 0dh, 0ah, '$'
ends data

segment code
    assume cs:code, ds:data, ss:stackseg
start:
    mov ax, data
    mov ds, ax
    mov dx, offset msg
    mov ah, 9h
    int 21h
    mov ax, 4C00h
    int 21h
ends code
```

end start

Асемблирането на програмата става със следните редове:

```
tasm prog55.asm
tlink prog55.obj
```

Получава се изпълнимата програма `prog55.exe`.

Горната програма съдържа три сегмента: сегмент за стека с име `stackseg`, сегмент за данните от програмата с име `data` и сегмент за кода на програмата с име `code`. При премахване на директивата `ideal` програмата преминава в режим `masm`.

С използването на опростените сегментационни директиви горния шаблон за EXE програма придобива следната форма в режим `masm`:

```
;prog56.asm

.model small
.stack 100H
.data
    msg db "Hello World", 0dh, 0ah, '$'
.code
start:
    mov ax, @data
    mov ds, ax
    mov dx, offset msg
    mov ah, 9h
    int 21h
    mov ax, 4C00h
    int 21h
end start
```

В режим `ideal` програмата от тип EXE има следната рамка:

```
;prog57.asm

ideal
```



```

model small

stack 100h

dataseg
    msg db "Hello World", 0dh, 0ah, '$'

codeseg
start:
    mov ax, dgroup
    mov ds, ax
    mov dx, offset msg
    mov ah, 9h
    int 21h

    mov ax, 4c00h
    int 21h
end start

```

5.2.5 Структура на командите, типове, основен цикъл

По време на оперативния си живот централният процесор непрекъснато изпълнява поток от команди (или инструкции). Инструкциите се представят като поредица от байтове и могат да бъдат с фиксирана или променлива дължина.

Инструкцията се състои от две части – код на операцията (КОП) и операнди.

КОП A1, A2, A3

КОП е двоичният код на операцията, а A1, A2, A3 са операнди. Операндите могат да бъдат – регистри, адреси, числа.

Операциите биват няколко вида:

- аритметико-логически - изчисляват булевите функции – AND, OR, NOT, операциите за сравнение, и аритметичните (събиране, изваждане, умножение, деление, модул),

- управляващи - инструкции, когато процесора е в привилегирован режим (например когато чете или пише в служебните регистри).
- за преход – променят последователното изпълнение. Включват инструкции за условен/безусловен преход и за извикване на подпрограми.
- транспортни - прехвърляне на данни. Могат да бъдат следните (Табл.5.1):

Табл.5.5 Прехвърляне на данни

<i>регистър – регистър</i>	<i>регистър – адресация памет</i>
<i>регистър – памет</i>	<i>памет – непосредствен операнд</i>
<i>памет – памет</i>	<i>едноадресна операция</i>

Инструкциите се изпълняват от ЦП по следния начин (основен цикъл на изпълнение на командата):

- извлича се команда от ОП
- дешифрира се КОП
- извлича се операнда(ите)
- изпълняват се командата върху операнда(ите)
- запазва се резултата
- преминава се към следващата инструкция

5.3 Получаване на изпълнима програма

Получаването на изпълнима програма написана на асемблер започва със създаването на първичен ASCII файл с кода на програмата написана на асемблер, преминава през асемблирането на програмата и завършва със свързването на различните програмни модули в изпълним файл.

5.3.1 Инсталиране на асемблер

Инсталирането на пакет Borland Turbo Assembler 5.0 започва със стартиране на командния процесор cmd.exe (command.com) и набирането в командния ред на името на инсталационния файл INSTALL, последвано от натискането на Enter. Следва натискане на Enter за преминаване през показания начален прозорец. Появява се въпрос за въвеждане на име на устройството където се

намира софтуерния пакет, обикновено това е устройство C: или D:. В следващия прозорец се въвежда пътя до папката на дистрибутива. В последния прозорец се избира C:\TASM, папката където ще се инсталира TASM; C:\WINDOWS или C:\WINXP, папката където е инсталирана операционната система Windows; всички останали менюта са с опция Yes. Преминаването по менюто се извършва със стрелките от клавиатурата ↑ и ↓.

Стартирането на същинската инсталация се извършва от менюто Start Installation или от клавиатурата с клавиш F9.

5.3.2 Набиране на текста на програма на асемблер

С помощта на текстов редактор програмата на асемблерен език се набира в код ASCII и се записва с разширение ASM. Това може да се извърши в средата на Microsoft Visual Studio. След стартиране на приложението се избира меню File/New. От отворения прозорец New се избира панел Files, в него Text File и накрая се натиска бутон OK. След отварянето на празен файл неговото име с разширение ASM се уточнява с менюто File/Save As... От там се избира и съответна папка, в която ще се съхранява файла.

5.3.3 Асемблиране на първичен файл

Асемблирането на първичния файл с разширение ASM е извършване на транслацията му посредством асемблера `tasm.exe` до получаването на обектен изходен файл с разширение OBJ. Обектният файл съдържа първичната програма в машинен код, информация необходима за свързването му с други модули, информация за настройка на програмата.

Опции на асемблера TASM

Асемблерът `tasm` се намира в папка `c:\tasm\bin`. Общият синтаксис на `tasm` е следния:

```
tasm [аргументи] ПървиченФ [ ,ОбектенФ] [ ,ЛистингФ]
      [ ,СправоченФ]
```

ПървиченФ е името на първичния файл, в който е текста на програмата написана на асемблерен език. Разширението ASM не е задължително. ОбектенФ е името на обектния файл, ЛистингФ е

името на листинга на програмата и е с разширение LST, а СправоченФ е името на справочния файл и е с разширение XRF. При условие, че името на първичния файл съвпада с имената на другите три файла, то е възможно изписването на следния команден ред:

```
tasm prog56 , , ,
```

Изходът от командата е генериране на три файла: `prog56.obj`, `prog56.lst` и `prog56.xrf`.

По подразбиране употребата само на ПървиченФ води до създаването само на обектен файл. Включването или изключването на някой от ОбектенФ, ЛистингФ или СправоченФ става с употребата на думата `nul` след съответната запетая. Следващата употреба на асемблера `tasm`:

```
tasm prog56 , , ,nul
```

води до създаването на обектен файл `prog56.obj` и листинг `prog56.lst`.

Често използвани аргументи са:

/с – изготвяне на справочен списък в листинга, съдържащ азбучен указател на всички срещани в програмата променливи, етикети и символи, с номерата на редовете в които са дефинирани и използвани. Същата информация се съдържа и в справочния файл.

/l, /la – указва създаването на /l – обикновен листингов файл или /la – разширен листингов файл, за връзка с езици от високо ниво при употреба на директива `.model`.

/x – включване в листинга на блоковете за условно асемблиране, дори те да не се изпълняват.

/z – извеждане на сгрешения израз заедно със съобщението за грешката.

/zd – включване в обектния файл информация за номерата на редовете от първичния файл.

/zi – включване в обектния файл пълна информация за настройка на програмата.

/zn – забрана за включване на информация необходима за настройка на програмата.

В командния процесор се изписва информация за номера на версията на пакет TASM, който се използва за трансляция на първичния файл. На следващия ред е изписано името на асемблирания файл. Ако първичната програмата съдържа грешки, то трансляторът изписва на екрана низове със съобщения започващи с думите **Error** (грешка) и **Warning** (предупреждение).

От наличието на дума Error, следва че в програмата има недопустими изрази от гледна точка на синтаксиса.

Наличието на Warning означава, че изразите са синтактично правилни, но не съответстват на някои от езиковите правила и това може да бъде източник на бъдещи грешки.

За отстраняване на грешка е необходимо да се определи мястото на нейното възникване и да се анализира отново съответния израз. Местоположението на грешката се установява по значението в скобите на съобщението за грешка. Това значение е номера на грешния ред. За локализация на грешка се използва информацията от листинговия файл. Той съдържа асемблерния код на първичния файл. За всяка инструкция е указан машинния й код и отместването й в сегмента. Освен това се формира таблица с информация за етикетите и сегментите използвани в програмата. Ако има грешки или предупредителни съобщения, то **tasm** включва информация за тях на следващия ред от тяхното срещане и в края на листинга. Полученият по-горе листингов файл **prog56.lst** има следното съдържание:

```
Turbo Assembler  Version 4.1      08/11/06 11:43:12
Page 1
prog56.asm
```

```
1                ;prog56.asm
2
3 0000            .model small
4 0000            .stack 100H
5 0000            .data
6 0000  48 65 6C 6C 6F 20 57+          msg  db  "Hello
World", 0dh, 0ah, '$'
7            6F 72 6C 64 0D 0A 24
8 000E            .code
9 0000            start:
10 0000  B8 0000s          mov  ax, @data
11 0003  8E D8            mov  ds, ax
12 0005  BA 0000r          mov  dx, offset msg
13 0008  B4 09            mov  ah, 9h
14 000A  CD 21            int  21h
15 000C  B8 4C00          mov  ax,4c00h
16 000F  CD 21            int  21h
17                end start
```

```
Turbo Assembler  Version 4.1      08/11/06 11:43:12
Page 2
Symbol Table
```

Symbol Name	Type	Value	Cref(defined	at
#)				
??DATE	Text	"08/11/06"		
??FILENAME	Text	"prog56 "		
??TIME	Text	"11:43:11"		
??VERSION	Number	040A		
@32BIT	Text	0	#3	
@CODE	Text	_TEXT	#3	#3 #8
@CODESIZE	Text	0	#3	
@CPU	Text	0101H		
@CURSEG	Text	_TEXT	#5	#8
@DATA	Text	DGROUP	#3	10
@DATASIZE	Text	0	#3	
@FILENAME	Text	PROG56		
@INTERFACE	Text	000H	#3	
@MODEL	Text	2	#3	
@STACK	Text	DGROUP		#3
@WORDSIZE	Text	2	#5	#8
MSG	Byte	DGROUP:0000	#6	12

```
START      Near  _TEXT:0000      #9  17
```

Groups & Segments Bit Size AlignCombineClass
Cref (defined at #)

DGROUP	Group	#3	3	10				
STACK	16	0100	Para	Stack	STACK	#4		
_DATA	16	000E	Word	Public	DATA	#3	#5	
_TEXT	16	0011	Word	Public	CODE	#3	3	#8 8

Ясно се вижда, че директивите не генерират машинен код.

За нормалното завършване на процеса на асемблиране се смята това без грешки и предупредителни съобщения.

Асемблиране с TASM и Microsoft Visual Studio

Асемблирането е възможно да се извърши с използване на средата Microsoft Visual Studio. За целта се избира команда **Tools/Customize** и панел **Tools** от появилия се диалогов прозорец. Всеки ред в списъка **Menu contents** е свързан с набор от параметри за извикването на външна програма. Последният елемент от списъка винаги е празен. Там е възможно добавянето на нови програми.

За създаването на команда извикваща програмата **tasm.exe** е нужно да се въведе в последния елемент от списъка нейното название, например **Build ASM**. Отварянето на реда за въвеждане се извършва с двойно щракване в него или с избор на бутон **New**. Завършва се с натискане на клавиш **Enter**.

В полето **Command** се въвежда **c:\tasm\bin\tasm.exe**. Полето **Arguments** се запълва с низа **/c /l \$(FileName).asm**. В полето **Initial directory** се изписва **\$(FileDir)**. Избира се отметката **Use Output Window**. Накрая с бутон **Close** се затваря диалоговия прозорец **Customize**. В резултат от тези действия се появява подменю **Tools/Build ASM**.

Работоспособността на създаденото подменю се проверява чрез асемблиране на произволен отворен файл в средата на Microsoft Visual Studio. Например, при отваряне на файл **prog56.asm** и избиране на подменюто **Tools/Build ASM** в прозорец **Output Window** се отваря панел **Build ASM** със следния текст:

```
Turbo Assembler Version 4.1 Copyright (c) 1988,
1996 Borland International
Assembling file: prog56.asm
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 412k
Tool returned code: 0
```

Освен това в папката, където е записан файл **prog56.asm**, в резултат от асемблирането се създават файлове **prog56.obj** и **prog56.lst**.

5.3.4 Свързване на обектните модули

След отстраняване на грешките и получаване на обектния файл се пристъпва към създаването на изпълнимия файл тип **EXE** или **COM**. Той е комбинация между различни програмни модули и се създава с помощта на **TLINK**. Разделението между двата процеса асемблиране и свързване е с цел обединяване на модули написани на различни програмни езици.

Опции на свързващата програма TLINK

Свързващата програма **TLINK** има следния синтаксис:

```
tlink [аргументи] СписъкОбектниФ [,ИзпълнимФ]
      [,КартенФ] [,БиблиотечниФ] [,ДефиниционенФ]
      [,РесурсенФ]
```

СписъкОбектниФ е задължителен параметър, съдържащ списък обектни файлове с разширение **OBJ**. Файловете се разделят с интервал или знак **+** (плюс). Например:

```
tlink main + proc1
```

ИзпълнимФ е незадължителен параметър, обозначаващ името на изпълнимия файл, който се цели да се получи. Ако не е указано, то това име по подразбиране съвпада с първото име от **СписъкОбектниФ**.

КартенФ е файл съдържащ всички имена, сегменти и техните относителни адреси; записват се и грешки при свързването. Типична грешка е неправилното определяне на стековия сегмент. Полученият по-долу картен файл prog56.map има следното съдържание:

Start	Stop	Length	Name	Class
00000H	00010H	00011H	_TEXT	CODE
00020H	0002DH	0000EH	_DATA	DATA
00030H	0012FH	00100H	STACK	STACK

Program entry point at 0000:0000

БиблиотечниФ представлява път към файлове библиотеки с разширение LIB. Тези файлове се създават и обслужват с програма tlib.exe от пакета TASM.

ДефиниционенФ е път към дефиниционен файл с разширение DEF. Използва се за получаване на Windows приложения.

РесурсенФ е незадължителен параметър, който е път към Windows ресурсен файл с разширение RES.

Често използвани са следните аргументи: /X – не се създава картен файл; /m – създава се картен файл с включени имена от тип public; /t – създава изпълними командни файлове с разширение COM (по подразбиране се създават EXE файлове; /v – включва символна информация, необходима за настройката на изпълнимия файл.

Например за получаване на изпълнимия файл prog56.exe и картния файл prog56.map от prog56.obj е необходимо да се извика tlink.exe със следните параметри:

```
tlink /v prog56.obj
```

Свързване с TLINK и Microsoft Visual Studio

При условие, че е наличен готов обектен модул е възможно използването на средата Microsoft Visual Studio, където в панел Tools от Tools/Customize е възможно да се добави елемент Link OBJ. В полето Command се изписва c:\tasm\bin\tlink.exe, в Arguments се въвежда

\$(FileName), а в Initial directory \$(FileDir); избира се отметката Use Output Window. Завършва се с натискане на бутон Close.

След избиране на създаденото подменю Tools/Link OBJ се създават изпълним файл prog56.exe и картен файл prog56.map. В прозорец Output Window се отваря панел Link OBJ и в него се появява следната информация:

```
Turbo Link Version 7.1.30.1. Copyright (c) 1987,
1996 Borland International
Tool returned code: 0
```

5.3.5 Стартиране на изпълним файл

Стартиране в команден процесор

Стартирането на готовия изпълним файл prog56.exe може да се извърши чрез изписване на неговото име в командния процесор и натискане на Enter. Изписва се съобщението Hello World.

Стартиране в команден процесор от средата на Microsoft Visual Studio

За да се извърши това е необходимо в панел Tools от Tools/Customize да се създаде елемент Run. В полето Command се изписва c:\windows\system32\cmd.exe, в Arguments се изписва /c \$(FileName), а в Initial directory \$(FileDir). Стартирането на командния процесор cmd.exe с параметър /C води до изпълнението на подадената към него команда (програма) с последващо приключване на работата от него.

Отстраняването на синтактичните грешки не гарантира, че програмата ще върши това за което е предназначена. Етап от процеса на разработка на програмата е процеса на нейното тестване.

5.3.6 Средства за тестване и настройка

Тестване с програмата Turbo Debugger

Програмата Turbo Debugger (TD) позволява да се определи мястото на логическата грешка и нейната причина. Тя притежава

следните възможности: трасировка на програмата в права и обратна посока (последователно изпълнение на машинните инструкции в права и обратна посока) и преглед и изменение на ресурсите на процесора по време на трасирането.

TD няма възможност да внася изменения в изпълнимия код (EXE или COM) на тестваната програма. Изменения могат да се внасят само по време на работа на TD. С тези изменения програмата е възможно отново да се зареди и изпълни. След завършване на работа измененията не се записват в изпълнимия код на програмата. За да се извърши това е необходимо те да се внесат в асемблерския код и отново да се премине през стъпки асемблиране и свързване.

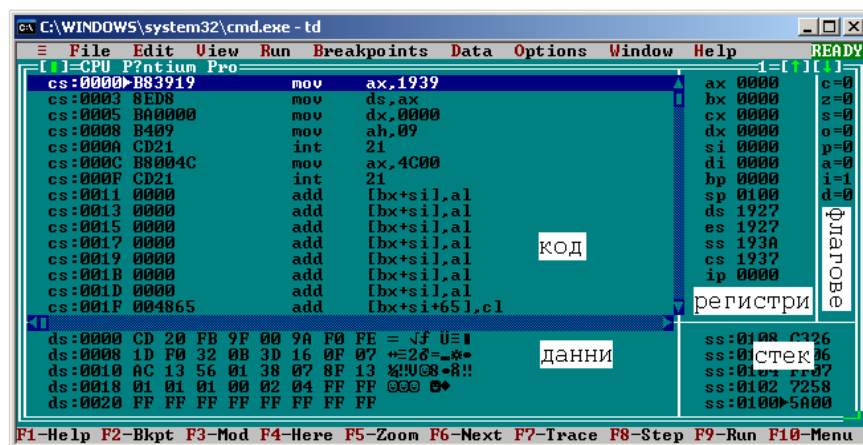
TD има следния синтаксис:

```
td [аргументи] [ИзпълнимФ [аргументиФ] ]
```

ИзпълнимФ е името на изпълнимия файл, който ще се тества с TD. Например:

```
td prog56
```

В резултат от горната команда се отваря прозореца на TD със следния вид:



Фиг.5.1 Основен прозорец на програмата Turbo Debugger

Във всеки един момент активен може да бъде само един подчинен прозорец от тези, които са по подразбиране: за код, данни, регистри на процесора, стек и флагове.

Съществуват два вида менюта в TD:

-- главно меню – активизира се с клавиш F10; със стрелките се избира кое меню и подменю ще се избере;

-- контекстно зависимо меню – активизира се в активен прозорец на TD с клавишна комбинация Alt+F10 (или десен бутон на мишката в съответния прозорец).

Забелязва се т.н. курсор за изпълнение (във вид на триъгълник ►), който сочи текущата инструкция подлежаща на изпълнение. В случая курсора за изпълнение сочи входната точка на програмата отбелязана с етикет start (посочен в директива end).

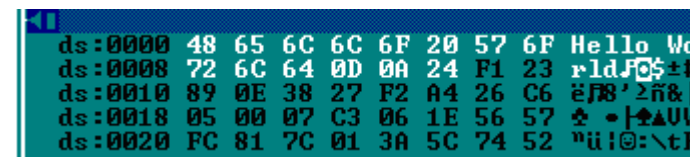
Стартирането на изпълнима програма в средата на TD се извършва с команда Run/Run или клавиш F9.

Правилността на програмата се проверява чрез трасирането ѝ през всяка инструкция с клавиш F7 (меню Run/Trace into) или с клавиш F8 (меню Run/Step over), където извикването на процедури или прекъсвания се обработва като една команда.

След двукратно натискане на клавиш F7 се преминава на ред за изпълнение:

```
cs:0005►BA0000          mov     dx, 0000
```

В този момент регистър DS вече се намира началото на сегмента за данни и е възможно в прозореца за данните да се наблюдават декларираните данни. За целта е възможно с клавиш Tab да се премине в прозореца за данните. От контекстно зависимото меню се избира Goto... и в отворения прозорец Enter address to position to се изписва началния адрес на данните, а именно ds:0. За потвърждение се натиска бутон OK. Сега в прозореца за данни може да се наблюдава стойността на променливата msg.



Фиг.5.2 Текст Hello World в програмата Turbo Debugger

След изписването на екрана съобщението `Hello World` е възможно резултата от изпълнението да се провери чрез клавишната комбинация `Alt+F5` (меню `Window/User screen`). Връщането към тестването на изпълнимата програма е с натискане на произволен клавиш.

По време на трасирането стойностите във всички прозорци се изменят динамично спрямо текущото изпълнение на инструкциите. Променените стойности се оцветяват в бяло.

Извикването на TD може да се извърши от средата Microsoft Visual Studio. За целта в панел `Tools` от `Tools/Customize` е необходимо да се създаде елемент `Debug EXE`. В полето `Command` се изписва `c:\tasm\bin\td.exe`, в `Arguments` се изписва `$(FileName)`, а в `Initial directory` `$(FileDir)`. Появява се менюто `Tools/Debug EXE`.

Изходът от програмата TD е с клавишна комбинация `Alt+X` (меню `File/Quit`).

В кода на отворената изпълнима програмата може да се поставят точки на прекъсване (*break points*). При избор на команда `Run` изпълнението на програмата се преустановява временно в тази точка. Така се дава възможност на програмиста да наблюдава поведението на регистрите и др. точно в определен момент. Поставянето и премахването на точка на прекъсване се извършва от менюто `Breakpoints/Toggle` или с клавиш `F2`.

6 Основи на програмирането в MS-DOS

6.1 Изход на екрана в текстов режим

6.1.1 Средства на DOS

В разгледаните програми от `prog51.asm` до `prog57.asm` изходът на екрана се извършва с функция `09h` на `21h` прекъсване на DOS. Проверката за натискане на клавишната комбинация **`Ctrl+Break`** означава, че при засичане на такава по време на извеждането на низа на екрана се извиква прекъсване `23h`, което по подразбиране извършва изход от програмата.

Съществуват и други способи за изход на екрана, а именно това са посочените в **Табл.3.4** функции. Най-близката от тях до `09h` е `40h`. Тя е предназначена за запис във файл, но при зареждане на `BX` с 1, то изходът ще е на `stdout`, а при `bx = 2` изходът ще е на `stderr`. Функция `40h` може да изведе символ за долар (\$) на екрана:

`prog61.asm`

```
.model small
.data
message db 'Hello World $'
message_length equ $-message

.code
start:
    mov ax, @data
    mov ds, ax
    mov dx, offset message ; DS:DX - адрес на низа
    mov cx, message_length ; CX - дължина на низа
    mov ah, 40h             ; зареждане на функция
    mov bx, 1               ; изход в stdout
    int 21h

    mov ah, 4ch
    int 21h
end start
```

Изчисляването на дължината на низа за извеждане се извършва с адресен брояч.

С функции 09h и 40h е възможно извеждането и на единични символи. Същото може да се направи и с функции 02h и 06h. Следващата програма използва 02h функция на 21h прекъсване на DOS за показване на низ 'asm':

```
;prog62.asm
```

```
.model small
.code
```

```
start:
mov dl, 'a'
mov ah, 02h
int 21h
```

```
mov dl, 's'
mov ah, 02h
int 21h
```

```
mov dl, 'm'
mov ah, 02h
int 21h
```

```
mov ah, 4ch
int 21h
end start
```

С тези две функции е възможно обработването на управляващите знаци BEL, BS, LF, CR. Замяната на функция 02h с 06h ще доведе до същия резултат.

Показването на всички символи от ASCII/IBM системата от знаци във възходящ ред се извършва с помощта на инструкция **LOOP** и **INC**:

```
;prog63.asm
```

```
.model small
```

```
.code
```

```
start:
mov cx, 256d ; брой повторения
mov dl, 00h
repeat:
mov ah, 02h
int 21h
inc dx ;или inc dl
loop repeat
```

```
mov ah, 4ch
int 21h
end start
```

6.1.2 Средства на BIOS

Функциите на DOS за изход на екрана не позволяват показването на знаци в произволна позиция и с атрибути (цвят на знака, цвят на фона, мигане). За целта се използват видео функциите на BIOS извиквани с прекъсване 10h:

Табл.6.1 Често използвани функции на прекъсване 10h на BIOS

Функция	Вход	Действие	Изход
00h	ah = 00h al = видеорежим 00 40x25 черно-бял текст 01 40x25 16 цвята текст 02 80x25 16 отенъка на сивото текст 03 80x25 16 цвята текст 04 320x200 4 цвята графика 05 320x200 4 отенъка на сивото графика 06 640x200 черно-бяла графика 07 80x25 монохромен текст	Задаване на видео режим. Цветовете 04, 05, 06 се установяват с функция 11.	--
02h	ah = 02h bh = номер страница dh = номер ред (0-24) dl = номер колона (0-79 или 0-39) Невидим курсор – преместен на несъществуваща позиция.	Установяване позицията на курсора.	--

Функция	Вход	Действие	Изход
03h	ah = 03h bh = номер на видео страница	Четене на позицията на курсора.	dh, dl = ред и колона ch и cl = начална и крайна позиция на мигация курсор
05h	ah = 05h al = номер видеостраница	Дефиниране на текущата видеостраница.	--
08h	ah = 08h bh = номер на видео страница	Четене на символ и неговия атрибут	al = ASCII код на символа ah = атрибут
09h	ah = 09h bh = номер на видео страница cx = брой повторения al = ASCII код на символа bl = атрибут	Запис на символ и неговия атрибут	--
0Ah	ah = 0ah al = ASCII код на символа bh = номер на видео страница bl = атрибут на символа в графичен режим cx = брой повторения	Запис на символ (с цвета на стария символ)	--
0Eh	AH = 0Eh AL = ASCII код на символа BL = цвят на фон на символа (само за графичен режим)	Запис на символ (с цвета на стария символ).	--
13h	ah = 13h al = изходен режим (0 – атрибут в BL, запазване позицията на курсора, 1 – атрибут в BL, презапис позицията на курсора, 2 – атрибути в низа, запазване позицията на курсора, 3 – атрибути в низа, презапис позицията на курсора) bl = символен атрибутен байт cx = брой символи dh, dl = екранен ред и колона bh = видео страница es:bp = начален адрес на низа	Извеждане на низ със зададени атрибути. Управляващите символи BEL, BS, LF, CR се обработват.	--

Много BIOS функции изискват като параметър координатите на екрана. В текстов и графичен режим началото на координатната система е горния ляв ъгъл на екрана. Придвижването на дясно увеличава X-координатата, докато придвижването надолу увеличава Y-координатата. В текстов режим 80x25, долния десен ъгъл има координати 79, 24, а при графичния 320x200, долния десен ъгъл е с координати 319x199.

Видео страницата е изображение в паметта на това, което може да бъде показано на екрана. В текстов режим с 80 позиции в ред се използват 4 видеостраницы (от 0 до 3), а в режим от 40 позиции – 8 видео страници (от 0 до 7).

Всеки символ се описва с два байта: байт за ASCII код и байт за атрибути. Битовете, определящи атрибутите, изпълняват следните условия: 0, 1, 2 – цвят на знака (символа), 3 – яркост (интезивност) на цвета, 4, 5, 6 – цвят на фона, 7 – мигане (или яркост на фона). Избира се следните цветове в текстов режим:

Табл.6.2 Видове цветове в текстов режим

Двоични числа	Цвят	Двоични числа	Цвят
0000	черно	1000	тъмно-сиво
0001	синьо	1001	светло синьо
0010	зелено	1010	светло-зелено
0011	сиво	1011	светло-сиво
0100	червено	1100	светло-червено
0101	пурпурно	1101	светло-пурпурно
0110	кафяво	1110	жълто
0111	светло-сиво	1111	бяло

Следващият пример изписва пет пъти знак “а” от позиция 10,10 със светло-червен цвят върху светло-зелен фон. Накрая премества курсора на екрана след изписания текст.

```
; prog64.asm
.model small
```

```
.code
start:
```

```

xor bh, bh
mov dh, 10
mov dl, 10
mov ah, 02h
int 10h                ; позиция на курсора 10,10

mov al, "a"
mov bl, 10101100b
mov cx, 5              ; брой символи за запис
mov ah, 09h
int 10h

xor bh, bh
mov dh, 10
mov dl, 15
mov ah, 02h
int 10h                ; позиция на курсора 15,10

exit:
    mov ah, 4ch
    int 21h
end start

```

6.1.3 Пряк достъп до видео паметта

За да се появи изображение на екрана (текст или графика) е необходимо то да е записано във видео паметта, която заема сегменти А (0A000h) и В (0B000h). За текстовите режими видео паметта обикновено е в границите от абсолютния адрес 0B800h:0000h до 0B800h:7FFFh. Първият символ от екрана (в горния ляв ъгъл), е също първи във видео паметта, намиращ се на отместване 0000h. Следващия е на позиция 0002h. Понеже всеки от тях заема 2 байта, то целия екранен ред заема 160 байта. Тъй като адресното пространство е линейно, то първия символ от втория ред стои след последния символ от първи ред. За да се достигне до определен символ от даден ред, колона и страница при текстов режим 80x25 се използва следната формула:

$$\text{Позиция на отместването} = \text{ред} * 160 + \text{колона} * 2 + \text{страница} * 4096$$

Следващата програма първо измества курсора на позиция (1, 0) и след това показва на адрес 0B800h:0000h символ "А" със светло-червен цвят върху светло-зелен фон (позиция (0, 0)):

;prog65.asm

.model small

.code

start:

mov ax, 0B800h

mov es, ax

mov di, 0

xor bh, bh

mov dh, 0

mov dl, 1

mov ah, 02h

int 10h ; позиция на курсора (1, 0)

mov ah, 10101100b ; атрибути

mov al, "А" ; символ

mov es:[di], ax ; запис на символ и атрибути

mov ah, 4ch

int 21h

end start

Показването на низ на екрана с използване на пряк достъп до видео паметта е възможно да се извърши с използване на инструкциите за обработване на низове: LODSB и MOVSW. Използването на инструкция LOOP предполага зареждането на регистър CX с дължината на низа.

;prog66.asm

```

.model small

.data
message db 'Direct video memory access!'
message_len equ $-message

.code
start:
mov ax,@data
mov ds,ax
mov si,offset message

mov ax, 0b800h
mov es,ax
mov di,0h

xor bh, bh
mov dh, 0
mov dl, message_len
mov ah, 02h
int 10h          ; курсора след низа

mov cx, message_len ; брой символи за запис
et1:
    lodsb
    mov ah, 01001110b ; атрибути на низа
    stosw
loop et1

exit:
    mov ah, 4Ch
    int 21h
end start

```

Низът `message` от адрес `DS:SI` е байтов и извличането на елементите от него и записът им в регистър `AL` се извършва с `LODSB`. Атрибутите към символите се записват в регистър `AH`. По този начин информацията за всеки символ е двубайтова.

Необходимо е използването на инструкцията `STOSW`, която записва получената дума от `AX` в низ на адрес `0B800h:DI`.

6.2 Вход от клавиатурата

При натискане/отпускане на произволен клавиш от клавиатурата се генерира електрически импулс, съответстващ на този клавиш. Процесорът на клавиатурата, намиращ се в нея, преобразува електрическия импулс, съответстващ на позицията на клавиша, в число, наречено позиционен код. Този позиционен код може да се прочете от входно/изходния порт `60h`.

Позиционния код е уникално число, еднозначно определящо натиснатия/отпуснатия клавиш, но не е ASCII код. При изпращане от клавиатурата на позиционен код към процесора се предизвиква хардуерно прекъсване `IRQ1`, което предизвиква прекъсване `09h`. Подпрограмата за управление на клавиатурата приема тези кодове и ги преобразува в съответните им ASCII кодове. Получените ASCII кодове се поставят в буфер на клавиатурата.

6.2.1 Средства на DOS

Съществува набор от функции на DOS за четене на данни от клавиатурата, Табл.3.4. Използва се стандартното устройство за вход `stdin`. Това дава възможност за използване като източник за данни файл или друга програма. Следващият пример използва `08h` функция на `21h` за прочитане на символ от `stdin`. Тъй като кода на символа е получен в регистър `AL` е необходимо копирането на кода в регистър `DL`. Показването на символа се извършва с `02h` функция на `21h` прекъсване.

```
;prog67.asm
```

```
.model small
.code
```

```
start:
mov ah, 08h ; вход от stdin в AL
int 21h
```

```
mov dl, al ; запис в регистър DL на ASCII код
```

```
mov ah, 02h ; извеждане на символ с ASCII код в
DL
int 21h
```

```
mov ah, 4ch ; изход от програмата
int 21h
end start
```

Поставянето на въвеждането и извеждането от горната програма в цикъл може да се извърши с помощта на команда LOOP като предварително в CX се постави броя на итерациите на цикъла:

```
;prog68.asm
```

```
.model small
.code
```

```
start:
```

```
mov cx, 10d
```

```
et1:
    mov ah, 08h ; вход от stdin в AL
    int 21h
    mov dl, al ; запис в регистър DL на ASCII код
    mov ah, 02h ; символ на екран с ASCII код в DL
    int 21h
loop et1
```

```
mov ah, 4ch ; изход от програмата
int 21h
end start
```

Прекъсване на въвеждането, преди натискането на 10 клавиша, може да се извърши чрез натискане на клавишната комбинация Ctrl+C. Със същата клавишна комбинация се прекъсва и безкраен цикъл, организиран с инструкция JMP. Това е показано в следващия пример.

```
;prog69.asm
```

```
.model small
.code
```

```
start:
```

```
et1:
    mov ah, 08h ; вход от stdin в AL
    int 21h
    mov dl, al ; запис в регистър DL на ASCII код
    mov ah, 02h ; символ на екран с ASCII код в DL
    int 21h
jmp et1
```

```
mov ah, 4ch ; изход от програмата
int 21h
end start
```

За едностъпково прочитане на знак от клавиатурата и ехо на екран е възможна употребата на 01h функция на 21h прекъсване. Използването на двете функции - 08h и 02h на 21h прекъсване, улеснява обработката на прочетените от клавиатурата знаци преди показването им на екрана. По този начин е възможно да се извърши показване на екрана на знаци удовлетворяващи определено условие.

За избягване на прекъсването на програмата с Ctrl+C е възможно цикъла да се прекъсне при натискане на клавиш ESC (1Bh). Проверката на условие се извършва с инструкцията за сравнение CMP, която установява няколко флага. Инструкциите за условен преход JXX анализират тези флагове и в зависимост от тяхното състояние предават управлението на адреса свързан с етикет или към следващата инструкция. В следващата програма е използвана инструкция JE, която прекъсва цикъла на въвеждане и извеждане на знаци при натискане на ESC.

```
;prog610.asm
; Прочитане на клавиатурния вход и показване
; на въведените знаци. Прекъсване
; на програмата с клавиш Esc или Ctrl+C
```

```

.model small
.code

start:

et1:
    mov ah, 08h ; вход от stdin в AL
    int 21h

    cmp al, 1Bh ; проверка за натиснат Esc
    je exit

    mov dl, al ; запис в регистър DL на ASCII код
    mov ah, 02h ; символ на екран с ASCII код в DL
    int 21h

    jmp et1

exit:
    mov ah, 4ch ; изход от програмата
    int 21h
end start

```

При постъпково проследяване на работата на изпълнимата програма prog610.exe в TD при натискане на Esc при подканващото мигане на курсора се забелязва, че регистър AL се зарежда със стойност 1Bh. В резултат от изпълнението на последващата инструкция CMP регистър ZF се вдига (установява, установява в 1). По този начин се изпълнява условието, свързано с инструкция JE. Управлението се предава към етикет exit. Инструкция JE може да се замени с еквивалентната на нея инструкция JZ.

При работа с електронни документи често се случва да се попълват полета, в които се въвеждат само определен набор от знаци: само цифри (години, бройки и др), само букви (имена и др.) или и двете заедно. Следващата програма позволява въвеждането само на цифри. Тъй като ASCII кодовете на цифрите са в интервал от 30h до 39h с инструкция JB се отхвърлят всички натиснати

знаци с ASCII код по-малък от 30h, а с инструкция JA се отхвърлят всички натиснати знаци с ASCII код по-голям от 39h.

```

; prog611.asm
; Прочитане на клавиатурния вход и показване
; на екран само на въведените цифри. Прекъсване
; на програмата с клавиш Esc или Ctrl+C

```

```

.model small
.code
start:

```

```

et1:
    mov ah, 08h ; вход от stdin в AL
    int 21h

```

```

; проверка за натиснат Esc
    cmp al, 1Bh
    je exit

```

```

; проверка за натискане на цифра
    cmp al, 30h ; '0'
    jb et1
    cmp al, 39h ; '9'
    ja et1

```

```

    mov dl, al ; запис в регистър DL на ASCII код
    mov ah, 02h ; символ на екран с ASCII код в DL
    int 21h
    jmp et1

```

```

exit:
    mov ah, 4ch ; изход от програмата
    int 21h
end start

```

Горната програма ще работи коректно при замяна на инструкция JB с нейните еквиваленти JC или JNAE и на инструкция JA с JNBE.

```

; prog612.asm
; Прочитане на клавиатурния вход и показване
; на екран само на въведените цифри. Прекъсване
; на програмата с клавиш Esc или Ctrl+C

.model small
.code
start:
et1:
    mov ah, 08h ; вход от stdin в AL
    int 21h

; проверка за натиснат Esc
    cmp al, 1Bh
    je exit

; проверка за натискане на цифра
    cmp al, 30h ; '0'
    jnae et1
    cmp al, 39h ; '9'
    jnbe et1

    mov dl, al ; запис в регистър DL на ASCII код
    mov ah, 02h ; символ на екран с ASCII код в DL
    int 21h
    jmp et1

exit:
    mov ah, 4ch ; изход от програмата
    int 21h
end start

```

6.2.2 Средства на BIOS

Клавиатурата е основното входно устройство на компютъра. Използват се т.н. MF-II (*multy functional*) клавиатури, характеризиращи се с трите основни типа клавиши:

-- **символни (азбучно-цифрови)** – букви от а до z, цифрите от 0 до 9, символи @, #, \$, % и др.;

-- **функционални** – Insert, Delete, Home, End, Page Up, Page Down, BackSpace, Enter, F11 – F12;

-- управляващи – Alt, Ctrl и Shift.

Функционалните клавиши не са свързани със символи, а формират действие. Програмистът определя, че натискането на клавиш End премества курсора в края на текста на екрана. Функционалните клавиши са свързани с разширените ASCII кодове.

BIOS предоставя по-големи възможности за работа с клавиатурата в сравнение с DOS. Прекъсване 16h притежава няколко функции за четене на клавиатурата и нейното състояние.

Табл.6.3 Функции на 16h прекъсване на BIOS за работа с клавиатурата

Функция	Вход	Действие	Изход
00h	АН = 00h (83/84 кл.), 10h (101/102 кл.), 20h (122 кл.)	Чете символи от буфера на клавиатурата. Ако той не съдържа символ, функцията чака до въвеждането на такъв и го извлича от буфера.	Ако AL = 0, то АН = разширен код на клавиш. Ако AL > 0, то АН = ASCII код на клавиш. АН = позиционен код на клавиш.
01h	АН = 01h (83/84 кл.), 11h (101/102 кл.), 21h (122 кл.)	Чете символи от буфера на клавиатурата, като го проверява за символ, който е готов за въвеждане. Символът остава в буфера и може да се прочете отново.	ZF = 1: няма символ в буфера ZF = 0: има символ в буфера Ако AL = 0, то АН = разширен код на клавиш. Ако AL > 0, то АН = ASCII код на клавиш. АН = позиционен код на клавиш.

Функция	Вход	Действие	Изход
02h	AH = 02h (83/84 кл.), 12h (101/102 кл.), 22h (122 кл.)	Чете и връща статус на управляващи клавиши	AL = състояние на клавиатурата. При битове равни на единица от адрес 0040h:0017h: бит 0: Right Shift натиснат бит 1: Left Shift натиснат бит 2: Ctrl Key натиснат бит 3: Alt Key натиснат бит 4: Scroll Lock включен бит 5: Num Lock включен бит 6: Caps Lock включен бит 7: Insert включен, 0040h:0018h: бит 0: Left Ctrl натиснат бит 1: Left Alt натиснат бит 2: Right Ctrl натиснат бит 3: Right Alt натиснат бит 4: Scroll Lock включен бит 5: Num Lock включен бит 6: Caps Lock включен бит 7: Sys Rq включен,
05x	AH = 05h CH = Позиционен код на клавиш CL = ASCII код на клавиш	Симулира натискане на клавиш.	AL = 00h: Успешно завършване; AL = 01h: Грешка – пълен клавиатурен буфер.

Следващата програма симулира въвеждането от клавиатурата на команда `dir` с последващо натискане на клавиш `Enter`.

```
; prog613.asm
; Записва в буфера на клавиатурата команда
; dir, която ще се изпълни след завършване
; работата на програмата

.model small
.code
```

```
start:
mov cl, 'd' ; CL = ASCII-код на знак "d"
mov ah, 5
mov ch, 0
int 16h

mov cl, 'i' ; ASCII-код на знак "i"
mov ah, 5
mov ch, 0
int 16h

mov cl, 'r' ; ASCII код на знак "r"
mov ah, 5
mov ch, 0
int 16h

mov cl, 0Dh ; симулиране на Enter
mov ah, 5
mov ch, 0
int 16h

mov ah, 4ch ; изход от програмата
int 21h

end start
```

На четири места от `prog613.asm` се повтаря един и същи код, който е удобно да се оформи в процедура. Програмата добива следния компактен вид:

```
; prog614.asm
; Записва в буфера на клавиатурата команда
; dir, която ще се изпълни след завършване
; работата на програмата

.model small
.stack 100h
.code

start:
```

```

    mov  CX, 'D'
    call KBDWrite

    mov  CX, 'i'
    call KBDWrite

    mov  CX, 'R'
    call KBDWrite

    mov  CX, 0Dh
    call KBDWrite

    mov  ah, 4ch
    int  21h

KBDWrite PROC near
    mov  AH, 5
    int  16h
    RET
KBDWrite ENDP

ProgExit PROC near
    mov  AH, 4Ch
    int  21h
ProgExit ENDP

end start

```

Възможно е отделянето на готовата процедура **KBDWrite** от главната програма, която се модифицира по следния начин:

```

; prog615.asm
; Записва в буфера на клавиатурата команда
; dir, която ще се изпълни след завършване
; работата на програмата
; Използва се външен обектен модул KBDWrite.obj

```

```

.model small
.stack 100h
.code
EXTRN KBDWrite:FAR
start:

Main PROC near
    mov  CX, 'D'
    call KBDWrite

    mov  CX, 'I'
    call KBDWrite

    mov  CX, 'R'
    call KBDWrite

    mov  CX, 0Dh
    call KBDWrite

    mov  CX, 0Dh
    call KBDWrite

ProgExit PROC near
    mov  AH, 4Ch
    int  21h
ProgExit ENDP

Main ENDP

end start

```

Отделеният програмен модул има следното съдържание:

```

; prog616.asm
; Процедура за симулиране на натискане на
; клавиш,
; зареден в CX

.model small
.stack 100h
.code
start:
PUBLIC KBDWrite

```



```

KBDWrite PROC far
    mov AH, 5
    int 16h
    RET
KBDWrite ENDP
end start

```

Свързването на двата асемблирани файла prog615.obj и prog616.obj се извършва със следната команда:

```
tlink prog615.obj + prog616.obj
```

Получава се изпълнима програма prog615.exe.

Възможно е изнасянето на процедурата ProgExit в отделен обектен модул:

```

; prog617.asm
; Процедура за изход от изпълнима програма

```

```

.model small
.stack 100h
.code
start:
PUBLIC ProgExit
ProgExit PROC far
    mov AH, 4Ch
    int 21h
ProgExit ENDP
end start

```

Главната програма prog615.asm се променя по следния начин:

```

; prog618.asm
; Записва в буфера на клавиатурата команда
; dir, която ще се изпълни след завършване
; работата на програмата
; Използват се външните обектни модули
; KBDWrite.obj и ProgExit.obj

```

```

.model small
.stack 100h
.code
EXTRN KBDWrite:FAR
EXTRN ProgExit:FAR
start:

```

```

Main PROC near
    mov CX, 'D'
    call KBDWrite

    mov CX, 'I'
    call KBDWrite

    mov CX, 'R'
    call KBDWrite

    mov CX, 0Dh
    call KBDWrite

    call ProgExit

```

```
Main ENDP
```

```
end start
```

Свързването на трите обектни модула става по следния начин:

```
tlink prog618+prog617+prog616
```

Получава се изпълнима програма prog618.exe.

С използването на програма TLIB е възможно свързването на двата обектни модула prog616.obj и prog617.obj в една библиотека с име mylib.lib:

```
tlib mylib+prog616+prog617
```

Свързването на главната програма и библиотеката се извършва със следната команда:

```
tlink prog618+mylib
```

6.3 Извеждане на екрана на числа

6.3.1 Извеждане на числа в десетична бройна система

За да се изведе число в десетична бройна система е необходимо да се извърши деление на числото на 10 докато частното стане равно на нула. Получените остатъци (измежду цифрите от нула до девет) се извеждат в обратен ред на получаването им.

Делението на беззнакови числа се извършва с инструкция DIV. Остатъците е възможно да се запазят в програмния стек с инструкция PUSH. Извличането им от него се извършва с инструкция POP. Поставените цифри в стека са краен брой, затова е необходимо преди тях да се постави маркер за край, например това може да е числото 77h. При всяко извеждане на число от стека се проверява дали това не е 77h. Ако това е така, то извеждането приключва.

Показването на екран на изведените от стека цифри се извършва като преди това към тях се прибавя 30h. По този начин се получава номера на техния ASCII знак. Това е възможно, защото в ASCII таблицата цифрите от 0 до 9 са последователно наредени, заемайки ASCII номера от 30h до 39h.

```
; prog619.asm
; Показва на екран число в десетична
; бройна система, записано в регистър AX
; Използва делител байт
```

```
.MODEL small
.STACK 100h
.CODE
main:
```

```
    mov ax,77h    ; маркер за край
```

```
    push ax        ; поставяне маркера в стека
    mov ax,234d    ; число за извеждане
    mov CL,10d     ; делител на числото
```

```
labelDIV:
```

```
    mov ah, 00h
    div CL        ; AX/CL -> частно AL, остатък AH
    push AX       ; запазване на остатъка в стека
    cmp AL,0      ; проверка за нулево частно
    jne labelDIV
```

```
    mov ah, 02h
```

```
labelPRINT:
```

```
    pop DX
    cmp DX, 77h   ; проверка за край
    je  ProgExit
    mov dl, dh
    add dl,30h
    int 21h
    jmp labelPRINT
```

```
ProgExit:
```

```
    mov ah,4ch
    int 21h
```

```
END main
```

Друг начин за извършване на този краен брой извеждания от стека е преброяването на извършените деления и изпълнение на същия брой извеждания от стека.

6.3.2 Дъмп

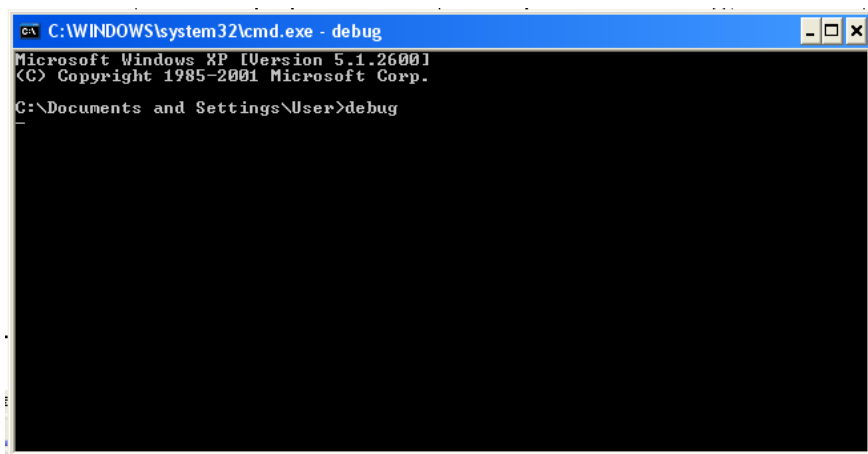
Дъмп (*dump*) е компютърен термин, който се отнася за начина, по който съдържанието на текстов файл се отпечатва на екрана – само текстовите символи или освен тях и шестнадесетични числа. Програмите дъмпери показват съдържанието във формат ASCII. Прави се разлика между нормалните ASCII символи и управляващите символи, като LF или CR.

Определянето на BIOS версията става по следния начин:

- От Start бутона изберете менюто Run;
- Стартирайте командния процесор на MS-DOS - `cmd.exe` или `command.com`, чрез написване на името му в полето. Натиснете OK. Стартира се командният процесор.
- Извикате програмата `debug.exe` чрез изписване на думата `debug` и натискане на Enter.

debug

Стартира се програмата debug:



Фиг.6.1 Работа на програмата debug

- Въведете следния ред за да се визуализира съдържанието на байтовете в края на ROM-BIOS:

```
d ffff:0005 l 8
```

Следващият ред показва съдържанието на 08h байта от паметта започващи от шестнадесетичен адрес `ffff:0005`:

```
FFFF:0000 30 31 2F-30 38 2F 30 34 01/08/04
```

Версията на BIOS е от 01.08.2004 г.

- За изход от debug въведете 'q'.
- За затваряне на командния процесор въведете `exit` и натиснете Enter.

6.4 Задачи

Задача 6.1 Заредете `prog63.exe` в TD. Поставете точка на прекъсване на ред `mov ah, 4ch`. Стартирайте програмата от менюто Run/Run. Коментирайте съдържанието на използваните регистри.

Задача 6.2 Променете ред `inc dx` на `inc dl`. Получете изпълнима програма. Изпълнете заданието от горната задача.

Задача 6.3 Модифицирайте програма `prog63.asm` до програма, която да показва а) всички символи от ASCII/IBM системата от знаци в низходящ ред; б) символи с номера от 55h до 66h включително.

Задача 6.4 Запишете низ "Hello World" с черни символи на жълт фон на втора видеостраница с атрибути вътре в низа. Използвайте функция 13h на 10h прекъсване. Покажете видеостраницата на екрана.

Задача 6.5 Модифицирайте `prog66.asm` по следния начин: към всеки символ от променливата `message` асоциирайте атрибут. Покажете съобщението на екрана като използвате `LODSW` и `STOSW`.

Задача 6.6 Модифицирайте `prog611.asm` така, че да извежда само цифри до натискане на Esc като проверява едновременно изпълнение на условията натиснатите знаци да имат ASCII кодове по-големи от 30h и по-малки от 39h.

Задача 6.7 Модифицирайте `prog611.asm` така, че да извежда:

- а) само малки букви от английската азбука;
- б) само големи букви от английската азбука;
- в) само малки и големи букви от английската азбука.

7 Задачи

7.1 Решени примерни задачи

Задача 7.1 Запознайте се с работата на инструкцията XOR. Съставете програма на TASM, която да използва тази инструкция. Тествайте изпълнимата програма с TD.

```
.model small
.code
xor ax, ax
int 16h
mov ah, 4ch
int 21h
end
```

Или

```
code_segment segment
    assume cs:code_segment
xor    ax, ax    ;ax=0
int    16h      ;wait for a key
mov    ah, 4ch
int    21h      ;terminate program
code_segment ends
end
```

Задача 7.2 Използвайте функцията 01h на 21h прекъсване за показване на натиснатия клавиш от клавиатурата на екрана.

```
code_segment segment
    assume cs:code_segment
mov    ah, 01h
int    21h      ;stdin -> stdout
mov    ah, 4ch
int    21h      ;terminate program
code_segment ends
end
```

```
code_segment segment
```

```
    assume cs:code_segment

repeat: mov    ah, 01h
        int    21h      ;stdin -> stdout
        jmp    repeat

        mov    ah, 4ch
        int    21h      ;terminate program

code_segment ends
end
```

Задача 7.3 Използвайте 08h функцията на 21h прекъсване за изход на екрана на натиснатите клавиатурни символи.

```
code_segment segment
    assume cs:code_segment

        mov    ah, 08h    ; console input without echo
        int    21h        ; al = input sign
    mov    dl, al    ; dl = output sign
        mov    ah, 02h    ; output sign from dl
        int    21h

        mov    ah, 4ch    ; exit in DOS
        int    21h

code_segment ends
end
```

Задача 7.4 Използвайки две различни променливи последователно покажете съобщенията “hello” и “exit” на екрана.

```
.model small

.stack 100h

.data
var1 dB 'hello',13,10,'$'    ; 10d = new line
                                ; 13d = carriage return
var2 dB 'exit',13,10,'$'

.code
```

```

mov ax, @data      ; ax -> data
mov ds, ax         ; ds -> data
mov dx, offset var2 ; dx -> var2
mov ah, 09
int 21h

mov ah, 4ch        ; terminate program
int 21h

```

end

Задача 7.5 Запознайте се с работата на команда **LOOP**. Изпишете на екрана 10 пъти съобщението “hello” като използвате регистър **CX** като брояч.

```

.model small

.stack 100h

.data
var1 db 'hello',13,10,'$' ; 10d = linefeed
                                ; 13d = carriage return
.code
    mov cx, 10      ; counter
repeat:
    mov ax, @data   ; ax -> data
    mov ds, ax      ; ds -> data
    mov dx, offset var1 ; dx -> var1 ; ds:dx
    mov ah, 09
    int 21h
    loop repeat

    mov ah, 4ch      ; terminate program
    int 21h

```

end

Задача 7.6 Покажете символите от ASCII таблицата на екрана.

```

code_segment segment
    assume cs:code_segment

```

```

start:    mov cx, 256d    ; character to output
          mov dl, 00h
          repeat:

```

```

          mov ah, 02h    ; 21,02h display output
          int 21h        ; character in DL
          inc dl
          loop repeat

```

```

end_program:
    mov ah, 4ch          ; terminate in DOS
    int 21h

```

```

code_segment ends
end start

```

Задача 7.7 Сумирайте масива от цифри 1, 3, 5, 7, 9 в регистър **AX** и проверете правилността на изпълнение на получената програма с TD.

```

; suma of 4isla v masiv
.model small
.stack 50h
.data
masiv dw 1,3,5,7,9,'$'
.code
    mov ax,@data
    mov ds,ax ;
    mov si,0 ; ds:si->data
    mov ax,0 ;
    mov cx,5 ;

```

```

    et1:
    add ax,[si+masiv]
    add si,2
    loop et1

```

```

mov ah, 4ch
int 21h
end

```

Или

```

.model small
.stack 50h
.data
masiv dw 1,3,5,7,9
.code
    mov ax,@data
    mov ds,ax
    mov ax,0
    mov si,0
    add ax,[si+masiv]
    add si,2
    add ax,[si+masiv]
    add si,2
    add ax,[si+masiv]
    add si,4
    add ax,[si+masiv]
    mov ah,4ch
    int 21h
end

```

Задача 7.8 Напишете кратка програма за използване на процедури.

```

code_segment SEGMENT
    assume cs:code_segment
a1:  call subroutine
a2:  inc ax
a3:  call subroutine
a4:  inc bx
     mov ah,4ch
     int 21h

subroutine proc near
     mov ax,0
     mov bx,0
     ret
subroutine endp

code_segment ends
end

```

Задача 7.9 Напишете програма, която изписва ASCII знаците на екрана. Използвайте процедури.

```

.model small
.stack
.code

start:
print_ascii proc
     mov dl,00h
     mov cx,256
     print_loop:
         call write_char
         inc dl
         loop print_loop

     mov ah,4Ch
     int 21h
print_ascii endp

write_char proc
     mov ah,02h
     int 21h
     ret
write_char endp

end start

```

Задача 7.10 Покажете на екрана малките букви от латинската азбука (от **a** до **z**). Използвайте процедури.

```

.model small
.stack
.code

print_a_z proc
     mov dl,'a'
     mov cx,26
     print_loop:
         call write_char
         inc dl
         loop print_loop

```

```

        mov  ah,4Ch
        int  21h
print_a_z  endp

write_char  proc
        mov  ah,02h
        int  21h
        ret
write_char  endp

end

```

Задача 7.11 Съставете програма на TASM, с използване на макроси, която да изчаква натискането на пет клавиша, показва ги на екрана и след това ги показва в обратен ред. Например:

```

abcdf
fdcba

```

macro.inc

pokazvane_cifra_dl macro

```

        push  ax
        push  dx

        add  dl, 30h ;ili 48d
        mov  ah, 02h
        int  21h

        pop  dx
        pop  ax
endm

```

exit macro

```

        mov  ah, 4ch
        int  21h
endm

```

pokazvane_znak_dl macro nomer_znak

```

        push  ax
        push  dx

```

```

        mov  dl, nomer_znak
        mov  ah, 02h
        int  21h
        pop  dx
        pop  ax

```

endm

eho_al macro

```

        push  bx ;za da se zapazi ah
        mov  bh, ah ;
        mov  ah, 01h
        int  21h
        mov  ah, bh
        pop  bx

```

endm

pokazvane_dl macro

```

        push  ax
        mov  ah, 02h
        int  21h
        pop  ax

```

endm

pet_but.asm

```

include macro.inc
.model small
.stack 100h
.data
.code
start:
        mov  cx, 5
povtori1:
        mov  ah, 01h
        int  21h
        mov  ah, 0h
        push  ax
        loop povtori1

```

```

pokazvane_znak_dl 10d
pokazvane_znak_dl 13d

```

```

    mov cx, 5
povtori2:
    pop dx
    mov ah, 02h
    int 21h
    loop povtori2
exit
end start

```

Задача 7.12 Съставете програма на TASM, която да изчаква натискането на клавиши от клавиатурата и да изписва веднага ASCII кода им в десетичната бройна система. Въвеждането да продължава до натискане на ESC.

```

.model small
.stack 100h
.data
    scan_table dB 10 dup (0,1,2,3,4,5,6,7,8,9)
    scan_table_lenght equ 10
    interval dB ' ', '$'
    new_line dB 13,10,'$'
.code
cool:
    mov ax, @data
    mov es, ax
    mov di, offset scan_table
    mov cx, scan_table_lenght
    mov ah, 01h
    int 21h
    call inter
    push ax
    xor ah, ah
    mov bx, 000Bh
    push bx ; border
    mov bx, 000Ah ;
et:
cwd
    div bx ; ax=quotient dx=remainder
    push dx ; push remainder to the stack

```

```

    repne scasb ; searching 0..9 ?
    je stop_div ; equal ax=scan_table (0..9)
    mov cx, 10d
    jmp et
stop_div:
    push ax
start_output:
    pop dx
    cmp dx, 000bh
    je go_loop
    add dl, 30h
    mov ah, 02h
    int 21h
    jmp start_output
go_loop:
    pop dx
    cmp dl, 1bh
    je end_prog
    call nl
    jmp cool
end_prog:
    mov ah, 4ch
    int 21h
inter proc near
    push ax
    push ds
    push dx
    mov ax, @data
    mov ds, ax
    mov dx, offset interval
    mov ah, 09h
    int 21h
    pop dx
    pop ds
    pop ax
    ret
inter endp
nl proc near
    push ax
    push ds
    push dx
    mov ax, @data
    mov ds, ax
    mov dx, offset new_line

```



```

        mov  ah, 09h
        int  21h
        pop  dx
        pop  ds
        pop  ax
        ret
nl endp
end cool

```

Задача 7.13 Съставете програма на TASM, която при въведена цифра изписва цифрата повдигната на втора степен.

```

include macro.inc
.model small
.stack 300h
.data
kvadrati db 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
.code
mov ax, @data
mov ds, ax
mov bx, offset kvadrati
povtori:
    mov cl, 10
    eho_al
    and al, 0fh
    pokazvane_znak_dl 32d
    xlat kvadrati
    xor ah, ah
    div cl
    mov dl, al
    pokazvane_cifra_dl
    mov dl, ah
    pokazvane_cifra_dl
    pokazvane_znak_dl 10d
    pokazvane_znak_dl 13d
    jmp povtori
exit
end

```

7.2 Задачи за самостоятелна работа

Задача 7.14 Съставете програма на TASM, която приема от клавиатурата двуцифрено число в интервала от 57 до 89 и показва

на екрана съответното му двоично число. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.15 Съставете програма на TASM, която показва на екран всички цели положителни десетични числа в интервала от 75 до 863, които се делят на 2 без остатък. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.16 Съставете програма на TASM, която приема от клавиатурата 3 числа в интервала от 15 до 75 и извежда на екрана сумата от цифрите на тези числа. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.17 Съставете програма на TASM, която приема от клавиатурата цифри в интервала от 0 до 9 и извежда на екрана текстов низ с името на всяка цифра. При натискане на 0 се извежда: nula, 1 -> edno, 2 -> dve, 3 -> tri, 4 -> chetiri, 5 -> pet, 6 -> shest, 7 -> sedem, 8 -> osem, 9 -> devet. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.18 Съставете програма на TASM, която приема като вход низ от 7 символа и показва на следващия ред низ образуван от първоначалния, на който всеки символ е отпечатан още по веднъж. Програмата работи до натискане на клавиш ESC. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.19 Съставете програма на TASM, която обхожда предварително дефиниран масив от 6 числа в интервала от 10 до 25 и показва на екран само тези, които са по-големи от 17. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.20 Съставете програма на TASM, която обхожда предварително дефиниран масив от 6 числа в интервала от 23 до 77 и показва на екран само тези, на които сбора от двете им цифри е по-голям от 8. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.21 Съставете програма на TASM, която обхожда предварително дефиниран масив от 7 числа в интервала от 11 до 76

и показва на екран само тези, на които сбора от двете им цифри е равен на 9. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.22 Съставете програма на TASM, която обхожда предварително дефиниран масив от 5 числа в интервала от 10 до 98 и показва на екран само тези, които са по-големи от 60 и общия брой на тези числа. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.23 Съставете програма на TASM, която приема като вход четирицифрени двоични числа в интервала от 0B до 1111B и показва на екран броя на единиците и броя на нулите. Програмата работи до натискане на клавиш ESC и приема като входни данни само символите 0,1,ESC. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.24 Съставете програма на TASM, която изчаква въвеждането на низ от 15 символа 'a', 'c', 'd' от клавиатурата и установява броя на срещанията на всеки от символите. Програмата работи до натискане на клавиш ESC и приема от клавиатурата само символи 'a', 'c', 'd'. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.25 Съставете програма на TASM, която изчаква въвеждания от клавиатурата на числа в интервала от 10 до 95 и показва на екран 'da' или 'ne' в зависимост от това дали остатъка от делението на въведеното число и цифрата 5 е 3 или не е 3. Добавете към решената задача използвани файлове с процедури или макроси.

Задача 7.26 Съставете програма на TASM, която изисква въвеждането на цифри от клавиатурата и показва на следващия ред квадрата на въведената цифра. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC.

Задача 7.27 Съставете програма на TASM, която приема от клавиатурата двуцифрено число в интервала от 57 до 89 и показва на екрана съответното им двоично число. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC.

Задача 7.28 Съставете програма на TASM, която обхожда предварително определен списък от 7 цели положителни десетични

числа в интервала от 130 до 850, намира и извежда на екран минималното число.

Задача 7.29 Съставете програма на TASM, която обхожда предварително определен списък от 7 цели положителни десетични числа в интервала от 130 до 850, намира и извежда на екран максималното число.

Задача 7.30 Съставете програма на TASM, която показва на екран всички цели положителни десетични числа в интервала от 75 до 863, които се делят на 2 без остатък.

Задача 6. Съставете програма на TASM, която показва на екран всички цели положителни десетични числа в интервала от 25 до 740, които съдържат в запис си цифрата 5.

Задача 7.31 Съставете програма на TASM, която приема от клавиатурата две положителни десетични числа в интервала от 230 до 750, намира и извежда на екрана най-големия им общ делител. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC.

Задача 7.32 Съставете програма на TASM, която приема от клавиатурата числа в интервала от 175 до 750 и извежда на екрана сумата от цифрите на тези, които четени отляво надясно и отдясно наляво са еднакви. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC.

Задача 7.33 Съставете програма на TASM, която показва на екран всички цели положителни десетични числа в интервала от 10 до 99, на които сумата от цифрите им се дели на 7 без остатък.

Задача 7.34 Даден е списък от 25 числа в интервала от 10 до 99. Съставете програма на TASM, която приема от клавиатурата число от същия интервал и извежда всички индекси на числата от списъка, които съвпадат с въведеното число. Програмата да работи до натискане на клавиш ESC и приема като вход само цифрите от 0 до 9 и клавиш ESC.

Задача 7.35 Съставете програма на TASM, която да изписва на екрана ASCII таблицата в низходящ ред (от 255 до 0).

Задача 7.36 Съставете програма на TASM, която Покажете символ „H” на екрана с пряк достъп до видеопаметта в сегмент 0B800 h, на отместване изчислено по следните критерии:

страница 0, ред 12, колона 73 + последната цифра от Вашия Ф.Номер, т.е. Формулата + последна цифра от Ф.Н.

Задача 7.37 Съставете програма на TASM, която симулира въвеждането от клавиатурата на MS DOS командите **path** и **tree**

8 Основни инструкции на Асемблер

Означения:

regB, regW, regDW – операнди в регистър с големина байт, дума, двойна дума

varB, varW, varDW, varQW – операнди (променливи) в паметта с големина байт, дума, двойна дума, четворна дума, 48-битова променлива

strB, strW – низ от байтове, думи

reg – регистър

var – променлива в паметта

segreg – сегментен регистър

const - константа

Флагове:

U – неопределена стойност

r – стойността зависи от резултата след изпълнението на инструкцията

ADD

ADD приемник, източник

```
04 ib ADD AL, imm8
05 iw ADD AX, imm16
05 id ADD EAX, imm32
80/0 ib ADD r/m8, imm8
81/0 iw ADD r/m16, imm16
81/0 id ADD r/m32, imm32
83/0 ib ADD r/m16, imm8
83 /0 ib ADD r/m32, imm8
```

```

00/r ADD r/m8, r8
01/r ADD r/m16, r16
01/r ADD r/m32, r32
02/r ADD r8, r/m8
03/r ADD r16, r/m16
03/r ADD r32, r/m32

```

Събиране на два целочислени операнда.

Действие: приемник = приемник + източник. При препълване на приемника флаг CF се установява на 1.

Флагове: OF=r, SF=r, ZF=r, AF=r, PF=r, CF=r

CMP

CMP оп1, оп2

```

3C ib    CMP AL, imm8
3D iw    CMP AX, imm16
3D id    CMP EAX, imm32
80/7 ib  CMP r/m8, imm8
81/7 iw  CMP r/m16, imm16
81/7 id  CMP r/m32, imm32
83/7 ib  CMP r/m16, imm8
83/7 ib  CMP r/m32, imm8
38/r    CMP r/m8, r8
39/r    CMP r/m16, r16
39/r    CMP r/m32, r32
3A/r    CMP r8, r/m8
3B/r    CMP r16, r/m16
3B/r    CMP r32, r/m32

```

Сравнение на два операнда

Действие: оп1 и оп2 се изваждат, при което операндите не се променят. Според резултата от сравнението се установяват и флаговете. Действието на инструкция CMP е еднакво с това на

инструкция SUB, само че CMP не записва резултата от изваждането на мястото на оп1.

Флагове: OF = r, SF = r, ZF = r, AF = r, PF = r, CF = r

DEC

Намаляване (**decrement**) с единица на операнд.

Мнемоничен код: dec операнд

Операнди: var/reg

Флагове: OF SF ZF AF PF

DIV

DIV делител

```

F6/6 DIV r/m8
F7/6 DIV r/m16
F7/6 DIV r/m32

```

Беззнаково деление

Действие: делимото се задава неявно и неговия размер зависи от размера на делителя, който се указва явно в инструкцията.

Местоположението на делимото, делителя, частното и остатъка в зависимост от техните размерности при изпълнение на инструкция DIV са показани в следващата таблица.

Табл.8.1 Изпълнение на инструкция DIV

Размер на операнда	Делимо	Делител	Частно	Остатък	Максимално частно
дума/байт	AX	r/m8	AL	AH	255
двойна дума/дума	DX:AX	r/m16	AX	DX	65 535

Размер на операнда	Делимо	Делител	Частно	Остатък	Максимално частно
четворна дума/ двойна дума	EDX:EAX	r/m32	EAX	EDX	$2^{32}-1$

При деление не се сигнализира за грешка при надвишаване на стойността за максималното частно. Изпълнява се програмно прекъсване при деление на нула (т.н. препълване при деление).

Флагове: OF=U SF=U ZF=U AF=U PF=U CF=U

INC

Увеличаване (*increment*) с единица на операнд.

Мнемоничен код: `inc` операнд

Операнди: var/reg

Флагове: OF SF ZF AF PF

INT

Извикване на подпрограма за обслужване на прекъсване (*interrupt*).

Мнемоничен код: `int` номер_прекъсване

Флагове: IF, TF

INTO

При установяване на флаг OF на 1 се извършва прекъсване при препълване (*interrupt of overflow*). Тогава се изпълнява прекъсване `int 4`.

Флагове: IF, TF

IRET

IRET/IRETD

IRET/IRETD

CF

Връщане от прекъсване (*interrupt return*). Използва се връщане на управлението на прекъснатата програма. В реален режим от стека се прочитат IP, CS и Flags.

Флагове: всички

Действия: зависи от режима на работа на процесора.

В реален режим се изпълнява последователно извличане от стека на съдържимото на регистри EIP/IP, CS и EFLAGS/FLAGS, и работата на прекъснатата програма се възстановява.

В защитен режим действията на команда IRET определят флаговете NT и VM в регистър EFLAGS, а също и значението на VM в образа на EFLAGS, съхранен в стека. В зависимост от тях се изпълнява връщане от режим V86; връщане в режим V86; връщане към кода на друго ниво; връщане от вложена задача.

Флагове: всички EFLAGS

JMP

`jmp [мод] цел`

EB cb JMP rel8

E9 cw JMP rel16

E9 cd JMP rel32

FF/4 JMP r/m16

FF/4 JMP r/32

EA cd JMP ptr16:16

EA cp JMP ptr16:32

FF/5 JMP m16:16

FF/5 JMP m16:32

Безусловен преход към цел.

Действие: в зависимост от операнда цел се извършва различни преходи:

rel8/16/32 — близък относителен преход. Стойностите на rel8/16/32 се третира знаково и са относителни спрямо следващата след JMP инструкция в кодовия сегмент. Адресът е равен на (EIP/IP) + (rel8/16/32);

r16(32)/m16(32) — близък абсолютен косвен преход. Цел — регистър r16(32) или клетка памет m16(32), съдържащи адрес на преход в текущия кодов сегмент;

ptr16:16(32) — далечен абсолютен преход;

m16:16(32) — далечен абсолютен косвен преход. Цел — адрес на клетка от паметта с размер 32(48) бита.

Примери:

```
jmp short m1
; не повече от 35-40 инструкции (+127 байта)
```

```
m1:
-----
```

```
m1:
; не повече от 35-40 инструкции (-126 байта)
jmp short m1
```

Jxx

Jxx етикет

Преход, ако е изпълнено условието xx.

Действие: Инструкциите за условен преход, в зависимост от своята мнемоника, анализират флагове, и ако проверяемото условие е истина, то се извършва преход към етикет. Ако проверяваното условие не е истина, то се извършва преход към следващата инструкция. Идентификатор етикет се преобразува от асемблер в

непосредствена стойност rel8 (use16) или rel16/rel32 (use32), които се прибавят към текущото значение на IP/EIP.

Табл.8.2 Видове инструкции за условен преход Jxx

Маши- нен код	Мнемоничен код	Преход	Тип операнди	Флагове
70 cb	JO rel8	Къс, ако има препълване		OF = 1
71 cb	JNO rel8	Къс, ако няма препълване		OF = 0
72 cb	JB rel8	Къс, ако е под (по- малко)	БЗн	CF = 1
	JC rel8	Къс, ако CF е установен		
	JNAE rel8	Къс, ако не е над или равно		
73 cb	JAЕ rel8	Къс, ако е над (по- голямо) или равно	БЗн	CF = 0
	JNB rel8	Къс, ако не е под (по-малко)		
	JNC rel8	Къс, ако няма пренос		
74 cb	JE rel8	Къс, ако е равно		ZF = 1
	JZ rel8	Къс, ако е нула		
75 cb	JNZ rel8	Къс, ако не е нула		ZF = 0
	JNE rel8	Къс, ако не равно		
76 cb	JBE rel8	Къс, ако е под (по- малко) или равно	БЗн	CF = 1 или ZF = 1
	JNA rel8	Къс, ако не е над (по-голямо)		
77 cb	JA rel 8	Къс, ако е над (по- голямо)	БЗн	CF = 0 и ZF = 0
	JNBE rel8	Къс, ако не е под (по-малко) или равно		
78 cb	JS rel8	Къс, ако SF = 1		SF = 1
79 cb	JNS rel8	Къс, ако SF = 0		SF = 0
7A cb	JP rel8	Къс, ако PF = 1		PF = 1
	JPE rel8			

Маши- нен код	Мнемоничен код	Преход	Тип операнди	Флагове
7B cb	JPO rel8	Къс, ако четността е нечетна (няма четност)		PF = 0
	JNP rel8	Къс, ако няма четност		
7C cb	JL rel8	Къс, ако е по-малко	3н	SF <> OF
	JNGE rel8	Къс, ако не по-голямо или равно		
7D cb	JGE rel8	Къс, ако е по-голямо или равно	3н	SF = OF
	JNL rel8	Къс, ако не е по-малко		
7E cb	JLE rel8	Къс, ако е по-малко или равно	3н	SF <> OF или ZF = 1
	JNG rel8	Къс, ако не е по-голямо		
7F cb	JG rel8	Къс, ако е по-голямо	3н	SF = OF и ZF = 0
	JNLE rel8	Къс, ако не е по-малко или равно		
E3 cb	JCXZ rel8	Къс, ако CX = 0		
	JECXZ rel8	Къс, ако ECX = 0		
OF 80 cw/cd	JO rel16/32	Близък, ако има препълване		OF = 1
OF 81 cw/cd	JNO rel16/32	Близък, ако няма препълване		OF = 0
OF 82 cw/cd	JB rel16/32	Близък, ако е под (по-малко)	БЗн	CF = 1
	JC rel16/32	Близък, ако флагът за пренос е установен		
	JNAE rel16/32	Близък, ако не е над (по-голямо) или равно		
OF 83 cw/cd	JAЕ rel16/32	Близък, ако е над (по-голямо) или равно	БЗн	CF = 0

Маши- нен код	Мнемоничен код	Преход	Тип операнди	Флагове
	JNB rel16/32	Близък, ако не е под (по-малко)		
	JNC rel16/32	Близък, ако няма пренос		
OF 84 cw/cd	JE rel16/32	Близък, ако е равно		ZF = 1
	JZ rel16/32	Близък, ако е нула		
OF 85 cw/cd	JNZ rel16/32	Близък, ако не е нула		ZF = 0
	JNE rel16/32	Близък, ако не е равно		
OF 86 cw/cd	JBE rel16/32	Близък, ако е под (по-малко) или равно	БЗн	CF = 1 или ZF = 1
	JNA rel16/32	Близък, ако не е над		
OF 87 cw/cd	JA rel16/32	Близък, ако е над (по-голямо)	БЗн	CF = 0 и ZF = 0
	JNBE rel16/32	Близък, ако не е под (по-малко) или равно		
OF 88 cw/cd	JS rel16/32	Близък, ако флагът за знак е установен		SF = 1
OF 89 cw/cd	JNS rel16/32	Близък, ако флагът за знак не е установен		SF = 0
OF 8A cw/cd	JP rel16/32	Близък, ако флагът за четност е установен		PF = 1
	JPE rel16/32	Близък, ако четността е четна		
OF 8B cw/cd	JPO rel16/32	Близък, ако четността е нечетна		PF = 0
	JNP rel16/32	Близък, ако няма четност		
OF 8C cw/cd	JL rel16/32	Близък, ако е по-малко	3н	SF <> OF
	JNGE rel16/32	Близък, ако не е по-голямо или равно		

Маши- нен код	Мнемоничен код	Преход	Тип операнди	Флагове
OF 8D cw/cd	JGE rel16/32	Близък, ако е по- голямо или равно	3н	SF = OF
	JNL rel16/32	Близък, ако не е по- малко		
OF 8E cw/cd	JLE rel16/32	Близък, ако е по- малко или равно	3н	SF <> OF или ZF = 1
	JNG rel16/32	Близък, ако не е по- голямо		
OF 8F cw/cd	JG rel16/32	Близък, ако е по- голямо	3н	SF = OF и ZF = 0
	JNLE rel16/32	Близък, ако не е по- малко или равно		

LODS/LODSB/LODSW/LODSD

LODS *низ_източник*
LODSB/LODSW/LODSD

AC LODS DS:(E)SI
AD LODS DS:SI
AD LODS DS:ESI
AC LODSB
AD LODSW
AD LODSD

Извличане на байт (дума, двойна дума) от низ.

Действие: команда извлича елемент (байт, дума, двойна дума) от низ, адресиран чрез DS:SI/ESI, и го записва в AL/AX/EAX. След записа на елемента регистър SI/ESI изменя стойността си в зависимост от флага DF. Ако DF = 0, SI/ESI се увеличава с 1/2 при байтове/думи или ESI с 4 при двойни думи. Ако DF = 1, SI/ESI се намалява с 1/2 при байтове/думи или ESI с 4 при двойни думи. При използване на LODS инструкцията изисква *низ_източник*, за да установи типа на низа. Името *низ_източник* може да отпадне, ако се използват инструкции LODSB, LODSW или LODSD.

LOOP/LOOPxx

LOOP/LOOPxx *етикет*

E2 cb LOOP rel8
E1 cb LOOPE\LOOPZ rel8
E0 cb LOOPNE\LOOPNZ rel8

Изпълняване на цикъл с брояч в регистър CX/ECX.

Действие LOOP: Намалява се с единица стойността в регистър CX/ECX. Проверява се регистър CX/ECX: ако CX/ECX = 0, то управлението се предава на инструкцията непосредствено след LOOP, ако CX/ECX > 0, управлението се предава на инструкцията указана с *етикет*.

Относителното отместването на *етикет* спрямо текущата стойност на регистър IP/EIP трябва да бъде в границите от -126 до +127 байта. При по-големи отмествания се използва инструкция JMP.

Действие LOOPE/LOOPZ/LOOPNE/LOOPNZ: Намалява се с единица стойността в регистър CX/ECX. Проверяват се регистър CX/ECX и флаг ZF. Ако CX/ECX = 0, то управлението се предава на инструкцията непосредствено след LOOPxx. Ако ZF = 0, то за инструкции LOOPE/LOOPZ това означава изход от цикъла, а за инструкции LOOPNE/LOOPNZ – преход към началото на цикъла. Ако ZF = 1, то за инструкции LOOPE/LOOPZ това означава преход към началото на цикъла, а за инструкции LOOPNE/LOOPNZ – изход от цикъла.

Ако след изчисляване на броя повторения преди началото на цикъла се окаже, че стойността в CX е нула, то след първата итерация инструкция LOOP ще намали CX от 0 на 0ffffh. Тъй като CX не е 0, то цикълът ще се повтори 65 536 пъти. За избягване на такива грешки е възможно използването на инструкции JECXZ и JCXZ за „прескачане” на цикъла:


```

mov cx, 15
jecxz m1

repeat:
    ; тяло на цикъла
loop repeat

m1:

```

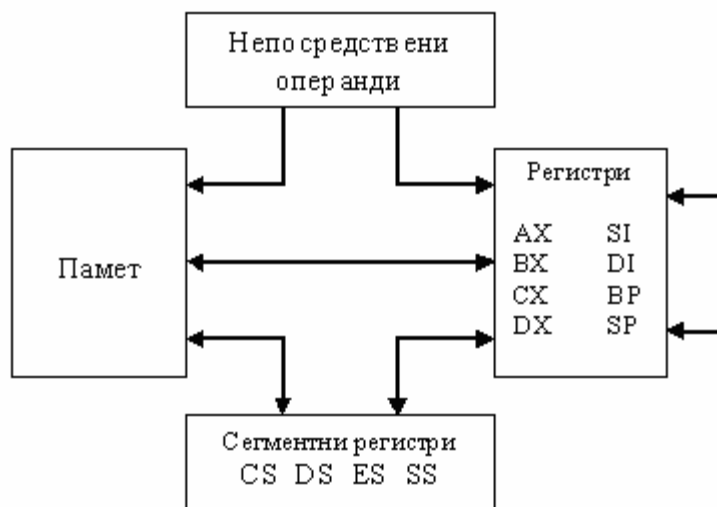
MOV

Инструкция MOV (*move*) е основната инструкция за прехвърляне на данни. Тя копира съдържанието на байт, дума или двойна дума от операнд_източник в операнд_цел:

```
mov операнд_цел, операнд_източник
```

Стойността на операнд_източник не се променя.

Позволените начини за прехвърляне на данни са показани на следващата схема. Стрелките показват посоките на прехвърляне.



Фиг.8.1 Посока на прехвърляне на данни

Не е позволено пряко прехвърляне на данни от една клетка от паметта в друга, от един сегментен регистър в друг и от непосредствен операнд в сегментен регистър. Това е обусловено от факта, че отсъстват такива машинни команди. Ако в даден алгоритъм това е необходимо да се извърши, то се използват регистрите с общо предназначение. Например, за зареждане на сегментен регистър DS с адреса на сегмента за данни (намиращ се в непосредствен операнд) е необходимо това да се направи двустъпково:

```

mov ax, @data; или mov ax, seg msg
mov ds, ax

```

Прехвърляне на стойността на клетка от паметта в друга клетка от паметта:

```

mov ax, [mem1]
mov [mem2], ax

```

Прехвърляне на стойност от сегментен регистър в друг сегментен регистър:

```

mov ax, ds
mov es, ax

```

POP

POP приемник

```

8F/0 POP m16
8F/0 POP m32
58 + rw POP r16
58+ rd POP r32
1F POP DS
07 POP ES
17 POP SS
0F A1 POP FS
0F A9 POP GS

```

Извеждане на стойност от стека

Действие: инструкцията прехвърля стойност от върха на стека в регистър, клетка от паметта или сегментен регистър, след това увеличава указателя на стека ESP/SP с 4 байта при use32 и с два байта при use16. Недопустимо е извеждане на стойност в сегментния регистър CS.

POPA/POPAD

POPA/POPAD

61

Възстановяване на регистрите с общо предназначение от стека.

Действие: инструкция POPA/POPAD възстановява съдържанието на всички регистри с общо предназначение (DI/EDI, SI/ESI, BP/EBP, SP/ESP, BX/EBX, DX/EDX, CX/ECX, AX/EAX) от стека, след което стойността на указателя на стека SP/ESP се увеличава с 16(32). Стойността на DI/EDI се възстановява първа. Стойността на SP/ESP не се възстановява. От установения размер на операнда зависи какви именно регистри - 16- или 32-битови – ще се възстановяват от стека.

POPF/POPPD

POPF/POPPD

9D

Възстановяване на флаговия регистър от стека

Действие: инструкцията POPF/POPPD възстановява от стека съдържанието на флаговия регистър FLAGS/EFLAGS, след което увеличава стойността на указателя на стека SP/ESP с 2(4). Размерността зависи от установения размер на операнда.

Флагове: променят се всички флагове и флаг VM.

PUSH

PUSH ИЗТОЧНИК

FF/6 PUSH r/m16
FF/6 PUSH r/m32
50 + rw PUSH r16
50 + rd PUSH r32
6A PUSH imm8
68 PUSH imm16/imm32
0E PUSH CS
16 PUSH SS
1E PUSH DS
06 PUSH ES
0F AO PUSH FS
0F A8 PUSH GS

Въвеждане на операнда в стека.

Действие: инструкцията въвежда стойността на ИЗТОЧНИК във върха на стека и намалява стойността указателя на стека SP/ESP с 2(4).

PUSHA/PUSHAD

PUSHA/PUSHAD

60

Записване на всички регистри с общо предназначение в стека.

Действие: инструкцията записва в стека регистрите с общо предназначение AX/EAX, CX/ECX, DX/EDX, BX/EBX, SP/ESP, BP/EBP, SI/ESI, DI/EDI (стойността на DI/EDI ще бъде на върха на стека) и намалява стойността на указателя на стека ESP/SP с 16(32) (в зависимост от атрибута за размера адреса – use16 или use32). В

стека се запомня стойността на SP/ESP, преди изпълнението на инструкцията.

PUSHF/PUSHFD

PUSHF/PUSHFD

9C

Записване на флаговете в стека.

Действие: намалява се стойността на указателя на стека SP/ESP с 2(4), след това на върха на стека се прехвърля съдържанието на регистър FLAGS/EFLAGS. Флаговете VM и RF (битове 16 и 17) не се записват, вместо това в образа на EFLAGS, съхранен в стека те се установяват равни на нула.

REP/REPE/REPZ/REPNE/REPNZ

REP/REPE/REPZ/REPNE/REPNZ
REP/REPE/REPZ – f3
REPNE/REPZ –f2

Повторение на низови операции

Действие: инструкциите REP/REPE/REPZ/REPNE/REPNZ извикват повторение на низови операции толкова пъти колкото е посочено в регистър ECX, или докато не се изпълни зависещо от флаг ZF условие. Префикс REP може да бъде добавен пред инструкции INS, OUTS, MOVS, LODS и STOS. Префикси REPE, REPNE, REPZ и REPNZ могат да бъдат добавени пред инструкции CMPS и SCAS. Префикси REPZ и REPNZ са синоними на префикси REPE и REPNE. Поведението на префикса REP не е определено при използването му с инструкции, различни от низовите.

Валидни са следните условия за завършване на повторенията на низовите инструкции при използване на определени префикси:

Табл.8.3 Видове префикси за повторение

Префикс за повторение	Условие за завършване №1	Условие за завършване №2
REP	ECX = 0	няма
REPE/REPZ	ECX = 0	ZF = 0
REPNE/REPZ	ECX = 0	ZF = 1

За изясняване на това, кое условие е довело до завършване на изпълнението на низова команда може да се анализира стойността в регистър ECX (инструкция JECXZ) или да се провери флаг ZF (инструкции JNZ или JNE).

Флагове: не се изменят (с изключение на случаите с използване на префикси с инструкции CMPS и SCAS, които установяват флагове в регистър EFLAGS).

RET

RET
RET число

СЗ (ret – близко връщане в извикваща процедура);
СВ (ret – далечно връщане в извикващата процедура);
С2 i16 (ret i16 – близко връщане с изваждане на i16 байта от стека);
СА i16 (ret i16 – далечно връщане с изваждане на i16 байта от стека)

Близко (далечно) връщане (преход) от(към) процедура.

Действие: предаване на управлението към адрес, разположен на върха на стека. Този адрес обикновено се поставя в стека от команда CALL. Незадължителният операнд определя количеството байтове, които ще бъдат извадени от стека след изваждане на адреса за връщане. При близко връщане извикващата процедура е в границите на текущия сегмент (вътрешносегментен преход). От върха на стека се извиква обратно значението на EIP. Регистър CS не се изменя. При далечно връщане извикващата процедура е

разположена в друг сегмент (междусегментен преход). От върха на стека се извеждат значенията на регистри EIP и CS. При далечно връщане в защитен режим се запазват регистри DS, ES, FS и GS, ако са свързани със сегменти. При връщане трябва да се укаже число за броя на регистрите, които ще се възстановят.

STOS/STOSB/STOSW/STOSD

STOS низ_приемник
STOSB/STOSW/STOSD

AA STOS ES:(E)DI
AA STOSB
AB STOS ES:(E)DI
AB STOSW/STOSD

Запис на байт (дума, двойна дума) в низ.

Действие: командата записва елемент от регистър AL/AX/EAX в низ_приемник адресиран от ES:DI/EDI. След записа на елемента регистър DI/EDI изменя стойността си в зависимост от флаг DF. Ако DF = 0, DI/EDI се увеличава с 1/2 при байтове/думи или EDI с 4 при двойни думи. Ако DF = 1, DI/EDI се намалява с 1/2 при байтове/думи или EDI с 4 при двойни думи. При използване на STOS инструкцията изисква низ_приемник, за да установи типа на низа. Името низ_приемник може да отпадне, ако се използват инструкции STOSB, STOSW или STOSD.

XOR

XOR цел, източник

Изключващо ИЛИ (eXclusive OR)

Източникът и целта се свързват побитово чрез логическата операция „изключващо ИЛИ”. Резултатът се записва в операнд цел.

Литература

- [1] Абелъ, П., *Асемблер и програмиране для IBM PC*, Технологически институт, Британская Колумбия, Издателство: Век +, Энтроп, НТИ, 2003 г., с. 736.
- [2] Асенова, П., *Справочник за MS-DOS*, Издателство “ТИЛИА”, София, 1995, с. 79.
- [3] Бадли, Д., *Програмиране на асемблер за персонален компютър IBM/PC*, ДИ „Техника”, София, 1989, 311 с.
- [4] Зубков, С., *Assembler для DOS, Windows и UNIX*, Москва: ДМК Пресс, 2000., с. 608.
- [5] Иванович, Ю., *Assembler: учебны курс. Справочная система по языку асемблера IMB PC*, Издателство “Питер”, 1998.
- [6] Крупник, А., *Изучаем асемблер*, - СПб.: Питер, 2005, 249 с.
- [7] Линк, В., *Програмиране на АСЕМБЛЕР*, Издателство Техника, София, 2002, 264 с.
- [8] Любенов, В., *Кратко ръководство за DOS за начинаещи незрящи потребители*, Второ актуализирано издание., 2004, <http://horizonti.bg>.
- [9] Магда, Ю. С., *Асемблер для процессоров Intel Pentium*, - СПб.: Питер, 2006, 410 с.
- [10] Нортън, П., *IBM/PC – поглед отвътре*, ДИ „Техника”, София, 1989, 238 с.
- [11] Орлов, С. и колектив, *Справочник по системе программирования Турбо Асемблер*, <http://www.codenet.ru>.
- [12] Пирогов, В.Ю., *ASSEMBLER. Учебный курс*, - М.: Издатель Молгачева С.В., Издателство Нолидж, 2001, 848 с.
- [13] Станев, С., *Компютърни архитектури, Свитък от лекции*, ШУ, 1998.
- [14] Станев, С., П. Смит, Ф. Иванов, *Компютърни системи и мрежи*, УИ „Еп. К. Преславски”, 2002, 199 с.
- [15] Юров, В.И., *Assembler*, Учебник для вузов. 2-е изд., - СПб.: Питер, 2003, 637 с.
- [16] Brouwer, A., *Keyboard scancodes*, v1.2e, 2004, <http://www.win.tue.nl/~aeb/linux/kbd/scancodes.html>.

- [17] Estey, G., *A Introduction to x86 Assembly*, 0.9 Beta Version, 1995.
- [18] Hyde, R., *The Art of Assembly Language*, 2001, p. 1530, <http://webster.cs.ucr.edu/AoA/index.html>.
- [19] Joehanes, R., *PC Assembly Tutorial*, 2000, <http://www.geocities.com/codeteacher/>.
- [20] Microsoft колектив, <http://www.microsoft.com>.
- [21] Microsoft колектив, *Help and Support Center*, Windows XP.
- [22] Nisoft колектив, *Компютърна енциклопедия*, Първа част, Издателство Nisoft, 1993, 386 с.
- [23] Nisoft колектив, *Компютърна енциклопедия*, Втора част, Издателство Nisoft, 1993, 353 с.
- [24] Nisoft колектив, *Компютърна енциклопедия*, Трета част, Издателство Nisoft, 1993, 361 с.
- [25] Opferman, T., *Assembly Tutorial*, 2005, <http://www.opferman.com>.
- [26] <http://bg.wikipedia.org/wiki/Асемблер>
- [27] http://bg.wikipedia.org/wiki/Машинен_език

доц. д-р Борислав Панайотов Стоянов

**РЪКОВОДСТВО ЗА УПРАЖНЕНИЯ ПО
ПРОГРАМИРАНЕ НА АСЕМБЛЕР.
ПЪРВА ЧАСТ**

Националност българска
Първо издание

ISBN 978-954-9775-37-2

Тираж 100